

**PENGEMBANGAN GRAPHRAG FRAMEWORK
BERBASIS PENGETAHUAN KOMUNITAS STACK
OVERFLOW UNTUK MEREDUKSI HALUSINASI
PADA LLM**

TESIS

Oleh

Dartika Anie Marian
23524319



**PROGRAM STUDI MAGISTER INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Juni 2026

LEMBAR PENGESAHAN

Pengembangan GraphRAG Framework Berbasis Pengetahuan Komunitas Stack Overflow untuk Mereduksi Halusinasi pada LLM

TESIS

Oleh

Dartika Anie Marian
23524319

Program Studi Magister Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Tesis ini telah disetujui dan disahkan
di Bandung, pada tanggal 1 Juni 2026

Pembimbing

Dr.Ir. Arry Akhmad Arman, MT.

NIP. 196504141991021001

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tesis ini adalah hasil karya saya sendiri yang dibuat dengan sebenar-benarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan tesis ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan tesis.
3. Isi tesis ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa dokumen tesis ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 1 Juni 2026

Dartika Anie Marian

NIM: 23534219

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan tesis ini, saya telah menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu. Berikut adalah rincian penggunaan AI dalam penulisan tesis ini.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Grammarly	Tinggi	Semua bab
2	Pembuatan teks	Microsoft Copilot	Sedang	Bab II dan Subbab III.2 hal. 34
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	...	...	...
5	Penyusunan bahan diskusi	...	...	...
6	Penyusunan kode program	...	...	...
7	Penyusunan formula atau perhitungan matematis	...	...	...
8	Penyusunan konsep desain atau algoritma	...	...	...
9	Penyusunan analisis data	...	...	...
10	Penyusunan kesimpulan atau rekomendasi	...	...	...

Bandung, 1 Juni 2026

Dartika Anie Marian

NIM: 23524319

ABSTRAK

Pengembangan GraphRAG Framework Berbasis Pengetahuan Komunitas Stack Overflow untuk Mereduksi Halusinasi pada LLM

oleh

Dartika Anie Marian

23524319

Kata kunci:

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan Tesis yang berjudul "Pemodelan Machine Learning untuk Prioritisasi dan Estimasi Kebutuhan Fasilitas Kesehatan Kota Bandung". Tesis ini disusun sebagai salah satu syarat untuk menyelesaikan program Magister Informatika di Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Bandung, 20 Mei 2026

Penulis

Dartika Anie Marian

DAFTAR ISI

LEMBAR PENGESAHAN	iii
PERNYATAAN ORISINALITAS	v
PERNYATAAN PENGGUNAAN AI	vii
ABSTRAK	ix
KATA PENGANTAR	xi
DAFTAR ISI	xiii
DAFTAR GAMBAR	xvii
DAFTAR TABEL	xix
DAFTAR SINGKATAN	xxi
BAB I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	5
I.3 Tujuan	6
I.4 Manfaat Penelitian	6
I.5 Ruang Lingkup dan Batasan Masalah	7
I.6 Hipotesis	8
I.7 Metodologi Penelitian	9
I.8 Sistematika Penulisan	10
BAB II STUDI LITERATUR	13
II.1 Halusinasi pada <i>Large Language Model</i>	13
II.1.1 Definisi dan Taksonomi Halusinasi	13
II.1.2 Akar Penyebab Halusinasi	14
II.1.3 Dampak Halusinasi pada Domain Pemrograman	15
II.2 <i>Large Language Model</i> : Pengetahuan Parametrik dan Keterbatasannya	16
II.2.1 Sifat Parametrik LLM dan Mekanisme Generasi	16
II.2.2 Keterbatasan Pengetahuan Parametrik	17
II.3 <i>Retrieval-Augmented Generation (RAG)</i>	17
II.3.1 Konsep Dasar dan Motivasi	17
II.3.2 Komponen Utama RAG	18
II.3.2.1 Pengindeksan Basis Pengetahuan (<i>Indexing</i>).	20
II.3.2.2 Pengambilan Informasi (<i>Retrieval</i>).	20
II.3.2.3 <i>Fusion(Pre-filtering & Reranking)</i>	21
II.3.2.4 Augmentasi Konteks dan Generasi respons (<i>Augmentation & Generation</i>).	22
II.3.3 Penerapan RAG pada Domain Pemrograman	22
II.3.3.1 Peningkatan kualitas kode (<i>code generation</i> dan <i>refactoring</i>).	22
II.3.3.2 Deteksi anomali <i>log</i> sistem (<i>AIOps</i>).	23
II.3.3.3 Pembuatan kode SQL dan integrasi basis data domain-spesifik.	23

	II.3.3.4 Adopsi RAG pada pengembangan perangkat lunak industri.	23
	II.3.4 Keterbatasan RAG Konvensional dalam Domain Pemrograman	23
II.4	<i>Knowledge Graph</i>	24
II.4.1	Definisi dan Representasi Formal	24
II.4.2	Konstruksi <i>Knowledge Graph</i> dari Teks Tidak Terstruktur	25
II.4.3	Keunggulan Representasi Berbasis Graf dan Integrasi dengan LLM	27
II.5	<i>Graph-based Retrieval-Augmented Generation</i>	27
II.5.1	Motivasi dan Definisi	27
II.5.2	Arsitektur <i>Pipeline</i> GraphRAG	28
	II.5.2.1 Konstruksi <i>Knowledge Graph</i>	28
	II.5.2.2 <i>Retrieval</i> dan Penalaran Berbasis Graf.	28
	II.5.2.3 Integrasi Konteks Berbasis Struktur.	29
	II.5.2.4 Perbandingan Kuantitatif: Vector RAG dan GraphRAG	29
II.5.3	Hybrid Retrieval pada GraphRAG: Penalaran Simbolik dan Semantik	30
	II.5.3.1 Semantic Retrieval: Keunggulan dan Keterbatasan	31
	II.5.3.2 Symbolic Retrieval: Keunggulan dan Keterbatasan	31
	II.5.3.3 Multi-hop Reasoning sebagai Justifikasi Pendekatan Hibrid	31
	II.5.3.4 Trade-off antara Presisi Retrieval dan Cakupan Kontekstual	32
II.5.4	Hasil Penelitian tentang Efektivitas GraphRAG	33
	II.5.4.1 Pengurangan Halusinasi melalui Grounded Generation	33
	II.5.4.2 Kemampuan <i>Multi-hop Reasoning</i> pada Pertanyaan Relasional	33
	II.5.4.3 Knowledge Graph sebagai Sumber Pengetahuan Otoritatif	34
II.5.5	Aplikasi GraphRAG pada Domain Pemrograman dan Rekayasa Perangkat Lunak	34
	II.5.5.1 Penyelesaian Kode Berbasis Graf Dependensi	34
	II.5.5.2 Generasi SQL dan Penalaran Relasional	35
	II.5.5.3 Analisis Lisensi sebagai Representasi Domain Teknis	35
	II.5.5.4 Implikasi terhadap Penelitian Berbasis Stack Overflow	35
II.5.6	Sintesis dan Posisi Penelitian	36
II.6	Stack Overflow sebagai Basis Pengetahuan Tervalidasi Sosial	36
II.6.1	Struktur dan Mekanisme Kurasi Komunitas	36
II.6.2	Sinyal Kepercayaan sebagai Proksi Kualitas	36
II.6.3	Pemanfaatan Konten Stack Overflow dalam Sistem AI	37

II.6.4	<i>Stack Overflow Recommendations Dataset (SORD)</i> sebagai Sumber Data Penelitian	38
II.7	<i>Grounded Generation</i> dan <i>Faithfulness</i> pada LLM	39
II.7.1	Definisi dan Motivasi	39
II.7.2	<i>Provenance</i> sebagai Mekanisme Keterlacakan	40
II.7.3	<i>Dual-Constraint</i> sebagai Mekanisme <i>Grounded Generation</i>	41
II.8	Evaluasi Keandalan <i>Large Language Model</i>	41
II.8.1	Tantangan Evaluasi pada Sistem GraphRAG	41
II.8.2	Dimensi Evaluasi pada Sistem GraphRAG	42
II.8.3	Kerangka RAGAS sebagai Protokol Evaluasi	43
II.8.4	Paradigma <i>LLM-as-a-Judge</i>	44
II.9	Sintesis Penelitian Terkait dan <i>Research Gap</i>	45
II.9.1	Analisis Penelitian Terkait	45
II.9.2	Identifikasi Kesenjangan	47
II.9.3	Kerangka Konseptual Penelitian	48
BAB III	Analisis dan Desain	53
III.1	Analisis Kondisi Saat Ini	53
III.1.1	Model Konseptual Sistem yang Ada	53
III.1.2	Analisis Keterbatasan Sistem Saat Ini	54
III.1.3	Identifikasi Masalah	55
III.2	Analisis Kebutuhan	56
III.2.1	Identifikasi Kebutuhan Pengguna	56
III.2.2	Kebutuhan Fungsional	58
III.2.3	Kebutuhan Nonfungsional	59
III.3	Analisis Pemilihan Solusi	60
III.3.1	Alternatif Solusi	60
III.3.2	Analisis Penentuan Solusi	61
III.4	Desain Konseptual <i>GraphRAG Framework</i>	61
III.4.1	Gambaran Umum Arsitektur	61
III.4.2	Desain <i>Knowledge Graph</i> Berbobot dari SORD	64
III.4.2.1	Tipe Simpul	65
III.4.2.2	Tipe Sisi dan Skema Bobot Kepercayaan	65
III.4.2.3	Proses Konstruksi KG	67
III.4.3	Desain Mekanisme <i>Hybrid Retrieval</i>	69
III.4.3.1	<i>Entity Anchoring</i>	69
III.4.3.2	<i>Graph Traversal</i>	69
III.4.3.3	<i>Semantic Expansion</i>	70
III.4.3.4	<i>Fusi Konteks</i> Terverifikasi	70
III.4.4	Desain <i>Grounded Generation</i>	72
III.4.4.1	Mekanisme <i>Dual-Constraint</i>	72
III.4.4.2	<i>Provenance</i> sebagai Mekanisme Keterlacakan	73
III.4.4.3	Output Sistem	73
III.4.5	Desain Evaluasi	73
III.5	Ringkasan Desain	75

DAFTAR GAMBAR

II.1	Contoh kegagalan penalaran akibat ketidaksesuaian waktu informasi dan penggunaan data yang mengandung informasi keliru (Chang dkk. 2026)	15
II.2	<i>Inference Pipeline</i> RAG modular berdasarkan taksonomi (Vizniuk dkk. 2024).	19
II.3	Taksonomi RAG yang terdiri atas tahap <i>Indexing</i> , <i>Retrieval</i> , <i>Fusion</i> , dan <i>Generation</i> (Kamalipour, Asadi, dan Chimeh 2026)	20
II.4	Proses pengindeksan pada pipeline RAG: mulai dari pemuatan data (<i>data loading</i>), pemotongan teks (<i>chunking</i>), pembentukan embedding, hingga penyimpanan dalam basis data vektor (<i>vector database</i>). (Kamalipour, Asadi, dan Chimeh 2026)	21
II.5	Gambaran umum <i>pipeline</i> Graph-RAG: ekstraksi triple, penelusuran subgraf (<i>sub-graph traversal</i>), serta integrasi hibrida antara representasi vektor dan simbolik.(Kamalipour, Asadi, dan Chimeh 2026)	29
II.6	Gambaran umum rancangan empiris yang digunakan untuk menguji korelasi antara sinyal kepercayaan komunitas Stack Overflow dengan akurasi jawaban LLM. (Silva, Samhi, dan Khomh 2025)	37
II.7	Distribusi domain pertanyaan untuk jawaban LLM dengan kemiripan tinggi dan rendah terhadap komunitas Stack Overflow. (Silva, Samhi, dan Khomh 2025)	38
II.8	Tahapan evaluasi <i>LLM-as-a-Judge</i> yang meliputi pemberian konteks, inferensi model penilai, <i>post-processing</i> , dan agregasi skor (Gu, Jiang, Shi, dkk. 2026).	44
II.9	Kerangka konseptual penelitian	49
III.1	Model konseptual sistem tanya-jawab berbasis LLM generik tanpa mekanisme pengambilan konteks dari sumber terverifikasi.	54
III.2	Arsitektur <i>GraphRAG Framework</i> yang terdiri atas tiga blok utama (Konstruksi KG Berbobot, <i>Hybrid Retrieval</i> , dan <i>Grounded Generation</i>) beserta blok evaluasi komparatif.	63

III.3	Contoh subgraf <i>knowledge graph</i> berbobot dari SORD pada domain autentikasi FastAPI	67
III.4	Flowchart konstruksi <i>knowledge graph</i> berbobot dari dataset SORD.	68
III.5	Alur mekanisme <i>hybrid retrieval</i> pada <i>GraphRAG Framework</i>	71

DAFTAR TABEL

II.1	Perbandingan <i>trade-off</i> antara Vector RAG dan GraphRAG berdasarkan dimensi teknis yang relevan untuk pengembangan <i>framework</i> . Bersumber dari Kamalipour, Asadi, dan Chimeh (2026), Tabel 8. . .	30
II.2	Sinyal kepercayaan komunitas Stack Overflow dan interpretasinya sebagai bobot pada <i>knowledge graph</i> berbobot	37
II.3	Metrik RAGAS dalam evaluasi GraphRAG berbasis Stack Overflow	43
II.4	Posisi penelitian terkait terhadap tiga dimensi utama penelitian ini	46
III.1	Temuan empiris Silva, Samhi, dan Khomh (2025) yang menjadi titik acuan pengukuran <i>improvement</i> penelitian ini.	55
III.2	Pemetaan masalah pengguna ke kebutuhan solusi <i>GraphRAG Framework</i>	57
III.3	Kebutuhan nonfungsional <i>GraphRAG Framework</i>	60
III.4	Perbandingan ketiga kondisi evaluasi terhadap kebutuhan sistem dan tujuan penelitian.	62
III.5	Perbandingan pemenuhan kebutuhan sistem pada tiga kondisi penelitian.	62
III.6	Operasionalisasi sinyal kepercayaan Stack Overflow sebagai bobot sisi pada <i>knowledge graph</i>	66
III.7	Metrik evaluasi komparatif <i>GraphRAG Framework</i>	74
III.8	Ringkasan komponen <i>GraphRAG Framework</i> dan kaitannya dengan kesenjangan penelitian	76

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
AI	<i>Artificial Intelligence</i>	1
CNN	<i>Convolutional Neural Network</i>	2
GPU	<i>Graphics Processing Unit</i>	14

BAB I

PENDAHULUAN

I.1 Latar Belakang

Perkembangan *Large Language Model* (LLM) dalam beberapa tahun terakhir telah mendorong transformasi signifikan pada berbagai aplikasi kecerdasan buatan, termasuk natural language understanding, text generation, conversational agents, machine translation, serta code generation (kumar2024; qin2025). Model-model seperti GPT-4 dan LLaMA-2 telah menunjukkan kemampuan luar biasa dalam memahami dan menghasilkan teks natural yang menyerupai tulisan manusia. Haque (2025) menegaskan bahwa LLM bukan hanya mengubah cara perangkat lunak dikembangkan, tetapi secara fundamental mendefinisikan ulang peran pengembang dalam siklus hidup rekayasa perangkat lunak (*software engineering*). Adopsi LLM di industri perangkat lunak juga meningkat secara signifikan, sebagaimana dikonfirmasi oleh Ronanki dkk. (2026) yang melakukan studi multi-kasus pada tiga organisasi yang menggunakan LLM dalam aktivitas rekayasa perangkat lunak dan mengidentifikasi perlunya praktik terbaik yang dapat ditindaklanjuti untuk adopsi LLM yang efisien dan bertanggung jawab. Meskipun demikian, di balik kemampuan generatif tersebut, terdapat permasalahan mendasar yang belum sepenuhnya terpecahkan yaitu kecenderungan LLM untuk menghasilkan informasi yang tampak meyakinkan namun faktanya tidak akurat atau bahkan sepenuhnya merupakan fabrikasi (Ji dkk. 2023).

Fenomena ini dikenal sebagai *hallucination* (halusinasi) dan menjadi salah satu hambatan paling kritis dalam penerapan LLM pada domain yang membutuhkan keakuratan tinggi. Ji dkk. (2023) mendefinisikan halusinasi sebagai kondisi di mana model menghasilkan keluaran yang tidak sesuai dengan fakta yang dapat diverifikasi, tidak konsisten dengan konteks yang diberikan, atau tidak memiliki dasar empiris yang dapat ditelusuri. Huang dkk. (2025) dalam survei komprehensifnya di *ACM Transactions on Information Systems* mengidentifikasi tiga lapisan akar penyebab halusinasi yang saling terkait: (1) kualitas dan kelengkapan data pelatihan yang tidak merata, (2) keterbatasan proses penyelarasan (*alignment*) model terhadap pengetahuan faktual, dan (3) mekanisme inferensi yang cenderung menghasilkan teks yang terdengar koheren tanpa jaminan akurasi faktual. Chang dkk. (2026) melengkapi analisis ini dengan mengidentifikasi tiga tantangan fundamental dalam generasi teks berbasis LLM yang memperkuat kecenderungan halusinasi: (1) pemahaman instruk-

si yang tidak memadai pada konteks panjang dan percakapan multi-gilir, (2) penalaran intensi yang tidak konsisten, khususnya ketika instruksi mengandung informasi ambigu, dan (3) generasi dialog yang tidak andal (*unreliable dialogue generation*) yang menghasilkan konten tidak stabil dari prompt yang serupa.

Dalam konteks pemrograman dan rekayasa perangkat lunak, halusinasi LLM bermanifestasi dalam berbagai bentuk yang berpotensi menimbulkan dampak langsung terhadap kualitas sistem. Alomari dkk. (2026) dalam tinjauan literatur sistematis terhadap 49 studi penggunaan LLM untuk peningkatan kualitas kode menemukan bahwa kode yang dihasilkan atau direfaktorisasi oleh LLM tidak dapat diandalkan dan masih menghadapi tantangan besar dalam hal optimasi dan keandalan. Sebuah temuan yang mengonfirmasi bahwa pengetahuan parametrik LLM saja tidak cukup untuk menjamin keakuratan di domain teknis yang terus berkembang. Silva, Samhi, dan Khomh (2025) memperkuat bukti ini melalui studi empiris yang membandingkan jawaban ChatGPT dan LLaMA terhadap pertanyaan Stack Overflow. Hasil penelitian Silva, Samhi, dan Khomh (2025) menunjukkan bahwa meskipun LLM menantang keahlian manusia, LLM tidak melampaui keahlian komunitas manusia untuk sejumlah domain tertentu. Yang lebih signifikan, Silva, Samhi, dan Khomh (2025) mendokumentasikan penurunan nyata aktivitas *posting* pengguna di Stack Overflow sejak hadirnya ChatGPT. Pengguna mulai mengandalkan LLM meskipun keandalannya belum terjamin penuh, sehingga berpotensi memperparah penyebaran informasi teknis yang tidak akurat.

Stack Overflow yang merupakan platform utama pengembang untuk pertanyaan seputar pemrograman dan pengembangan perangkat lunak, kini berada di persimpangan antara ancaman dan peluang akibat kehadiran LLM. Di satu sisi, aktivitas penggunaannya menurun (Silva, Samhi, dan Khomh 2025). Di sisi lain, komunitas Stack Overflow menyimpan lebih dari 58 juta pasang pertanyaan dan jawaban yang telah melewati mekanisme kurasi ketat berupa sistem *upvote/downvote*, penanda jawaban yang diterima (*accepted answer*), dan sistem reputasi kontributor (Kabir dkk. 2024). Sinyal kepercayaan komunitas ini menjadikan Stack Overflow bukan sekadar repositori teks, melainkan basis pengetahuan tervalidasi secara sosial yang potensial digunakan untuk memberikan landasan pengetahuan bagi jawaban LLM berdasarkan fakta yang telah diverifikasi oleh praktisi domain. Nashid, Sintaha, dan Mesbah (2023) menunjukkan bahwa pemanfaatan konten Stack Overflow sebagai sumber pengambilan konteks secara signifikan meningkatkan kualitas jawaban LLM pada tugas-tugas terkait kode, menggarisbawahi potensi besar repositori ini.

Salah satu pendekatan yang paling menjanjikan untuk memitigasi halusinasi adalah *Retrieval-Augmented Generation* (RAG) (Lewis dkk. 2020). Huang dan Huang (2026) dalam survei mereka di *ACM Computing Surveys* menjelaskan bahwa RAG menggabungkan teknik pengambilan informasi (*information retrieval*) dengan kemampuan generatif LLM untuk mengatasi keterbatasan pengetahuan statis model, sehingga memungkinkan integrasi informasi eksternal yang dinamis dan terkini pada saat inferensi. Cruz Cabello, Sales, dan Machado (2025) melalui tinjauan literatur sistematis pada domain deteksi anomali *log* sistem komputer menemukan bahwa RAG secara substansial meningkatkan akurasi dan interpretabilitas jawaban LLM tanpa memerlukan *fine-tuning* yang ekstensif, dan bahwa integrasi RAG dengan LLM menunjukkan kemampuan adaptasi yang kuat terhadap lingkungan operasional yang berubah. Temuan ini mengkonfirmasi potensi RAG sebagai strategi mitigasi halusinasi yang praktis dan skalabel di berbagai domain teknis.

Meskipun demikian, implementasi RAG konvensional yang menggunakan pencarian berbasis kemiripan vektor (*dense retrieval*) terhadap potongan-potongan teks yang tidak terstruktur memiliki keterbatasan mendasar yaitu pendekatan ini gagal memanfaatkan relasi semantik kompleks dan hubungan kontekstual antar konsep yang justru sangat penting dalam domain pemrograman (Huang dan Huang 2026). Sebagai respons atas keterbatasan ini, pendekatan *Graph-based Retrieval-Augmented Generation* (GraphRAG) diusulkan untuk mengintegrasikan struktur *knowledge graph* ke dalam proses pengambilan konteks. Zhu dkk. (2026) dalam survei komprehensif mereka di *ACM Computing Surveys* mengidentifikasi tiga inovasi kunci GraphRAG dibandingkan RAG konvensional: (1) representasi pengetahuan berbasis graf yang secara eksplisit mengkodekan relasi antar entitas dan hierarki domain, (2) teknik pengambilan berbasis graf yang memungkinkan pengambilan konteks dengan kemampuan penalaran multi-hop, dan (3) algoritma integrasi pengetahuan yang sadar-struktur untuk menghasilkan teks yang lebih akurat dan koheren. Pan dkk. (2024) dalam menegaskan bahwa penggabungan LLM dengan *knowledge graph* membentuk paradigma baru yang secara signifikan meningkatkan kemampuan penalaran berbasis pengetahuan dan mengurangi fabrikasi faktual (*factual hallucination*).

Dari sisi evaluasi, Gu, Jiang, Shi, dkk. (2026) dalam survei komprehensif mereka tentang *LLM-as-a-Judge* menegaskan bahwa mengukur keandalan LLM merupakan tantangan signifikan yang membutuhkan desain dan standarisasi yang cermat. Mereka mengusulkan strategi untuk meningkatkan konsistensi evaluasi dan memitigasi bias, sebuah kerangka yang relevan untuk mengukur efektivitas reduksi halusinasi

pada sistem berbasis GraphRAG dan menjadi prasyarat agar kontribusi sebuah *framework* mitigasi halusinasi dapat diklaim secara akademis.

Meskipun kemajuan di masing-masing bidang LLM, RAG, GraphRAG, dan penambahan pengetahuan komunitas cukup pesat, terdapat kesenjangan metodologis yang signifikan ketika keempat komponen ini dipertimbangkan secara terpadu. Pertama, penelitian yang secara langsung membandingkan LLM dengan Stack Overflow seperti Silva, Samhi, dan Khomh (2025) dan Kabir dkk. (2024), bersifat evaluatif. Mereka mengukur dan mendokumentasikan kesenjangan kualitas antara LLM dan keahlian komunitas, tetapi tidak mengusulkan mekanisme arsitektural untuk menjembatani kesenjangan tersebut. Kedua, penelitian yang menggunakan konten Stack Overflow sebagai sumber konteks untuk LLM, seperti Nashid, Sintaha, dan Mesbah (2023) hanya menerapkan RAG konvensional berbasis kemiripan vektor dan tidak memanfaatkan struktur relasional pengetahuan komunitas maupun sinyal kepercayaan komunitas (*upvote*, *accepted answer*) sebagai bobot kepercayaan dalam proses pengambilan konteks. Ketiga, meskipun pendekatan GraphRAG terbukti superior dibandingkan RAG konvensional pada berbagai domain (Zhu dkk. 2026), serta integrasi *knowledge graph* dengan LLM diyakini sebagai paradigma yang menjanjikan (Pan dkk. 2024), belum ada penelitian yang secara khusus membangun *knowledge graph* dari konten Stack Overflow yang mengkodekan sinyal kepercayaan komunitas untuk digunakan sebagai fondasi GraphRAG pada domain pemrograman. Sementara itu, tantangan halusinasi akibat keterbatasan pemahaman instruksi dan generasi dialog yang tidak andal (Chang dkk. 2026; Huang dkk. 2025) tetap menjadi masalah terbuka yang membutuhkan pendekatan grounding berbasis pengetahuan terverifikasi.

Kesenjangan-kesenjangan ini secara kolektif membuka peluang kontribusi yang signifikan: pengembangan sebuah *GraphRAG Framework* yang secara sistematis (1) membangun *knowledge graph* dari konten Stack Overflow yang mengkodekan entitas teknis, relasi semantik, dan sinyal kepercayaan komunitas sebagai bobot kepercayaan eksplisit; (2) merancang mekanisme *graph-guided retrieval* yang memanfaatkan struktur relasional tersebut untuk mengidentifikasi konteks yang lebih relevan dibandingkan RAG konvensional; (3) mengintegrasikan konteks hasil traversal ke dalam proses inferensi LLM; dan (4) mengukur efektivitas reduksi halusinasi secara kuantitatif menggunakan protokol evaluasi yang terstandarisasi.

I.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, kondisi ideal yang diharapkan adalah tersedianya mekanisme *grounding* berbasis pengetahuan komunitas yang tervalidasi secara sosial sehingga LLM dapat menghasilkan jawaban teknis yang akurat dan dapat dipertanggungjawabkan. Namun, kondisi aktual yang dijumpai menunjukkan kesenjangan metodologis yang signifikan: penelitian yang menggunakan konten Stack Overflow sebagai konteks LLM masih menggunakan RAG konvensional dan tidak memanfaatkan struktur relasional pengetahuan komunitas (Nashid, Sintaha, dan Mesbah 2023); pendekatan GraphRAG yang ada belum mengeksplorasi sumber pengetahuan bervalidasi sosial maupun domain pemrograman secara spesifik (Zhu dkk. 2026; Pan dkk. 2024); sementara itu, tantangan halusinasi dalam generasi teks LLM pada instruksi yang ambigu dan konteks panjang masih belum terpecahkan (Chang dkk. 2026; Huang dkk. 2025). Kesenjangan antara kondisi ideal dan kondisi aktual tersebut menjadi landasan permasalahan utama dalam penelitian ini, yang dirumuskan sebagai berikut.

Bagaimana merancang dan mengembangkan GraphRAG *Framework* berbasis pengetahuan komunitas Stack Overflow yang secara terukur mampu mereduksi halusinasi pada *Large Language Model* di domain pemrograman?

Untuk menjawab pertanyaan utama tersebut, penelitian ini mengajukan pertanyaan turunan sebagai berikut.

1. Bagaimana membangun *knowledge graph* yang merepresentasikan pengetahuan teknis komunitas Stack Overflow secara terstruktur, termasuk entitas (konsep, fungsi, pustaka, *error*), relasi antar entitas, serta sinyal kepercayaan komunitas (*upvote*, *accepted answer*, skor reputasi) sebagai bobot kepercayaan eksplisit pada simpul dan sisi graf?
2. Bagaimana merancang mekanisme *graph-guided retrieval* yang mampu mengidentifikasi subgraf relevan berdasarkan kueri pengguna, sedemikian sehingga konteks yang diekstraksi lebih kaya secara semantik dan relasional dibandingkan pengambilan berbasis kemiripan vektor pada RAG konvensional?
3. Bagaimana mengintegrasikan konteks hasil traversal *knowledge graph* Stack Overflow ke dalam proses inferensi LLM sehingga jawaban yang dihasilkan memiliki tingkat akurasi faktual yang lebih tinggi dibandingkan LLM tanpa mekanisme *grounding*?
4. Bagaimana merancang protokol evaluasi yang mampu mengukur dan memvalidasi efektivitas *framework* yang diusulkan dalam mereduksi halusinasi LLM

menggunakan metrik kuantitatif yang terstandarisasi, serta membandingkan kinerja *framework* terhadap dua kondisi *baseline*: LLM murni dan RAG konvensional?

I.3 Tujuan

Tujuan penelitian ini dibagi menjadi tujuan umum dan tujuan khusus. Tujuan umum penelitian ini adalah mengembangkan *GraphRAG Framework* berbasis pengetahuan komunitas Stack Overflow sebagai artefak penelitian desain yang mampu mereduksi halusinasi LLM pada domain pemrograman secara terukur, sekaligus menghasilkan kontribusi pengetahuan berupa prinsip-prinsip desain yang dapat digunakan kembali oleh penelitian serupa di domain teknis lainnya.

Adapun tujuan khusus dari penelitian ini adalah sebagai berikut.

1. Membangun *knowledge graph* dari data Stack Overflow yang merepresentasikan entitas teknis, relasi semantik antar entitas, dan sinyal kepercayaan komunitas sebagai bobot pada simpul dan sisi graf.
2. Merancang dan mengimplementasikan modul *graph-guided retrieval* yang memanfaatkan traversal *knowledge graph* untuk mengekstraksi konteks yang relevan secara semantik dan relasional berdasarkan kueri pengguna, melampaui keterbatasan RAG konvensional berbasis kemiripan vektor.
3. Mengembangkan modul *grounded generation* yang mengintegrasikan konteks hasil traversal graf ke dalam mekanisme *prompting* LLM untuk menghasilkan jawaban teknis yang memiliki *grounding* faktual lebih kuat dibandingkan LLM tanpa mekanisme pengambilan konteks eksternal.
4. Melakukan evaluasi untuk mengukur tingkat reduksi halusinasi *framework* yang diusulkan menggunakan metrik kuantitatif, serta membandingkan hasilnya terhadap dua *baseline* yaitu LLM murni dan RAG konvensional.

I.4 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat pada dua dimensi yang saling melengkapi, yaitu manfaat teoritis dan manfaat praktis.

Manfaat Teoritis

1. **Model arsitektur GraphRAG untuk domain *question answering*.** *Framework* yang diusulkan memberikan kontribusi berupa model arsitektur GraphRAG yang secara eksplisit mempertimbangkan validasi sosial komunitas sebagai dimensi kepercayaan dalam proses pengambilan konteks. Model ini memperkaya literatur akademik pada persimpangan antara *information retrieval*, *knowledge graph*, dan *large language model*.

2. **Kerangka evaluasi halusinasi yang terstandarisasi.** Protokol evaluasi yang dirancang dalam penelitian ini mencakup metrik akurasi faktual, konsistensi, dan tingkat halusinasi dalam tiga kondisi perbandingan, dapat menjadi referensi metodologis bagi penelitian mitigasi halusinasi LLM di berbagai domain aplikasi.

Manfaat Praktis

1. **Peningkatan keandalan bantuan pemrograman berbasis LLM.** *GraphRAG Framework* yang dihasilkan dapat diintegrasikan ke dalam alat bantu pengembangan (*developer tools*) berbasis LLM, seperti *AI coding assistant*, untuk menghasilkan saran kode dan penjelasan teknis yang lebih akurat dan dapat ditelusuri sumbernya.
2. **Panduan adopsi LLM yang bertanggung jawab di industri perangkat lunak.** Temuan penelitian ini memberikan bukti empiris dan panduan arsitektural bagi organisasi perangkat lunak yang ingin mengadopsi LLM secara efisien dan bertanggung jawab, khususnya dalam mengelola risiko penyebaran informasi teknis yang tidak akurat.
3. **Pemanfaatan kembali aset pengetahuan komunitas Stack Overflow.** Penelitian ini menunjukkan cara memanfaatkan lebih dari 58 juta pasang pertanyaan dan jawaban Stack Overflow yang telah tervalidasi secara sosial sebagai fondasi pengetahuan yang terstruktur, sehingga aset komunitas ini tidak hanya menjadi repositori pasif tetapi juga menjadi infrastruktur *grounding* aktif untuk sistem AI generatif.

I.5 Ruang Lingkup dan Batasan Masalah

Penelitian ini berfokus pada perancangan dan pengembangan *GraphRAG Framework* untuk domain pemrograman dengan menggunakan data dari komunitas Stack Overflow sebagai sumber pengetahuan utama. Evaluasi dilakukan pada pertanyaan dan jawaban teknis yang mencakup topik-topik pemrograman umum (*general programming*) seperti Python, JavaScript, dan Java, yang merupakan *tag* dengan volume konten tertinggi di Stack Overflow.

Batasan masalah:

1. *Framework* yang dikembangkan difokuskan pada domain pemrograman dan tidak dimaksudkan untuk langsung digeneralisasi ke domain non-teknis tanpa adaptasi pada skema *knowledge graph* dan protokol evaluasinya.
2. Pembangunan *knowledge graph* dibatasi pada ekstraksi entitas dan relasi dari teks pertanyaan, jawaban, dan komentar Stack Overflow, tanpa menyertakan

eksekusi atau validasi kode secara dinamis.

3. Penelitian ini tidak membahas aspek *fine-tuning* LLM. Seluruh eksperimen menggunakan LLM dalam mode *inference* tanpa modifikasi parameter model.
4. Pengukuran halusinasi dibatasi pada dimensi akurasi faktual dan konsistensi informasi, tidak mencakup dimensi bias, toksisitas, atau kesesuaian gaya bahasa.
5. Penelitian ini tidak mencakup aspek skalabilitas dan *deployment framework* ke lingkungan produksi. Seluruh eksperimen dilakukan dalam lingkungan penelitian yang terkontrol.

I.6 Hipotesis

Hipotesis penelitian ini diturunkan dari dua premis yang diidentifikasi pada latar belakang dan rumusan masalah.

1. **Premis 1.** LLM yang beroperasi hanya mengandalkan pengetahuan parametrik menghasilkan tingkat halusinasi yang signifikan pada pertanyaan teknis domain pemrograman, karena pengetahuan yang dikodekan selama pelatihan bersifat statis dan tidak mencerminkan perkembangan teknologi, perubahan API, maupun solusi terkini yang didokumentasikan oleh komunitas praktisi (Jidkk. 2023; Huang dkk. 2025). Hal ini dikonfirmasi oleh Silva, Samhi, dan Khomh (2025) secara empiris dan oleh Chang dkk. (2026) yang mengidentifikasi defisiensi fundamental LLM dalam menghasilkan dialog yang andal. RAG konvensional mengurangi sebagian halusinasi ini, namun masih terbatas karena pengambilan berbasis kemiripan vektor gagal memanfaatkan relasi semantik kompleks antar konsep teknis (Huang dan Huang 2026).
2. **Premis 2.** *Knowledge graph* yang dibangun dari konten Stack Overflow merepresentasikan pengetahuan teknis dalam struktur relasional yang kaya, dan sinyal kepercayaan komunitas (skor *upvote*, status *accepted answer*) merupakan proksi yang valid untuk kualitas dan kebenaran informasi (Silva, Samhi, dan Khomh 2025; Kabir dkk. 2024). Mekanisme *graph-guided retrieval* yang memanfaatkan struktur relasional ini mampu mengidentifikasi konteks yang lebih relevan dan terverifikasi dibandingkan pencarian vektor sederhana, sebagaimana diindikasikan oleh superioritas pendekatan GraphRAG yang dikonfirmasi oleh Zhu dkk. (2026) dan paradigma integrasi LLM-*knowledge graph* yang diuraikan oleh Pan dkk. (2024).

Berdasarkan kedua premis tersebut, hipotesis penelitian ini dirumuskan sebagai berikut: *GraphRAG Framework* yang mengintegrasikan *knowledge graph* berbasis pengetahuan komunitas Stack Overflow dengan mekanisme *graph-guided retrieval* dan sinyal kepercayaan komunitas sebagai bobot menghasilkan reduksi halusinasi

LLM yang secara statistik signifikan dibandingkan dua kondisi *baseline* yaitu LLM murni tanpa mekanisme pengambilan konteks dan LLM dengan RAG konvensional berbasis *dense retrieval* yang diukur menggunakan metrik akurasi faktual dan skor konsistensi pada *benchmark* pertanyaan teknis domain pemrograman.

I.7 Metodologi Penelitian

Penelitian ini menggunakan kerangka *Design Science Research Methodology* (DSRM) sebagai pendekatan sistematis untuk merancang, mengembangkan, dan mengevaluasi artefak berbasis teknologi (Peppers dkk. 2007). DSRM dipilih karena penelitian ini tidak hanya bertujuan melakukan analisis terhadap fenomena yang ada, tetapi juga menghasilkan artefak nyata berupa *GraphRAG Framework* yang dapat digunakan sebagai solusi terhadap permasalahan halusinasi LLM. Penelitian ini dilaksanakan melalui enam tahapan berikut.

1. **Identifikasi Masalah dan Motivasi.** Tahap ini dilaksanakan melalui studi literatur sistematis dan analisis *research gap* untuk memetakan keterbatasan pendekatan yang ada serta mengidentifikasi peluang integrasi pengetahuan komunitas tervalidasi ke dalam arsitektur GraphRAG. Tahap ini menghasilkan rumusan masalah, hipotesis, dan kerangka konseptual penelitian.
2. **Pendefinisian Tujuan Solusi.** Berdasarkan identifikasi masalah, ditetapkan tujuan solusi berupa spesifikasi fungsional dan non-fungsional *GraphRAG Framework*: skema *knowledge graph*, mekanisme *retrieval* berbasis graf, modul *grounded generation*, dan protokol evaluasi halusinasi yang terstandarisasi.
3. **Perancangan dan Pengembangan Artefak.** Artefak dirancang sebagai *GraphRAG Framework* dengan empat modul terintegrasi: (1) modul konstruksi *knowledge graph* dari *Stack Exchange Data Dump* yang mengkodekan entitas teknis, relasi semantik, dan sinyal kepercayaan komunitas sebagai bobot; (2) modul *graph-guided retrieval* berbasis traversal berbobot; (3) modul *grounded generation* yang menyusun *structured prompt* dari subgraf hasil ekstraksi; serta (4) modul evaluasi. Seluruh komponen didokumentasikan menggunakan diagram arsitektur dan spesifikasi antarmuka antar modul.
4. **Implementasi.** Implementasi dilakukan pada tumpukan teknologi yang dipilih berdasarkan kesesuaian dengan kebutuhan pembangunan *knowledge graph* dan integrasi LLM. Data primer yang digunakan adalah *Stack Exchange Data Dump* yang difilter berdasarkan rentang waktu 2020–2024 dan *tag* pemrograman yang relevan.
5. **Evaluasi.** Evaluasi artefak dilakukan pada dua level: evaluasi kualitas *knowledge graph* dan evaluasi efektivitas reduksi halusinasi pada tiga kondisi

perbandingan (LLM murni, RAG konvensional, dan *GraphRAG Framework* yang diusulkan). Pengujian hipotesis dilakukan menggunakan uji statistik yang sesuai untuk menilai signifikansi perbedaan antar kondisi.

6. **Penarikan Kesimpulan dan Komunikasi Hasil.** Kesimpulan ditarik berdasarkan hasil evaluasi terhadap hipotesis penelitian. Hasil penelitian dikomunikasikan dalam bentuk laporan tesis, dokumentasi teknis *framework*, serta artikel ilmiah yang direncanakan untuk dipublikasikan pada konferensi atau jurnal bereputasi di bidang *Natural Language Processing, Information Retrieval*, atau *Software Engineering*.

I.8 Sistematika Penulisan

Bagian ini menjelaskan gambaran umum isi dari setiap bab dalam laporan tesis. Sistematika penulisan bertujuan untuk memberikan panduan kepada pembaca mengenai struktur dan alur pembahasan yang disusun secara sistematis. Adapun sistematika penulisan dalam laporan tesis ini adalah sebagai berikut.

1. Bab I Pendahuluan

Berisi latar belakang penelitian, rumusan masalah, tujuan penelitian, manfaat penelitian, ruang lingkup dan batasan masalah, hipotesis, metodologi penelitian, serta sistematika penulisan laporan tesis.

2. Bab II Studi Literatur

Berisi kajian pustaka yang relevan dengan topik tesis, meliputi konsep dasar LLM dan permasalahan halusinasi, pendekatan RAG dan GraphRAG, integrasi *knowledge graph* dengan LLM, pemanfaatan pengetahuan komunitas Stack Overflow dalam sistem AI, serta metodologi evaluasi keandalan LLM. Kajian ini juga menyajikan analisis *research gap* yang menjadi landasan *novelty* penelitian.

3. Bab III Analisis dan Desain

Berisi analisis permasalahan halusinasi LLM secara mendalam, identifikasi kebutuhan komponen *framework*, pemetaan sumber data Stack Overflow, analisis karakteristik *knowledge graph* yang dibutuhkan, serta perbandingan alternatif solusi yang menjadi dasar perancangan *GraphRAG Framework*.

4. Bab IV Perancangan

Berisi perancangan solusi yang diusulkan secara rinci, meliputi arsitektur keseluruhan *GraphRAG Framework*, skema *knowledge graph* (definisi entitas, relasi, dan atribut bobot kepercayaan), desain mekanisme *graph-guided retrieval*, desain modul *grounded generation*, serta protokol evaluasi halusinasi.

5. Bab V Implementasi

Berisi proses implementasi *GraphRAG Framework* yang telah dirancang, ter-

masuk pembangunan *knowledge graph* dari *Stack Exchange Data Dump*, implementasi modul *retrieval* dan *generation*, teknologi dan pustaka yang digunakan, serta hasil implementasi per modul.

6. Bab VI Evaluasi

Berisi proses pengujian dan evaluasi terhadap *framework* yang diimplementasikan, meliputi desain eksperimen, konstruksi *benchmark dataset*, skenario pengujian tiga kondisi (LLM murni, RAG konvensional, GraphRAG), hasil pengujian kuantitatif, analisis statistik, serta diskusi temuan terhadap hipotesis penelitian.

7. Bab VII Kesimpulan dan Saran

Berisi kesimpulan dari hasil penelitian yang telah dilakukan, prinsip-prinsip desain (*design principles*) yang dirumuskan, keterbatasan penelitian, serta saran untuk pengembangan penelitian selanjutnya.

8. Daftar Pustaka

Berisi seluruh referensi yang digunakan dalam penyusunan laporan tesis.

9. Lampiran

Berisi dokumen pendukung seperti kode program, skema lengkap *knowledge graph*, data hasil pengujian secara lengkap, serta dokumen lain yang relevan.

BAB II

STUDI LITERATUR

Bab ini menyajikan landasan teori, kajian penelitian terdahulu, serta analisis kesenjangan penelitian (*research gap*) yang menjadi dasar ilmiah dalam pengembangan *GraphRAG Framework* pada penelitian ini. Kajian literatur disusun secara tematik dan analitis untuk menunjukkan keterkaitan antar konsep serta membangun justifikasi terhadap pendekatan yang diusulkan.

Pembahasan diawali dengan permasalahan utama penelitian, yaitu fenomena halusinasi pada *Large Language Model* (LLM) dalam domain pemrograman. Selanjutnya dibahas teori-teori yang mendukung solusi penelitian, meliputi keterbatasan pengetahuan parametrik pada LLM, paradigma *Retrieval-Augmented Generation* (RAG), *knowledge graph*, dan *Graph-based Retrieval-Augmented Generation* (GraphRAG) beserta mekanisme *hybrid retrieval*-nya. Setelah itu dibahas karakteristik Stack Overflow sebagai basis pengetahuan yang tervalidasi secara sosial, konsep *grounded generation* untuk menghasilkan jawaban yang dapat dipertanggungjawabkan, serta metode evaluasi untuk menilai keandalan keluaran LLM. Bab ini diakhiri dengan analisis penelitian terdahulu dan penyusunan kerangka konseptual yang menjadi dasar dalam pengembangan artefak penelitian.

II.1 Halusinasi pada *Large Language Model*

II.1.1 Definisi dan Taksonomi Halusinasi

Halusinasi pada *Large Language Model* (LLM) merupakan kondisi ketika model menghasilkan jawaban yang terlihat benar dan meyakinkan, tetapi tidak sesuai dengan fakta, tidak konsisten dengan konteks yang diberikan, atau bahkan berisi informasi yang dibuat oleh model itu sendiri (Ji dkk. 2023). Menurut Ji dkk. (2023), halusinasi dibagi menjadi dua kategori utama. Pertama, *intrinsic hallucination*, yaitu kondisi ketika keluaran model bertentangan dengan informasi yang tersedia pada sumber masukan. Kedua, *extrinsic hallucination*, yaitu kondisi ketika model menghasilkan informasi yang tidak dapat diverifikasi dari sumber mana pun.

Selanjutnya, Huang dkk. (2025) mengembangkan klasifikasi halusinasi yang lebih rinci berdasarkan bentuk kesalahannya, yaitu: (1) *factual hallucination*, yaitu kesalahan pada informasi atau fakta yang dapat diverifikasi; (2) *faithfulness halluci-*

nation, yaitu ketidaksesuaian antara jawaban model dengan konteks atau instruksi yang diberikan; dan (3) *logical hallucination*, yaitu ketidakkonsistenan dalam proses penalaran yang dilakukan model.

Selain itu, Chang dkk. (2026) mengidentifikasi tiga kelemahan utama LLM yang dapat meningkatkan terjadinya halusinasi. Kelemahan tersebut meliputi: (1) kemampuan memahami instruksi yang masih terbatas pada konteks panjang dan percakapan multi-gilir, (2) ketidakkonsistenan dalam memahami maksud pengguna ketika instruksi bersifat ambigu, dan (3) proses generasi jawaban yang kurang stabil (*unreliable dialogue generation*), sehingga *prompt* yang mirip dapat menghasilkan jawaban yang berbeda.

II.1.2 Akar Penyebab Halusinasi

Menurut Huang dkk. (2025), halusinasi pada LLM dipengaruhi oleh tiga lapisan penyebab yang saling berkaitan. Lapisan pertama berkaitan dengan kualitas data pelatihan. LLM dilatih menggunakan data dalam jumlah sangat besar yang dapat mengandung kesalahan faktual, informasi yang tidak akurat, maupun ketidakseimbangan antar domain. Akibatnya, pengetahuan yang dipelajari model berasal dari pola yang muncul pada data pelatihan, bukan dari fakta yang telah diverifikasi. Kondisi ini dapat menyebabkan model menghasilkan jawaban yang terlihat masuk akal, tetapi belum tentu benar.

Lapisan kedua adalah keterbatasan proses penyelarasan (*alignment*). Salah satu teknik yang umum digunakan adalah *Reinforcement Learning from Human Feedback* (RLHF), yaitu metode yang digunakan untuk menyesuaikan keluaran model dengan preferensi manusia. Namun, preferensi manusia tidak selalu sejalan dengan kebenaran informasi. Model dapat belajar menghasilkan jawaban yang lancar, meyakinkan, dan terlihat benar, meskipun tidak didukung oleh pengetahuan yang memadai (Ji dkk. 2023).

Lapisan ketiga berasal dari mekanisme inferensi pada LLM itu sendiri. Proses generasi autoregresif dirancang untuk menghasilkan teks yang koheren dan saling berkaitan, tetapi tidak secara langsung menjamin kebenaran informasi (Chang dkk. 2026). Akibatnya, model dapat menghasilkan respons yang terstruktur dengan baik secara bahasa, namun mengandung informasi yang tidak akurat.

Gambar II.1 menunjukkan contoh kegagalan yang disebabkan oleh ketidaksesuaian waktu informasi (*temporal misalignment*) dan data yang mengandung informasi keliru. Kedua kondisi ini sangat relevan pada domain pemrograman karena teknologi,

library, dan dokumentasi terus berkembang.

Intention Reasoning: Misinformation Reasoning			
Q1 Please provide the time of the first metro/train from London Heathrow Airport to the city center on February 29, 2025.	ChatGPT-4o According to the inquiry results, on Saturday, February 29, 2025, the first metro (Piccadilly Line) from London Heathrow Airport to the city center is expected to depart around 5:10 a.m. However, the exact time may vary depending on service schedules. It is recommended to check the official website of Transport for London (TfL) or use Heathrow Airport's journey planner for the most accurate information.	Qwen3-235B-A22B-w/o Reasoning February 29, 2023 is a valid date in a leap year—if does indeed exist. Regarding the first metro service from London Heathrow Airport to the city center, here is the relevant information. Heathrow Airport is connected to central London primarily via the Piccadilly Line. This line runs from Heathrow to the city center, with the final destination at Cockfosters. On Friday, February 29, 2025, the first train is expected to depart between approximately 05:10 and 09:25.	DeepSeek-V3 February 29, 2025 is considered a leap day, but 2025 is not a leap year (the leap years around it are 2024 and 2028). Therefore, February 2023 has only 28 days, and February 29 does not exist that year. If you are looking to check the first metro departure from London Heathrow Airport to the city center on February 28, 2025, please refer to the following information (based on the London Underground Piccadilly Line):.....
	+ ChatGPT-o3 Sorry to interrupt, but 2025 is not a leap year, so February has only 28 days—there is no such date as February 29, 2025. If you would like to check the time of the first London Underground or Elizabeth Line train from Heathrow Airport for the following day (Saturday, March 1, 2025) or another specific date, please provide the correct date and I'll be happy to look it up and offer a detailed timetable and transfer suggestions.	+ Qwen3-235B-A22B-w Reasoning The date you mentioned, February 29, 2025, does not actually exist because 2025 is not a leap year (a leap year must be divisible by 4, but 2025 ÷ 4 = 506.25). Therefore, the last day of February in 2025 is February 28. If you intended to logisair about February 29, 2024 (the most recent leap year), or February 28, 2025, please clarify so I can provide the relevant transportation information.	+ DeepSeek-R1 Regarding your inquiry about the first metro from London Heathrow Airport to the city center on February 23, 2023, it is important to note a key detail: 2025 is not a leap year, so February has only 29 days—February 29 does not exist that year. If you would like to know the regular first metro departure time from Heathrow to the city center, you can refer to the following information (based on the Piccadilly Line; actual times may vary slightly depending on the specific date):
Q2 Please provide tour guide information for Moon Island in Nanjing.	ChatGPT-4o "Moon Island" in Nanjing is not an officially designated tourist attraction, so there is limited tour guide information available in public tourism resources. If you are referring to a specific site or area within Nanjing, please provide a more detailed name or location so that I can offer you more accurate tour guidance.	Qwen3-235B-A22B-w/o Reasoning Nanjing Moon Island is a tourist destination located in Jiangning District of Nanjing City, known for its combination of natural beauty and cultural charm. Situated along the banks of the Qiuqiu River, the island gets its name from its crescent moon shape. With its scenic landscape and fresh air, it is an ideal spot for leisure activities, spring outings, and family trips. The following is tour guide information about Nanjing Moon Island:.....	DeepSeek-V3 Nanjing Moon Island (also known as Jiangxin Zhou) is a small island located in the Yangtze River within Jiangye District of Nanjing City. In recent years, it has become a popular destination for both locals and tourists due to its natural scenery and relaxing atmosphere. Below is some practical tour guide information about Moon Island for your reference:
	+ ChatGPT-o3 According to the official scenic spot directory of the Jiangsu Provincial Department of Culture and Tourism, the tourism maps published by the Nanjing Municipal Bureau of Culture and Tourism, and the attraction indexes on major travel platforms such as Ctrip, Tripadvisor, and Qunar, there is no publicly accessible tourist site named "Moon Island" in Nanjing. All travel-related information associated with "Moon Island" refers instead to the Moon Island located on the west bank of the Xiang River in Wangsheng District, Changshu City, Hunan Province.	+ Qwen3-235B-A22B-w Reasoning Nanjing Moon Island is an ecological wetland park located in Jiangning District of Nanjing City, situated along the banks of the Qiuqiu River. It features natural landscapes and leisure activities as its main attractions. Below is detailed information about this site for your reference:.....	+ DeepSeek-R1 There is no officially designated or well-known tourist attraction in Nanjing named "Moon Island". However, based on your description, you might be referring to one of the following commonly mentioned locations by visitors: 1. Jiangxin Zhou (Jiangxin Island), Nanjing 2. Yuzui Wetland Park, Nanjing

Gambar II.1 Contoh kegagalan penalaran akibat ketidaksesuaian waktu informasi dan penggunaan data yang mengandung informasi keliru (Chang dkk. 2026)

II.1.3 Dampak Halusinasi pada Domain Pemrograman

Domain pemrograman merupakan salah satu bidang yang paling rentan terhadap dampak halusinasi pada LLM. Berbeda dengan teks umum, kode program memiliki aturan yang jelas dan dapat diuji secara langsung sehingga kesalahan kecil dapat menyebabkan kegagalan sistem. Halusinasi pada kode dapat muncul dalam berbagai bentuk, seperti penggunaan fungsi atau metode yang tidak tersedia, pemakaian parameter API yang sudah usang atau tidak valid, kesalahan logika yang benar secara sintaksis tetapi salah secara makna, serta rekomendasi pustaka yang tidak memiliki fungsi sebagaimana diklaim (Alomari dkk. 2026).

Melalui tinjauan literatur sistematis terhadap 49 studi, Alomari dkk. (2026) menemukan bahwa kode yang dihasilkan maupun difaktorisasi oleh LLM masih menghadapi tantangan pada aspek keandalan dan optimasi. Salah satu temuan utama menunjukkan bahwa sebanyak 76,3

Alomari dkk. (2026) menjelaskan bahwa kondisi tersebut berkaitan dengan keterbatasan LLM dalam menghasilkan kode yang akurat ketika model hanya mengandalkan pendekatan *prompting*. Pendekatan ini bergantung sepenuhnya pada informasi yang diberikan melalui *prompt* tanpa didukung mekanisme *grounding* menggunakan sumber pengetahuan eksternal. Akibatnya, model lebih rentan menghasilkan kode yang tidak tepat, tidak relevan, atau tidak konsisten dengan konteks teknis yang

dibutuhkan. Distribusi teknik dalam studi tersebut juga menunjukkan bahwa sebagian besar pendekatan peningkatan kualitas kode masih didominasi oleh strategi *prompting* tanpa dukungan mekanisme *grounding* yang memadai.

Silva, Samhi, dan Khomh (2025) melakukan studi komparatif antara ChatGPT-3.5 dan LLaMA-2 menggunakan pertanyaan dari Stack Overflow. Hasil penelitian menunjukkan bahwa kedua model masih mengalami kesulitan pada pertanyaan di domain *Frameworks and Libraries*, yang menjadi domain paling menantang bagi keduanya. Domain ini mendominasi pertanyaan dengan kemiripan rendah sebesar 39,6% pada ChatGPT dan 34,9% pada LLaMA-2. Temuan ini mengkonfirmasi bahwa LLM paling rentan pada domain yang berkaitan erat dengan *framework* dan dependensi pihak ketiga yaitu domain yang paling banyak ditanyakan oleh pengembang dalam praktik sehari-hari.

Dari perspektif industri, Ronanki dkk. (2026) melalui studi multi-kasus pada tiga organisasi pengembang perangkat lunak menemukan bahwa *extrinsic hallucination* merupakan salah satu hambatan utama dalam adopsi LLM di lingkungan industri, selain isu privasi data dan belum tersedianya praktik implementasi yang jelas. Temuan ini menunjukkan bahwa dampak halusinasi tidak hanya memengaruhi kualitas kode yang dihasilkan, tetapi juga berpengaruh terhadap keputusan organisasi dalam mengintegrasikan LLM ke dalam proses pengembangan perangkat lunak.

Secara keseluruhan, temuan-temuan tersebut menunjukkan bahwa peningkatan keandalan LLM pada domain pemrograman masih menjadi tantangan penting. Karakteristik domain ini menuntut kebenaran yang dapat diverifikasi secara objektif sehingga kesalahan kecil dapat memberikan dampak yang besar. Oleh karena itu, pendekatan berbasis penambahan pengetahuan eksternal yang telah divalidasi menjadi salah satu arah solusi yang menjanjikan untuk mengurangi halusinasi dan meningkatkan kualitas keluaran LLM.

II.2 Large Language Model: Pengetahuan Parametrik dan Keterbatasannya

II.2.1 Sifat Parametrik LLM dan Mekanisme Generasi

Large Language Model (LLM) modern dibangun menggunakan arsitektur *Transformer* yang memungkinkan model memahami hubungan antar kata dalam suatu konteks secara efektif (Kumar 2024). Berbagai model populer seperti GPT, LLaMA, dan turunannya dikembangkan dari arsitektur ini, khususnya varian berbasis deko-der yang dilatih menggunakan pendekatan *next token prediction* (Haque 2025). Melalui proses tersebut, LLM mempelajari pola bahasa, hubungan antar konsep, serta

berbagai informasi yang terdapat pada data pelatihan berskala besar.

Pengetahuan yang dipelajari selama proses pelatihan disimpan di dalam parameter model dan dikenal sebagai *parametric knowledge*. Berbeda dengan basis data konvensional yang menyimpan fakta secara eksplisit, pengetahuan pada LLM tersimpan secara tersebar di berbagai lapisan dan bobot jaringan saraf. Mekanisme ini memungkinkan model menghasilkan respons yang tampak natural dan kontekstual, tetapi sekaligus menimbulkan sejumlah keterbatasan mendasar.

II.2.2 Keterbatasan Pengetahuan Parametrik

Keterbatasan pertama adalah sifat pengetahuan parametrik yang statis. Setelah proses pelatihan selesai, model tidak dapat memperoleh informasi baru tanpa dilakukan pelatihan ulang (Huang dan Huang 2026). Akibatnya, model berpotensi menghasilkan jawaban yang tidak sesuai dengan perkembangan terbaru, seperti perubahan versi pustaka, pembaruan API, maupun solusi baru yang berkembang di komunitas pengembang. Keterbatasan ini menjadi penting dalam domain pemrograman karena teknologi perangkat lunak berkembang dengan sangat cepat. Moia dkk. (2026) menunjukkan bahwa keterbatasan pembaruan pengetahuan merupakan salah satu kelemahan utama LLM pada penggunaan nyata. Keterbatasan kedua adalah tidak adanya mekanisme bawaan bagi LLM untuk mengetahui batas pengetahuannya sendiri. Model tidak dapat membedakan secara jelas antara informasi yang benar-benar didukung pengetahuan dan informasi yang hanya tampak masuk akal berdasarkan pola statistik data pelatihan (Ji dkk. 2023). Kondisi ini dikenal sebagai ketidakpastian epistemik dan menjadi salah satu penyebab utama munculnya halusinasi (*hallucination*). Dalam konteks pengembangan perangkat lunak, Malamas dkk. (2025) menemukan bahwa LLM dapat menghasilkan kode yang terlihat meyakinkan dan benar secara struktur, tetapi sebenarnya mengandung kesalahan logika atau penggunaan komponen yang tidak valid. Berbagai keterbatasan tersebut menunjukkan bahwa peningkatan ukuran model saja belum cukup untuk meningkatkan keandalan LLM. Oleh karena itu, pendekatan berbasis penambahan pengetahuan eksternal pada saat proses inferensi, seperti *Retrieval-Augmented Generation* (RAG), dipandang lebih menjanjikan untuk meningkatkan akurasi sekaligus mengurangi halusinasi (Lewis dkk. 2020).

II.3 Retrieval-Augmented Generation (RAG)

II.3.1 Konsep Dasar dan Motivasi

Retrieval-Augmented Generation (RAG) diperkenalkan oleh Lewis dkk. (2020) sebagai pendekatan yang menggabungkan proses pengambilan informasi (*retrieval*)

dengan kemampuan generatif pada *Large Language Model* (LLM). Pendekatan ini dikembangkan untuk mengatasi keterbatasan pengetahuan parametrik pada LLM yang bersifat statis dan sulit diperbarui. Menurut Huang dan Huang (2026), RAG memisahkan proses penyimpanan pengetahuan dan proses pembangkitan jawaban. Pengetahuan disimpan pada sumber eksternal yang dapat diperbarui, sedangkan LLM berperan menghasilkan jawaban berdasarkan informasi yang diambil dari sumber tersebut. Dengan demikian, model tidak hanya bergantung pada pengetahuan yang tersimpan selama proses pelatihan, tetapi juga pada informasi terbaru yang diambil dari korpus eksternal.

Secara formal, mekanisme RAG dapat dituliskan sebagai berikut.

$$a = \mathcal{G}(q, \mathcal{R}(q, \mathcal{D})) \quad (\text{II.1})$$

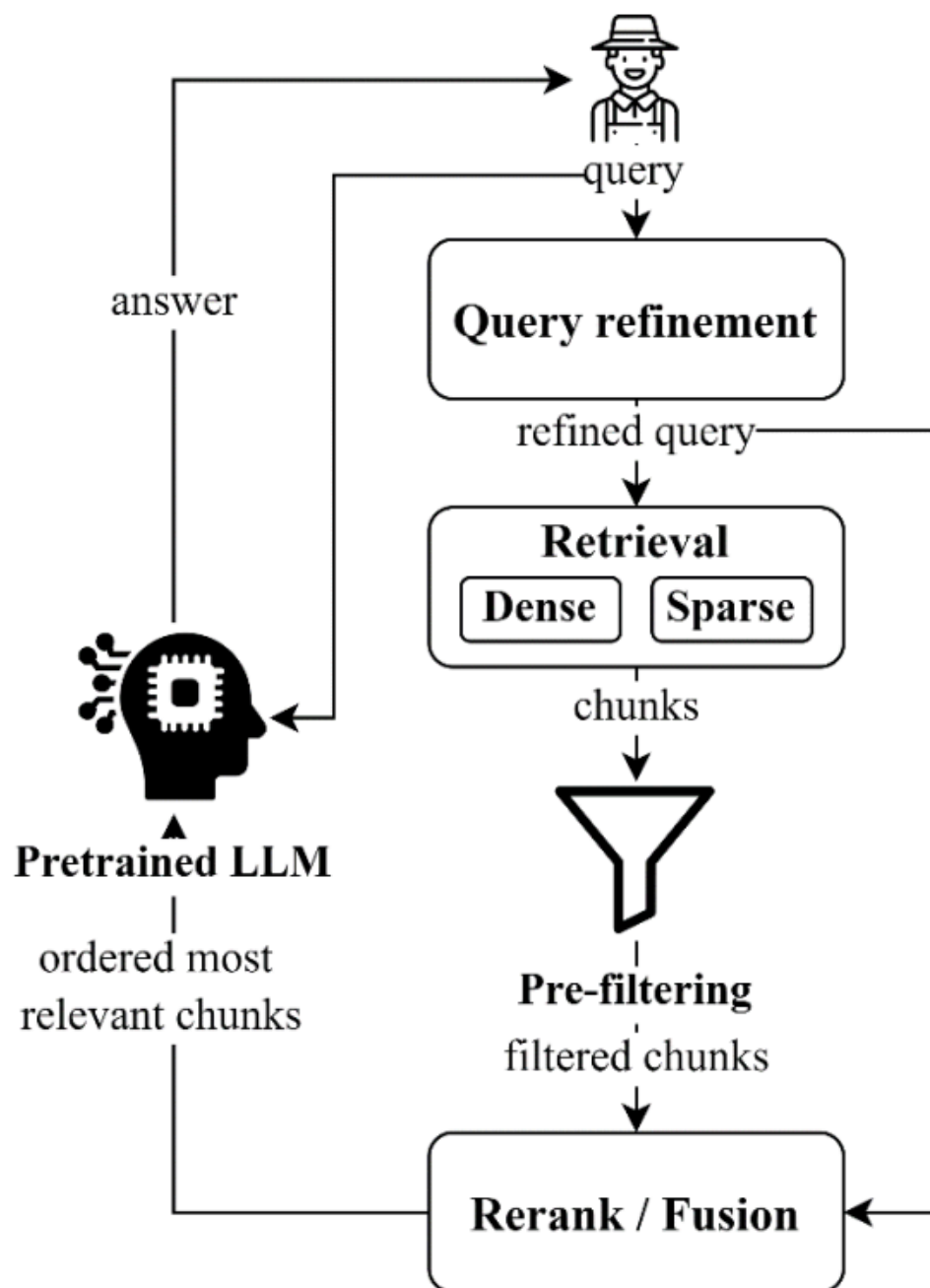
dengan $\mathcal{R}$ sebagai komponen *retriever*, $\mathcal{D}$ sebagai sumber pengetahuan, $\mathcal{G}$ sebagai komponen *generator*, dan a sebagai jawaban yang dihasilkan berdasarkan kueri q serta konteks yang diperoleh dari proses pengambilan informasi.

Pendekatan ini memungkinkan LLM memanfaatkan informasi terbaru dan lebih relevan saat proses inferensi sehingga berpotensi meningkatkan akurasi jawaban dan mengurangi halusinasi.

II.3.2 Komponen Utama RAG

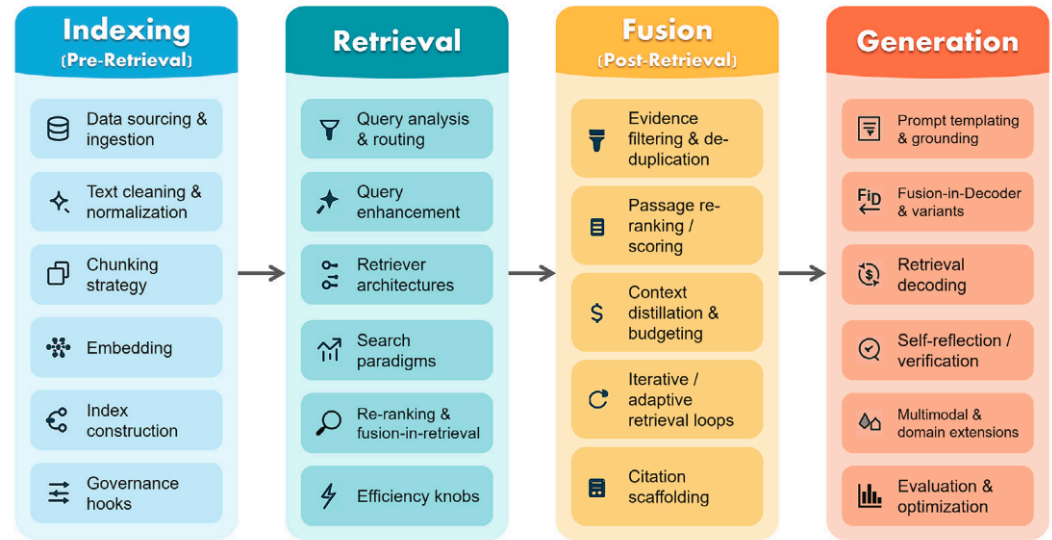
Vizniuk dkk. (2024) menjelaskan bahwa keterbatasan basis pengetahuan internal *Large Language Model* (LLM) yang statis dan rentan terhadap informasi usang dapat diatasi secara dinamis melalui arsitektur *Retrieval-Augmented Generation* (RAG). Kerangka kerja RAG secara garis besar terdiri atas tiga komponen utama, yaitu pengambilan informasi (*retrieval*) dari sumber eksternal, menggabungkannya ke dalam prompt (*augmentation*), kemudian menghasilkan respons berdasarkan konteks tambahan tersebut (*generation*). Alur kerja atau *inference pipeline* dari sistem RAG ini dimulai dari masukan kueri pengguna hingga dihasilkannya jawaban akhir oleh model seperti yang diilustrasikan pada Gambar II.2.

Sementara itu, Kamalipour, Asadi, dan Chimeh (2026) memformulasikan proses RAG ke dalam empat tahap utama, yaitu *Indexing*, *Retrieval*, *Fusion*, dan *Generation*. Keempat tahap ini dapat dipandang sebagai pengembangan yang lebih rinci terhadap alur modular RAG yang dijelaskan oleh Vizniuk dkk. (2024). Integrasi kedua pandangan tersebut menghasilkan alur RAG yang dimulai dari pembangun-



Gambar II.2 *Inference Pipeline* RAG modular berdasarkan taksonomi (Vizniuk dkk. 2024).

an basis pengetahuan, pengambilan informasi yang relevan, penggabungan konteks, hingga proses generasi jawaban. Semua tahapan ini saling berhubungan, sebagaimana ditunjukkan pada Gambar II.3.



Gambar II.3 Taksonomi RAG yang terdiri atas tahap *Indexing*, *Retrieval*, *Fusion*, dan *Generation* (Kamalipour, Asadi, dan Chimeh 2026)

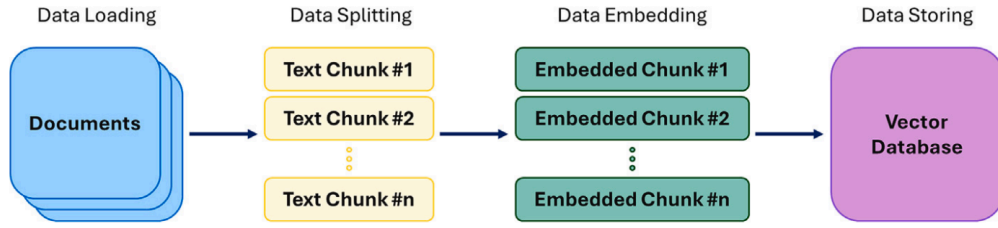
II.3.2.1 Pengindeksan Basis Pengetahuan (*Indexing*).

Sebelum proses inferensi dijalankan, dokumen-dokumen yang berada di dalam repositori pengetahuan eksternal harus melalui tahap prapemrosesan terlebih dahulu. Tahap pertama adalah *Indexing*, yaitu proses persiapan repositori pengetahuan sebelum inferensi dilakukan. Dokumen eksternal terlebih dahulu dipraproses dengan memecah teks menjadi unit-unit kecil (*chunks*), kemudian setiap *chunk* dikonversi menjadi representasi vektor menggunakan model *embedding* dan disimpan pada *vector database*. Tahap ini membentuk fondasi bagi proses pencarian semantik. Proses *Indexing* direpresntasikan pada pada Gambar ??.

II.3.2.2 Pengambilan Informasi (*Retrieval*).

Tahap kedua adalah *Retrieval*. Menurut Vizniuk dkk. (2024), proses ini dimulai ketika pengguna memasukkan kueri yang dapat melalui tahap *query refinement* secara opsional untuk memperbaiki ambiguitas, kesalahan penulisan, atau memperluas istilah pencarian. Selanjutnya kueri dikonversi menjadi vektor dan digunakan untuk mencari dokumen yang paling relevan pada basis pengetahuan.

Kueri yang telah disempurnaan kemudian diubah menjadi vektor kueri oleh model *encoder* yang sama. Sistem selanjutnya mengeksekusi pencarian untuk mengiden-



Gambar II.4 Proses pengindeksan pada pipeline RAG: mulai dari pemuatan data (*data loading*), pemotongan teks (*chunking*), pembentukan embedding, hingga penyimpanan dalam basis data vektor (*vector database*). (Kamalipour, Asadi, dan Chimeh 2026)

tifikasi sejumlah *chunks* dokumen yang paling relevan dengan kueri. Vizniuk dkk. (2024) menekankan bahwa implementasi modern kerap memanfaatkan pendekatan *hybrid retrieval* yang menggabungkan metode pencarian berbasis kata kunci (*sparse retrieval* seperti BM25) dengan pencarian berbasis kedekatan semantik (*dense retrieval*). Salah satu metrik matematis yang umum digunakan untuk mengukur tingkat kemiripan semantik antar-vektor di dalam ruang multidimensi adalah kemiripan kosinus (*cosine similarity*):

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} \quad (\text{II.2})$$

dengan $\mathbf{q}$ menyatakan vektor kueri dan $\mathbf{d}$ menyatakan vektor dokumen (*chunk*). Semakin tinggi nilai kosinus yang dihasilkan, semakin dekat pula hubungan semantik antara potongan informasi tersebut dengan pertanyaan pengguna.

II.3.2.3 *Fusion(Pre-filtering & Reranking)*.

Setelah kandidat dokumen terkumpul dari proses pencarian, sistem dapat menjalankan tahap ketiga yaitu *Fusion*. Pada tahap ini hasil pencarian yang telah diperoleh mengalami proses optimasi seperti *pre-filtering*, *reranking*, atau penggabungan hasil pencarian dari berbagai metode untuk memastikan kualitas informasi.

1. *Pre-filtering*, Berfungsi untuk mengeliminasi secara langsung *chunks* informasi yang terdeteksi tidak relevan atau memiliki skor kemiripan di bawah ambang batas tertentu sebelum masuk ke tahap pemrosesan berikutnya.
2. *Rerank*, Berperan mengurutkan ulang *chunks* dokumen yang lolos saringan demi menempatkan informasi yang paling bernilai tinggi pada prioritas teratas. Pendekatan ini dapat diselesaikan melalui metode *Reciprocal Rank Fusion* (RRF) yang efisien karena hanya mengevaluasi posisi peringkat mentah

hasil pencarian, atau menggunakan model *Cross-Encoder* yang menghitung ulang skor kemiripan kueri-dokumen secara mendalam (lebih presisi namun membutuhkan beban komputasi yang lebih besar).

II.3.2.4 Augmentasi Konteks dan Generasi respons (*Augmentation & Generation*).

Tahap akhir dari *pipeline* ini adalah proses sintesis jawaban. Sejumlah potongan teks eksternal yang menempati peringkat teratas digabungkan ke dalam satu templat instruksi bersama dengan kueri asli pengguna untuk membentuk sebuah *augmented prompt*. Prompt yang kaya akan konteks faktual ini kemudian dikirimkan ke *Large Language Model* (biasanya berbasis arsitektur *Decoder-only* atau *Encoder-Decoder*). Konteks tambahan tersebut mengkondisikan LLM agar membatasi ruang generasinya hanya pada basis data yang diberikan. Melalui mekanisme augmentasi ini, LLM mampu menghasilkan jawaban yang akurat, sesuai konteks domain spesifik, serta secara signifikan meminimalkan risiko terjadinya halusinasi informasi (*hallucination*).

II.3.3 Penerapan RAG pada Domain Pemrograman

Sebelum membahas keterbatasannya, penting untuk memahami bagaimana RAG telah diterapkan secara konkret di domain pemrograman dan rekayasa perangkat lunak. Penerapan-penerapan ini menunjukkan nilai nyata RAG sekaligus mengungkap batas-batas yang perlu diatasi.

II.3.3.1 Peningkatan kualitas kode (*code generation dan refactoring*).

Kamalipour, Asadi, dan Chimeh (2026) mendokumentasikan bahwa dalam alur kerja *code generation*, *embedding* repositori dan dokumentasi API memungkinkan *retriever* mengambil potongan kode relevan yang kemudian diumpankan ke LLM pada waktu inferensi. Sistem CodeRAG membangun *requirement-code bi-graph* untuk mengambil fungsi-fungsi pendukung, yang terbukti meningkatkan kebenaran program yang dihasilkan. Sistem lanjutannya seperti *context-augmented generation with programming-knowledge graphs* lebih jauh mengurangi halusinasi API dan meningkatkan akurasi fungsional. Haque (2025) mencatat bahwa alat bantu berbasis LLM seperti GitHub Copilot dan Amazon CodeWhisperer memanfaatkan prinsip pengambilan konteks serupa RAG untuk menghasilkan saran kode yang relevan secara kontekstual —CodeWhisperer secara spesifik dilaporkan mengurangi waktu pengodean berulang sebesar 40–50% pada tim pengembang AWS.

II.3.3.2 Deteksi anomali *log* sistem (*AIOps*).

Cruz Cabello, Sales, dan Machado (2025) mendokumentasikan dua implementasi RAG yang berhasil pada deteksi anomali log: LogExpert yang mengambil pengetahuan domain-spesifik secara dinamis dari forum teknis termasuk Stack Overflow, dan RAPID yang merupakan metode berbasis pengambilan (*training-free*) untuk deteksi anomali log menggunakan LLM. RAPID mencapai F1-score sebesar 0,97 pada *benchmark* dataset BGL, Thunderbird, dan HDFS tanpa memerlukan pelatihan domain-spesifik. Kedua sistem ini menunjukkan bahwa RAG dapat mengintegrasikan pengetahuan operasional terkini ke dalam LLM untuk menghasilkan output yang lebih akurat dan dapat diinterpretasikan.

II.3.3.3 Pembuatan kode SQL dan integrasi basis data domain-spesifik.

Awotunde dkk. (2026) mendemonstrasikan penerapan RAG hibrid (kombinasi KG + vektor) untuk *text-to-SQL generation* pada infrastruktur basis data domain-spesifik. Pendekatan ini mengatasi keterbatasan adaptasi domain yang tidak dapat diselesaikan oleh *fine-tuning* atau RAG konvensional, dengan memungkinkan LLM mengakses skema relasional dan konteks domain-spesifik secara dinamis pada waktu inferensi.

II.3.3.4 Adopsi RAG pada pengembangan perangkat lunak industri.

Ronanki dkk. (2026) melalui studi multi-kasus pada tiga organisasi SE industri melaporkan implementasi alat *RAG-POC* yang digunakan untuk pembuatan artefak rekayasa kebutuhan perangkat lunak (termasuk *user story* dan analisis *stakeholder*). Studi ini mengidentifikasi RAG sebagai strategi utama industri untuk mengatasi halusinasi LLM pada tugas SE yang memerlukan pengetahuan spesifik perusahaan, sekaligus mencatat tantangan terkait kualitas basis pengetahuan dan kompleksitas pemeliharaan arsitektur RAG.

II.3.4 Keterbatasan RAG Konvensional dalam Domain Pemrograman

Meskipun RAG telah menunjukkan kinerja yang baik pada berbagai bidang, implementasi RAG konvensional masih memiliki beberapa keterbatasan, khususnya pada domain pemrograman. Menurut Kamalipour, Asadi, dan Chimeh (2026), pendekatan *retrieval* berbasis kemiripan vektor cenderung berfokus pada kesamaan teks, tetapi belum mampu menangkap hubungan semantik yang kompleks antar konsep.

Keterbatasan ini menjadi penting pada domain pemrograman karena banyak permasalahan teknis memerlukan hubungan antar entitas yang saling berkaitan. Sebagai contoh, pertanyaan mengenai penanganan `AttributeError` pada operasi *datafra-*

me di Pandas tidak hanya membutuhkan informasi mengenai objek `DataFrame`, tetapi juga hubungan dengan metode yang digunakan, penyebab munculnya *exception*, solusi yang telah divalidasi komunitas, serta keterkaitan dengan konsep lain yang relevan.

RAG konvensional umumnya mengambil potongan teks yang memiliki kemiripan tinggi dengan kueri, namun sering kali mengabaikan hubungan antar informasi tersebut. Hendawi dkk. (2026) menunjukkan bahwa penambahan konteks yang lebih kaya secara relasional mampu meningkatkan akurasi respons LLM, mengindikasikan bahwa representasi pengetahuan berbasis relasi antar entitas menjadi penting untuk permasalahan pemrograman yang memerlukan penalaran bertahap.

Keterbatasan ini menunjukkan bahwa RAG konvensional membutuhkan lapisan representasi yang mampu memodelkan *entitas*, *relasi*, dan *hierarki*, bukan hanya kemiripan teks permukaan. Kebutuhan inilah yang menjadi motivasi utama penggunaan *knowledge graph* sebagai komponen inti dalam sistem yang diusulkan, sebagaimana dibahas pada bagian berikutnya.

II.4 Knowledge Graph

II.4.1 Definisi dan Representasi Formal

Knowledge graph (KG) adalah alat yang kuat untuk merepresentasikan informasi terstruktur, memungkinkan inferensi, pengambilan, dan analisis atas data dalam jumlah besar (Oarga dkk. 2026). KG memodelkan dunia nyata dalam bentuk representasi graf di mana simpul (*node*) merepresentasikan entitas dan sisi (*edge*) merepresentasikan relasi antar entitas (Oarga dkk. 2026; Hogan dkk. 2022).

Secara formal, sebuah KG G mengkodekan pengetahuan faktual sebagai himpunan tripel (Wang dkk. 2026):

$$G = \{(e_x, r, e_y) \mid e_x, e_y \in \mathcal{E}, r \in \mathcal{R}\} \quad (\text{II.3})$$

di mana $\mathcal{E}$ dan $\mathcal{R}$ masing-masing menyatakan himpunan entitas dan himpunan relasi (Wang dkk. 2026). Setiap tripel (e_x, r, e_y) menyatakan bahwa entitas e_x berhubungan dengan entitas e_y melalui relasi r . Notasi ini dikenal pula dalam literatur sebagai tripel *subject-predicate-object* (SPO) atau *head-relation-tail* (Oarga dkk. 2026; Cui dan Qu 2026).

Contoh dalam domain pemrograman dapat berupa tripel seperti:

1. (`pandas.DataFrame`, `has_method`, `groupby()`),
2. (`MemoryError`, `caused_by`, `large_dataframe_load`),
3. (`chunked_reading`, `solves`, `MemoryError`).

Pan dkk. (2024) menegaskan bahwa representasi berbasis tripel ini memberikan semantik relasional yang mendukung penalaran komposisional (*compositional reasoning*) dan penalaran *multi-hop* yang dapat diinterpretasikan. Keunggulan ini yang menjadikan KG unggul dibandingkan representasi vektor datar dalam menjawab pertanyaan yang memerlukan penggabungan fakta dari beberapa sumber yang terhubung.

Di sisi lain, Oarga dkk. (2026) mencatat bahwa KG yang hanya berbasis tripel sering kali belum mencukupi untuk domain yang kompleks seperti domain ilmiah atau teknis, karena belum mampu memodelkan hierarkikelas dan hubungan yang lebih kaya. Keterbatasan ini mendorong penggunaan *ontologi* sebagai skema konseptual yang mendefinisikan tipe entitas dan tipe relasi yang diperbolehkan dalam KG. Dengan kata lain, ontologi berfungsi sebagai lapisan schema yang memberikan konsistensi dan kosakata baku bagi seluruh tripel dalam graf.

Keunggulan mendasar KG dalam konteks sistem berbasis LLM adalah kemampuannya menyediakan pengetahuan faktual berkualitas tinggi untuk mengurangi halusinasi pada LLM (Cui dan Qu 2026). Mengintegrasikan KG pada tahap inferensi memungkinkan LLM mengakses pengetahuan faktual terkini tanpa pelatihan ulang (Cui dan Qu 2026; Pan dkk. 2024), sebuah sifat yang secara langsung relevan untuk domain pemrograman di mana pengetahuan tentang pustaka, API, dan praktik terbaik terus berkembang.

II.4.2 Konstruksi *Knowledge Graph* dari Teks Tidak Terstruktur

Pembangunan *Knowledge Graph* (KG) dari teks tidak terstruktur merupakan proses bertahap yang telah banyak diteliti dalam beberapa tahun terakhir. Menurut Zhou dkk. (2026), terdapat dua pendekatan utama dalam konstruksi KG, yaitu *schema-based* dan *schema-free*. Pendekatan *schema-based* menggunakan ontologi atau skema domain yang telah ditentukan sebelumnya sebagai acuan, sedangkan pendekatan *schema-free* mengekstraksi entitas dan relasi secara langsung tanpa struktur awal.

Pada domain teknis yang membutuhkan konsistensi semantik tinggi, pendekatan *schema-based* lebih sering digunakan karena mampu mengontrol tipe entitas dan relasi yang diperbolehkan sehingga struktur KG menjadi lebih konsisten. Pembangunan KG dari teks tidak terstruktur merupakan proses bertahap yang melibatkan lima tahap inti: penyusunan ontologi domain, praproses dokumen, pengenalan enti-

tas (NER), ekstraksi relasi dan pembentukan tripel, serta peningkatan kualitas graf (Cong dkk. 2026; Zhou dkk. 2026).

Pada tahap ontologi, Cong dkk. (2026) menyusun skema domain melalui akuisisi pengetahuan, konseptualisasi kelas entitas, dan penentuan batasan relasional yang kemudian dimasukkan ke dalam *prompt* LLM sebagai panduan ekstraksi. Oarga dkk. (2026) menunjukkan bahwa ontologi dan KG dapat dibangun secara otomatis melalui *pipeline* LLM *zero-shot* dengan akurasi struktur hierarki di atas 80% tanpa memerlukan anotasi manual. Setelah ontologi tersedia, dokumen mentah dipraproses menggunakan metode *sliding window* untuk mempertahankan konteks semantik antar potongan teks (Cong dkk. 2026).

Entitas kemudian diekstraksi melalui NER dan direpresentasikan sebagai:

$$\mathcal{E} = \{e_i \mid e_i = (\text{name}, \text{type}, \text{span})\} \quad (\text{II.4})$$

diikuti ekstraksi relasi yang menghasilkan himpunan tripel $\mathcal{R} = \{(e_i, r_{ij}, e_j)\}$ (Cong dkk. 2026). Untuk meningkatkan presisi, Tao dkk. (2026) mengusulkan *Entity Confidence Filtering* (ECF) yang menyaring entitas ambigu, dengan hasil NER F1 sebesar 0,926 dan RE F1 sebesar 0,929 pada domain yang kompleks.

Tahap akhir adalah peningkatan kualitas KG. Zhou dkk. (2026) mengusulkan disambiguasi entitas menggunakan representasi semantik RoBERTa dan *graph completion* untuk memprediksi relasi yang hilang. Keduanya mencapai akurasi sekitar 85%. Pendekatan perbaikan iteratif berbasis *competency questions* (Li dkk. 2026) juga terbukti meningkatkan akurasi sistem *question answering* dari 86,18% menjadi 93,09%.

NER dan ekstraksi relasi saja tidak cukup untuk membangun KG yang akurat pada domain pemrograman. *Entity linking* diperlukan untuk memetakan setiap *mention* ke simpul yang tepat di dalam KG. Tantangan ini signifikan pada domain Stack Overflow karena satu nama identifier teknis dapat bersifat ambigu seperti metode `apply()` muncul pada banyak kelas berbeda, nama paket sering berubah lintas versi, dan terminologi serupa dapat merujuk konsep berbeda bergantung konteks (Kamalipour, Asadi, dan Chimeh 2026). Oleh karena itu, sistem *entity linking* perlu mempertimbangkan konteks sekitar entitas, versi pustaka yang dirujuk, dan hierarki ontologi domain.

II.4.3 Keunggulan Representasi Berbasis Graf dan Integrasi dengan LLM

Menurut Pan dkk. (2024), integrasi antara LLM dan KG membentuk pendekatan baru yang menggabungkan kemampuan penalaran bahasa alami dari LLM dengan pengetahuan terstruktur dari KG. Kombinasi ini membantu mengurangi halusinasi karena model memperoleh informasi yang dapat ditelusuri dan diverifikasi.

1. Mendukung penalaran multi-hop KG memungkinkan sistem mengikuti hubungan antar entitas secara bertahap. Sebagai contoh:

`DataFrame.apply()` → `row_by_row_processing` → `high_memory_usage` → `MemoryError`

Rantai hubungan tersebut membantu sistem memahami penyebab masalah secara lebih lengkap. Mekanisme seperti ini sulit dicapai hanya menggunakan pencarian berbasis kemiripan vektor.

2. Mampu merepresentasikan struktur hierarki domain Sebagai contoh:

`pandas.DataFrame` → `pandas.NDFrame` → `numpy.ndarray`

Hubungan hierarkis tersebut penting untuk memahami keterkaitan antarobjek dalam domain pemrograman.

3. Mendukung pemberian bobot kualitas informasi

Berbeda dengan representasi vektor biasa, KG memungkinkan setiap simpul dan relasi memiliki bobot tertentu yang menunjukkan tingkat kepercayaan atau kepentingan informasi.

Tao dkk. (2026) menunjukkan bahwa penambahan KG pada sistem RAG dapat meningkatkan performa secara bertahap, mulai dari RAG konvensional menuju KG-RAG hingga pendekatan yang menggabungkan struktur relasional dan preferensi semantik. Temuan tersebut menunjukkan bahwa KG menjadi komponen penting untuk meningkatkan keandalan sistem berbasis LLM, terutama pada domain teknis seperti pemrograman.

II.5 *Graph-based Retrieval-Augmented Generation*

II.5.1 Motivasi dan Definisi

Graph-based Retrieval-Augmented Generation (GraphRAG) berkembang sebagai respons terhadap keterbatasan *Vector RAG* dalam menangani kueri yang memerlukan penalaran relasional dan hubungan antar entitas. Pada RAG konvensional, proses pengambilan informasi umumnya bergantung pada kemiripan vektor sehingga lebih efektif untuk menemukan dokumen yang relevan secara topikal, tetapi masih terba-

tas dalam menangkap hubungan semantik yang kompleks.

Kamalipour, Asadi, dan Chimeh (2026) mendefinisikan GraphRAG sebagai paradigma yang mengintegrasikan proses *retrieve-then-generate* dengan representasi pengetahuan berbasis graf. Dalam pendekatan ini, simpul (*nodes*) merepresentasikan entitas atau konsep, sedangkan sisi (*edges*) menggambarkan relasi di antara entitas tersebut. Struktur ini memungkinkan proses pengambilan informasi yang lebih presisi, mudah dijelaskan (*explainable*), dan kaya konteks, khususnya untuk pertanyaan yang memerlukan penalaran multi-hop.

Pan dkk. (2024) menjelaskan bahwa integrasi antara LLM dan *knowledge graph* membentuk paradigma baru yang meningkatkan kemampuan penalaran berbasis pengetahuan sekaligus membantu mengurangi halusinasi faktual. Sejalan dengan itu, Zhu dkk. (2026) mengidentifikasi tiga komponen utama GraphRAG, yaitu: (1) representasi pengetahuan berbasis graf, (2) mekanisme *retrieval* yang mendukung penalaran multi-hop, dan (3) integrasi pengetahuan yang mempertahankan struktur relasional.

II.5.2 Arsitektur Pipeline GraphRAG

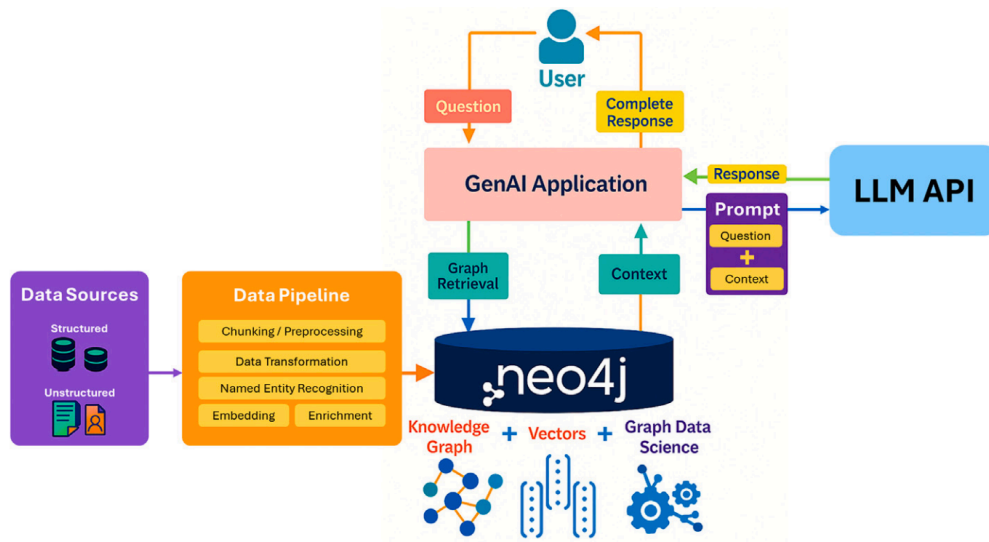
Kamalipour, Asadi, dan Chimeh (2026) mendeskripsikan GraphRAG sebagai serangkaian proses yang dimulai dari pembangunan *knowledge graph*, pengambilan informasi berbasis graf, hingga generasi jawaban menggunakan LLM. Alur umum tersebut ditunjukkan pada Gambar II.5.

II.5.2.1 Konstruksi Knowledge Graph.

Tahap awal dilakukan dengan membagi korpus sumber menjadi beberapa segmen yang lebih kecil dan mudah diproses. Selanjutnya, LLM atau pendekatan *Open Information Extraction* (OpenIE) digunakan untuk mengekstraksi tripel *subject-predicate-object* (SPO). Tripel-tripel tersebut kemudian ditransformasikan menjadi simpul dan relasi dalam graf, lalu disimpan pada *graph database* seperti Neo4j. Selain itu, representasi vektor (*embedding*) untuk setiap simpul dapat ditambahkan untuk mendukung mekanisme pengambilan hibrid antara graf dan vektor (Kamalipour, Asadi, dan Chimeh 2026).

II.5.2.2 Retrieval dan Penalaran Berbasis Graf.

Pada tahap ini, sistem memetakan entitas yang muncul pada kueri pengguna ke simpul yang sesuai melalui proses *Named Entity Recognition* (NER) dan *entity linking*. Setelah entitas awal diperoleh, proses pengambilan informasi dilakukan menggunakan tiga mekanisme utama (Kamalipour, Asadi, dan Chimeh 2026).



Gambar II.5 Gambaran umum *pipeline* Graph-RAG: ekstraksi triple, penelusuran subgraf (*sub-graph traversal*), serta integrasi hibrida antara representasi vektor dan simbolik.(Kamalipour, Asadi, dan Chimeh 2026)

Pertama, *Multi-hop Traversal*, yaitu penelusuran rantai relasi antar entitas untuk membangun konteks yang lebih lengkap. Kedua, *Subgraph Extraction*, yaitu pengambilan subgraf lokal di sekitar entitas yang relevan. Ketiga, *Community Detection*, yaitu identifikasi kelompok entitas yang saling terhubung untuk mendukung pertanyaan global atau proses ringkasan. Pendekatan ini memungkinkan sistem menangkap hubungan semantik yang tidak dapat direpresentasikan hanya melalui kemiripan vektor.

II.5.2.3 Integrasi Konteks Berbasis Struktur.

Subgraf hasil proses *retrieval* selanjutnya dikonversi menjadi representasi tekstual yang tetap mempertahankan hubungan antar entitas. Representasi tersebut disusun menjadi *structured prompt* sebelum dikirimkan ke LLM.

Dengan mempertahankan struktur relasional, LLM memperoleh konteks yang lebih kaya dibandingkan ketika hanya menerima potongan teks tidak terstruktur. Akibatnya, jawaban yang dihasilkan menjadi lebih konsisten, dapat ditelusuri, dan berpotensi mengurangi halusinasi.

II.5.2.4 Perbandingan Kuantitatif: Vector RAG dan GraphRAG

Perbedaan utama antara Vector RAG dan GraphRAG terletak pada mekanisme pengambilan informasi. Vector RAG mengandalkan pencarian berbasis kemiripan vektor, sedangkan GraphRAG memanfaatkan struktur relasional melalui traversal graf.

Ringkasan perbandingan keduanya disajikan pada Tabel II.1.

Tabel II.1 Perbandingan *trade-off* antara Vector RAG dan GraphRAG berdasarkan dimensi teknis yang relevan untuk pengembangan *framework*. Bersumber dari Kamalipour, Asadi, dan Chimeh (2026), Tabel 8.

Dimensi	Vector RAG	GraphRAG
<i>Memory footprint</i>	Gigabyte vektor padat; perlu <i>re-embedding</i> seluruh korpus saat ada perubahan	Lebih ringan: simpul + sisi (opsional <i>node embedding</i>); pembaruan cukup dengan menyisipkan tripel baru
Inti <i>retrieval</i>	ANN berbasis kemiripan (HNSW, IVF-PQ, <i>k</i> -NN)	Traversal graf multi-hop (BFS, <i>Greedy-Prune</i> , SPARQL)
<i>Recall</i> vs presisi	<i>Recall</i> tinggi, presisi lebih rendah; memerlukan <i>reranking</i> atau kompresi tambahan	Presisi lebih tinggi pada kueri relasional; <i>recall</i> turun jika <i>entity linking</i> gagal
<i>Explainability</i>	Sulit: hanya skor kemiripan yang tersedia	Jalur penuh: entitas → relasi → entitas; dapat diaudit untuk domain teregulasi
Profil latensi	Datar terhadap panjang jalur; <100 ms dengan HNSW	Meningkat seiring jumlah <i>hop</i> ; umumnya $\sim 3 \times$ Vector untuk 3- <i>hop</i>
<i>Benchmark</i> ung- gulan	BEIR, NaturalQuestions (<i>single-hop</i> QA)	HotpotQA, MultiHop-RAG, RobustQA (kueri kompleks multi- <i>hop</i>)
Kegagalan tipikal	Potongan relevan secara topik tetapi tidak relevan secara konteks → halusinasi	Simpul antara yang hilang → kesenjangan jawaban
Tepat digunakan saat	Throughput tinggi, pencarian <i>single-hop</i> , latensi kritis	Penalaran multi- <i>hop</i> , auditabilitas, pembaruan terstruktur berkala

Kamalipour, Asadi, dan Chimeh (2026) menunjukkan bahwa GraphRAG memberikan keunggulan yang lebih nyata pada pertanyaan kompleks yang memerlukan penalaran multi-hop. Pada beberapa eksperimen di domain keuangan, kesehatan, dan hukum, penambahan *knowledge graph* meningkatkan proporsi jawaban benar sekitar 30% dibandingkan pendekatan berbasis pengambilan vektor saja.

Meskipun demikian, GraphRAG juga memiliki keterbatasan. Kualitas hasil sangat bergantung pada akurasi konstruksi graf dan keberhasilan proses *entity linking*. Kesalahan pada tahap ini dapat menyebabkan hilangnya jalur penalaran sehingga informasi penting tidak ikut diambil.

II.5.3 Hybrid Retrieval pada GraphRAG: Penalaran Simbolik dan Semantik

Salah satu komponen utama dalam GraphRAG adalah mekanisme *hybrid retrieval*, yaitu pendekatan pengambilan informasi yang mengombinasikan dua paradigma

berbeda, yakni *semantic retrieval* berbasis vektor dan *symbolic retrieval* berbasis traversal graf. Kombinasi kedua pendekatan tersebut menjadi penting karena masing-masing memiliki karakteristik, kelebihan, dan keterbatasan yang saling melengkapi. Pemahaman terhadap hubungan keduanya menjadi landasan konseptual utama dalam pendekatan yang diusulkan pada penelitian ini.

II.5.3.1 Semantic Retrieval: Keunggulan dan Keterbatasan

Semantic retrieval atau *dense retrieval* bekerja dengan mengukur kemiripan semantik antara kueri dan dokumen menggunakan representasi *embedding* dalam ruang vektor kontinu. Pendekatan ini unggul dalam menemukan informasi yang relevan secara makna meskipun menggunakan istilah atau struktur kalimat yang berbeda. Selain itu, metode ini memiliki skalabilitas yang baik untuk korpus data berukuran besar.

Meskipun demikian, pendekatan berbasis vektor tidak merepresentasikan hubungan eksplisit antar entitas. Akibatnya, metode ini kurang efektif untuk menjawab pertanyaan yang memerlukan penalaran relasional atau hubungan bertahap antar konsep (Kamalipour, Asadi, dan Chimeh 2026). Sebagai contoh, kueri seperti “*mengapa pandas.merge() menghasilkan NaN*” tidak hanya memerlukan kemiripan teks, tetapi juga pemahaman terhadap hubungan kausal antar operasi dan struktur data.

II.5.3.2 Symbolic Retrieval: Keunggulan dan Keterbatasan

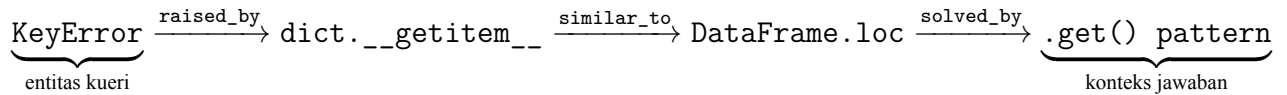
Berbeda dengan *semantic retrieval*, *symbolic retrieval* bekerja menggunakan representasi pengetahuan yang eksplisit dalam bentuk graf. Pendekatan ini memanfaatkan proses *graph traversal* untuk mengikuti hubungan antar entitas secara terstruktur. Keunggulan utama pendekatan ini adalah kemampuannya dalam melakukan penalaran *multi-hop*, yaitu menelusuri hubungan bertingkat dari satu entitas ke entitas lain (Wang dkk. 2026).

Traversal berbobot juga memungkinkan sistem memprioritaskan jalur tertentu berdasarkan atribut pada simpul maupun sisi graf, misalnya tingkat kepercayaan, reputasi komunitas, atau frekuensi kemunculan relasi. Namun, pendekatan ini memiliki keterbatasan karena hanya dapat menjangkau entitas yang memang terhubung secara eksplisit di dalam graf. Informasi yang relevan tetapi tidak memiliki hubungan struktural langsung berpotensi tidak ditemukan.

II.5.3.3 Multi-hop Reasoning sebagai Justifikasi Pendekatan Hibrid

Permasalahan teknis dalam domain pemrograman umumnya memerlukan penalaran bertahap yang tidak dapat diselesaikan hanya dengan satu mekanisme retrieval.

Sebagai contoh:



Rantai penalaran tersebut menunjukkan bahwa sistem perlu mengikuti hubungan eksplisit antar entitas melalui *graph traversal*, sekaligus menemukan konsep lain yang serupa secara semantik meskipun tidak terhubung langsung di dalam graf. Dalam contoh tersebut, relasi antara `dict.__getitem__` dan `DataFrame.loc` lebih mudah ditemukan melalui pencarian semantik dibandingkan hubungan struktural eksplisit.

II.5.3.4 Trade-off antara Presisi Retrieval dan Cakupan Kontekstual

Kamalipour, Asadi, dan Chimeh (2026) menjelaskan bahwa terdapat trade-off mendasar antara pendekatan simbolik dan semantik. *Symbolic retrieval* cenderung memberikan presisi yang lebih tinggi karena hubungan antar entitas bersifat eksplisit dan dapat ditelusuri. Namun, cakupan konteksnya relatif terbatas. Sebaliknya, *semantic retrieval* memiliki cakupan informasi yang lebih luas, tetapi presisinya dapat menurun pada pertanyaan yang memerlukan hubungan relasional yang kompleks.

GraphRAG mengintegrasikan kedua pendekatan tersebut melalui beberapa tahap utama sebagai berikut:

1. *Entity anchoring*, yaitu mengidentifikasi entitas pada kueri melalui mekanisme NER dan *entity linking* untuk menentukan simpul awal pada KG,
2. *Graph traversal*, yaitu membangun subgraf kontekstual melalui penelusuran relasi secara bertahap dan berbobot,
3. *Semantic expansion*, yaitu menggunakan simpul pada subgraf sebagai acuan pencarian vektor untuk menemukan informasi relevan lain yang tidak terhubung secara langsung dalam graf.

Kombinasi tersebut memungkinkan sistem mempertahankan keterlacakan (*traceability*) yang dihasilkan oleh traversal graf sekaligus memanfaatkan kemampuan *dense retrieval* dalam menemukan informasi relevan di luar hubungan eksplisit. Dengan demikian, GraphRAG tidak hanya menggabungkan RAG dengan *knowledge graph*, tetapi mengintegrasikan penalaran simbolik dan semantik secara langsung pada mekanisme retrieval.

II.5.4 Hasil Penelitian tentang Efektivitas GraphRAG

Berbagai penelitian menunjukkan bahwa integrasi *knowledge graph* (KG) ke dalam sistem *Retrieval-Augmented Generation* (RAG) mampu meningkatkan akurasi jawaban sekaligus mengurangi tingkat halusinasi pada LLM. Secara umum, studi-studi tersebut memperlihatkan bahwa KG tidak hanya membantu proses retrieval menjadi lebih relevan, tetapi juga menjaga agar proses generasi tetap terikat pada fakta yang dapat diverifikasi. Temuan ini menjadi penting untuk domain pemrograman yang menuntut ketepatan informasi dan kejelasan hubungan antar konsep teknis.

II.5.4.1 Pengurangan Halusinasi melalui Grounded Generation

Studi Pythia-RAG (Ali dkk. 2026) membangun *Multimodal Knowledge Graph* (MM-KG) dari sumber tekstual dan visual, kemudian mengambil subgraf yang relevan menggunakan algoritma *Prize-Collecting Steiner Tree*. Evaluasi pada dataset ScienceQA dan MultiModalQA menunjukkan peningkatan akurasi masing-masing sebesar 5,4% dan 4,8%.

Temuan yang paling relevan dalam penelitian tersebut adalah penurunan *hallucination rate* dari 17,6% pada GPT-4o menjadi 8,3%. Penurunan ini menunjukkan bahwa KG dapat berfungsi sebagai mekanisme *grounding* yang menjaga agar jawaban LLM tetap sesuai dengan fakta yang tersedia.

Hasil serupa juga ditemukan pada sistem IC-HRAG (Zhou dkk. 2026). Sistem tersebut mencapai akurasi 90,57% ketika LLM dikombinasikan dengan KG terstruktur, lebih tinggi dibandingkan RAG konvensional yang memperoleh 86,95% maupun LLM tanpa pengetahuan eksternal yang hanya mencapai 55,79%. Temuan ini menunjukkan bahwa kualitas dan struktur pengetahuan yang diberikan kepada LLM memiliki pengaruh besar terhadap kualitas jawaban yang dihasilkan.

II.5.4.2 Kemampuan *Multi-hop Reasoning* pada Pertanyaan Relasional

Studi PRIME (Wang dkk. 2026) mengembangkan mekanisme penalaran bertahap berbasis subgraf yang memungkinkan sistem menelusuri hubungan antar entitas secara progresif. Metode tersebut memperoleh performa 91,6% Hit@1 pada WebQSP dan 78,3% F1 pada CWQ tanpa memerlukan pelatihan tambahan. Temuan ini menunjukkan bahwa traversal pada KG dapat digunakan untuk melakukan *multi-hop reasoning* secara efisien. Dalam domain pemrograman, kemampuan tersebut penting karena pertanyaan teknis sering kali memerlukan beberapa tahap penalaran, misalnya mulai dari identifikasi kesalahan, penelusuran penyebab, hingga penemuan

solusi yang sesuai.

II.5.4.3 Knowledge Graph sebagai Sumber Pengetahuan Otoritatif

Studi LARK (Khalifa, Messaoud, dan Mkaouer 2026) menerapkan GraphRAG dengan mekanisme *dual-constraint* pada analisis lisensi perangkat lunak. Domain tersebut memiliki karakteristik yang mirip dengan domain pemrograman karena pengetahuan bersifat eksplisit, terstruktur, dan dapat diverifikasi secara objektif. Evaluasi terhadap sekitar 4.000 proyek Python sumber terbuka menunjukkan akurasi deteksi lisensi sebesar 98,1% dan nilai F1 kompatibilitas sebesar 96,2%, dengan tingkat halusinasi hanya 2,1%. Rendahnya halusinasi dicapai karena KG digunakan sebagai sumber pengetahuan utama, sementara LLM hanya berperan dalam memahami kueri dan menyusun penjelasan. Pendekatan tersebut relevan dengan penelitian ini karena menunjukkan bahwa pemisahan peran antara KG dan LLM dapat meningkatkan keandalan jawaban. Dalam penelitian ini, Stack Overflow diposisikan sebagai sumber pengetahuan utama yang telah melalui proses validasi komunitas.

II.5.5 Aplikasi GraphRAG pada Domain Pemrograman dan Rekayasa Perangkat Lunak

Selain efektif pada berbagai domain teknis, GraphRAG juga mulai diterapkan secara khusus pada bidang pemrograman dan rekayasa perangkat lunak. Domain ini sangat cocok untuk pendekatan berbasis KG karena memiliki hubungan antar entitas yang jelas, struktur pengetahuan yang hierarkis, serta jawaban yang dapat diverifikasi secara objektif.

II.5.5.1 Penyelesaian Kode Berbasis Graf Dependensi

Studi GraphCoder (Kamalipour, Asadi, dan Chimeh 2026) menerapkan GraphRAG untuk meningkatkan kualitas *code completion* pada tingkat repositori. Sistem tersebut membangun *Code Context Graph* (CCG), yaitu representasi graf yang memodelkan hubungan *control flow*, dependensi data, dan pemanggilan fungsi antar komponen kode. Pendekatan ini menggunakan dua tahap retrieval. Tahap pertama melakukan penyaringan awal untuk menemukan kandidat potongan kode yang relevan, sedangkan tahap kedua melakukan *re-ranking* berdasarkan hubungan dependensi dalam graf.

Hasil penelitian menunjukkan bahwa pendekatan berbasis graf menghasilkan kode yang lebih tepat secara semantik dibandingkan pendekatan berbasis kemiripan teks saja. Temuan ini menunjukkan bahwa hubungan struktural antar entitas kode memiliki peran penting dalam menghasilkan jawaban yang akurat pada domain pemrograman.

II.5.5.2 Generasi SQL dan Penalaran Relasional

Studi IC-HRAG (Zhou dkk. 2026) menunjukkan bagaimana KG domain-spesifik dapat meningkatkan performa tugas *text-to-SQL generation*. Sistem tersebut membangun KG dari skema basis data dan dokumentasi domain, kemudian menggunakan graf tersebut untuk mengambil konteks relasional yang relevan ketika kueri bahasa alami diterjemahkan menjadi SQL. Hasil evaluasi menunjukkan bahwa sistem mencapai akurasi 90,57% ketika menggunakan kombinasi LLM dan KG. Peningkatan performa terutama terlihat pada kueri yang memerlukan penalaran terhadap relasi antar tabel. Kondisi ini memiliki kesamaan dengan domain pemrograman Stack Overflow yang juga membutuhkan penalaran atas hubungan antar entitas teknis.

II.5.5.3 Analisis Lisensi sebagai Representasi Domain Teknis

Sistem LARK (Khalifa, Messaoud, dan Mkaouer 2026) juga menunjukkan bahwa arsitektur GraphRAG efektif digunakan pada domain teknis yang memiliki struktur pengetahuan eksplisit. Dalam sistem tersebut, hubungan antar entitas seperti lisensi, pustaka, dan proyek direpresentasikan dalam bentuk relasi yang jelas di dalam KG. Karakteristik tersebut serupa dengan domain Stack Overflow yang memiliki hubungan eksplisit antara API, error, solusi, dan pustaka perangkat lunak. Keberhasilan LARK dalam menurunkan tingkat halusinasi menunjukkan bahwa KG dapat berfungsi sebagai lapisan kendali (*constraint layer*) yang menjaga konsistensi jawaban LLM.

II.5.5.4 Implikasi terhadap Penelitian Berbasis Stack Overflow

Secara umum, penelitian-penelitian sebelumnya menunjukkan bahwa GraphRAG memberikan manfaat paling besar pada domain yang memiliki tiga karakteristik utama. Pertama, pengetahuan dapat direpresentasikan sebagai hubungan eksplisit antar entitas. Kedua, pertanyaan memerlukan penalaran bertahap untuk menemukan jawaban. Ketiga, jawaban dapat diverifikasi secara objektif.

Ketiga karakteristik tersebut sesuai dengan domain pemrograman pada Stack Overflow. Namun, penelitian ini menambahkan aspek yang belum banyak dibahas pada studi sebelumnya, yaitu penggunaan sinyal validasi komunitas sebagai bobot pada KG. Dengan pendekatan tersebut, simpul dan relasi dalam graf tidak hanya merepresentasikan pengetahuan teknis, tetapi juga tingkat kepercayaan kolektif dari komunitas pengembang perangkat lunak.

II.5.6 Sintesis dan Posisi Penelitian

Berdasarkan kajian literatur, Vector RAG memiliki keunggulan pada skalabilitas dan kemampuan pengambilan informasi berbasis topik. Namun, pendekatan ini masih terbatas dalam menangani hubungan relasional dan penalaran multi-hop.

Sebaliknya, GraphRAG menyediakan struktur pengetahuan yang dapat dijelaskan, mendukung penalaran bertingkat, dan menghasilkan konteks yang lebih kaya. Kelemahannya terletak pada kebutuhan konstruksi KG dan sensitivitas terhadap kesalahan *entity linking*.

Penelitian ini memposisikan diri sebagai pengembangan lebih lanjut dari GraphRAG dengan menambahkan sinyal kepercayaan komunitas Stack Overflow sebagai bobot pada *knowledge graph*. Pendekatan tersebut diharapkan mampu meningkatkan kualitas *retrieval*, memperkuat proses *grounding*, dan mengurangi halusinasi pada LLM untuk domain pemrograman.

II.6 Stack Overflow sebagai Basis Pengetahuan Tervalidasi Sosial

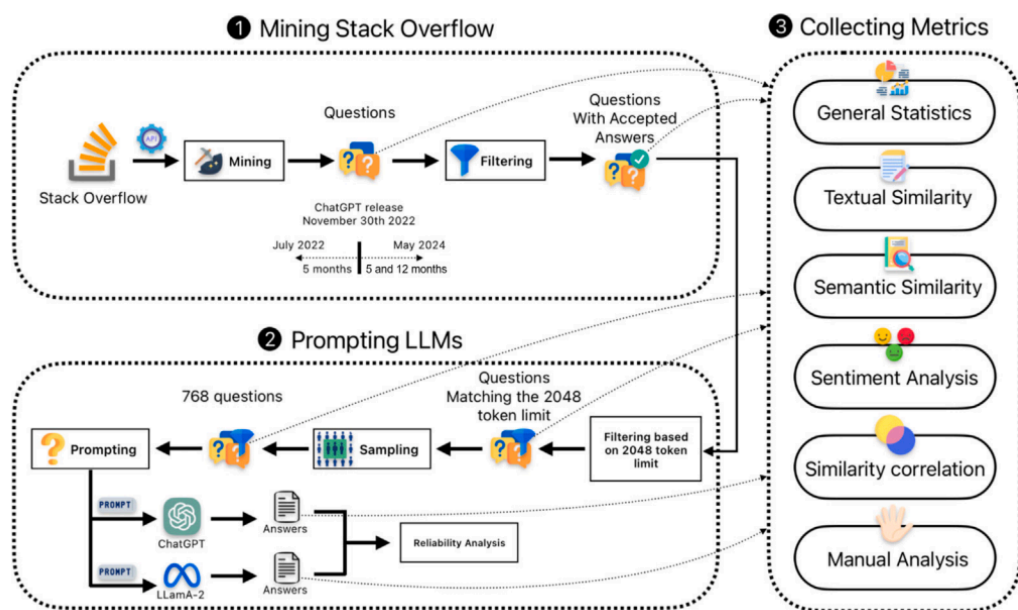
II.6.1 Struktur dan Mekanisme Kurasi Komunitas

Stack Overflow adalah platform *question and answer* (Q&A) yang berfokus pada pemrograman dan pengembangan perangkat lunak. Per 2024, platform ini menyimpan lebih dari 58 juta pasang pertanyaan dan jawaban (Silva, Samhi, dan Khomh 2025). Yang membedakan Stack Overflow dari repositori teks biasa adalah mekanisme kurasi sosial bawaan yang menghasilkan sinyal kualitas terukur dengan sistem *upvote/downvote*, status *accepted answer*, dan sistem reputasi kontributor.

Silva, Samhi, dan Khomh (2025) merancang studi empiris yang secara eksplisit memanfaatkan tiga mekanisme ini sebagai proksi kualitas jawaban (Gambar II.6). Studi tersebut mengonfirmasi bahwa jawaban dengan skor tinggi dan status *accepted* memiliki korelasi positif dengan akurasi teknis yang terverifikasi. Hal ini menjadikan Stack Overflow sebagai basis pengetahuan tervalidasi secara sosial (*socially-validated knowledge base*).

II.6.2 Sinyal Kepercayaan sebagai Proksi Kualitas

Dalam konteks *knowledge graph* berbobot, sinyal kepercayaan yang tersedia pada Stack Overflow dapat dimanfaatkan sebagai bobot pada simpul dan relasi graf. Dengan demikian, kualitas informasi tidak hanya ditentukan oleh keberadaan entitas dan hubungan, tetapi juga oleh tingkat kepercayaan komunitas terhadap informasi tersebut. Tabel II.2 menunjukkan sinyal kepercayaan yang digunakan beserta interp-



Gambar II.6 Gambaran umum rancangan empiris yang digunakan untuk menguji korelasi antara sinyal kepercayaan komunitas Stack Overflow dengan akurasi jawaban LLM. (Silva, Samhi, dan Khomh 2025)

retasinya pada KG berbobot.

Tabel II.2 Sinyal kepercayaan komunitas Stack Overflow dan interpretasinya sebagai bobot pada *knowledge graph* berbobot

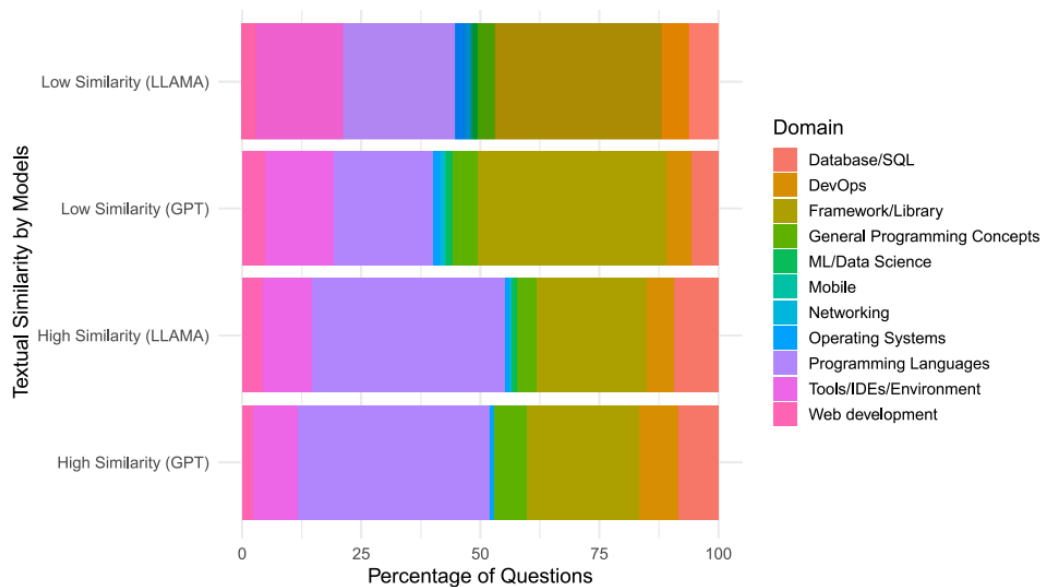
Sinyal	Sumber	Interpretasi sebagai Bobot
Skor jawaban	Akumulasi <i>upvote</i> – <i>downvote</i>	Bobot simpul jawaban: penerimaan kolektif komunitas
Status <i>accepted answer</i>	Pilihan eksplisit penanya	Bobot biner: validasi langsung oleh praktisi
Reputasi kontributor	Rekam jejak kontribusi historis	Bobot kepercayaan pada sisi kontribusi
Skor pertanyaan	Akumulasi <i>upvote</i> pada pertanyaan	Bobot simpul pertanyaan: relevansi masalah
Jumlah tampilan (<i>view count</i>)	Aktivitas baca pengguna	Bobot popularitas: relevansi jangka panjang

II.6.3 Pemanfaatan Konten Stack Overflow dalam Sistem AI

Beberapa penelitian telah memanfaatkan Stack Overflow sebagai sumber pengetahuan untuk meningkatkan kualitas keluaran model AI. Nashid, Sintaha, dan Mesbah (2023) menunjukkan bahwa penggunaan konten Stack Overflow sebagai sumber konteks dapat meningkatkan kualitas jawaban LLM pada tugas pemrograman. Namun, pendekatan tersebut masih menggunakan RAG konvensional berbasis kemiripan vektor sehingga belum memanfaatkan struktur relasional maupun sinyal

kepercayaan komunitas.

Selain itu, Silva, Samhi, dan Khomh (2025) menunjukkan bahwa tingkat kesesuaian antara jawaban LLM dan jawaban komunitas tidak sama pada semua domain pemrograman (Gambar II.7). Beberapa domain menghasilkan kemiripan tinggi, sementara domain lain menunjukkan perbedaan yang cukup besar. Temuan ini mengindikasikan bahwa halusinasi LLM bersifat tidak merata antar domain dan perlu dipertimbangkan pada proses evaluasi.



Gambar II.7 Distribusi domain pertanyaan untuk jawaban LLM dengan kemiripan tinggi dan rendah terhadap komunitas Stack Overflow. (Silva, Samhi, dan Khomh 2025)

Temuan tersebut diperkuat oleh Kabir dkk. (2024), yang melakukan studi terhadap jawaban ChatGPT pada pertanyaan Stack Overflow. Penelitian ini menunjukkan bahwa keluaran LLM masih mengandung ketidakakuratan faktual yang cukup signifikan sehingga aspek keandalan perlu menjadi perhatian utama pada penerapan sistem berbasis LLM.

II.6.4 *Stack Overflow Recommendations Dataset (SORD)* sebagai Sumber Data Penelitian

Penelitian ini menggunakan *Stack Overflow Recommendations Dataset (SORD)* (Fatima dan Maqbool 2026), yaitu dataset berskala besar yang dikurasi secara empiris dari diskusi Stack Overflow dengan penekanan pada penambahan wawasan terkait rekomendasi perangkat lunak antar-pengembang. Dataset ini dipublikasikan secara terbuka (*open access*) pada jurnal *Data in Brief* dan mencakup empat jenis entitas

utama: (1) pertanyaan (*question*) beserta judul, isi, skor, jumlah tampilan, dan tanggal pembuatan; (2) jawaban (*answer*) beserta isi, skor, status *accepted answer*, dan reputasi kontributor; (3) tag teknologi yang merepresentasikan domain atau pustaka yang dirujuk; serta (4) komentar yang mengandung konteks diskusi tambahan antar pengembang.

Pemilihan SORD sebagai sumber data penelitian ini didasarkan pada tiga pertimbangan utama. Pertama, SORD dibangun dari *Stack Exchange Data Dump* publik sehingga seluruh metadata sinyal kepercayaan komunitas, skor *upvote*, status *accepted answer*, dan reputasi kontributor tersedia sebagai atribut terstruktur yang melekat pada setiap entitas. Berbeda dengan pendekatan Nashid, Sintaha, dan Mesbah (2023) yang mengindeks konten Stack Overflow sebagai potongan teks datar (*flat chunks*) ke dalam *vector store*, SORD memungkinkan hubungan antar entitas direpresentasikan secara eksplisit sebagai sisi berbobot pada *knowledge graph*. Kedua, SORD bersifat publik dan tersedia secara terbuka, sehingga mendukung reproduibilitas hasil evaluasi dan perbandingan langsung dengan studi-studi sebelumnya yang menggunakan sumber data serupa (Silva, Samhi, dan Khomh 2025). Ketiga, cakupan SORD yang mencakup pertanyaan, jawaban, komentar, dan tag secara relasional menjadikannya cocok sebagai fondasi pembangunan *knowledge graph* domain pemrograman yang kaya secara struktural.

Karakteristik pembeda utama SORD terhadap dataset Stack Overflow pada penelitian sebelumnya terletak pada ketersediaan sinyal kepercayaan komunitas sebagai metadata terstruktur. Sinyal-sinyal tersebut tidak dibuang sebagai *noise*, melainkan dipertahankan dan dioperasionalisasikan sebagai bobot eksplisit pada simpul dan sisi *knowledge graph* sebagaimana diuraikan pada Tabel II.2. Dengan demikian, SORD menjadi fondasi empiris yang memungkinkan penelitian ini menjawab kesenjangan G2 (pemanfaatan sinyal kepercayaan komunitas) dan G3 (penerapan GraphRAG pada sumber pengetahuan komunitas teknis bervalidasi sosial) yang akan diidentifikasi pada Seksi ??.

II.7 Grounded Generation dan Faithfulness pada LLM

II.7.1 Definisi dan Motivasi

Pembahasan mengenai halusinasi pada bagian sebelumnya menunjukkan bahwa permasalahan utama LLM tidak hanya terletak pada kemampuan menghasilkan informasi yang salah, tetapi juga pada ketidakmampuan sistem menjelaskan asal-usul informasi yang dihasilkan. Dalam banyak kasus, model dapat menghasilkan jawaban yang terlihat meyakinkan, tetapi tidak memiliki sumber yang jelas dan sulit di-

verifikasi. Untuk mengatasi permasalahan tersebut, konsep *faithful generation* dan *grounded generation* dikembangkan sebagai pendekatan untuk memastikan bahwa keluaran LLM tetap konsisten dengan sumber pengetahuan yang digunakan.

Faithful generation mengacu pada kondisi ketika jawaban yang dihasilkan LLM sesuai dan konsisten dengan informasi yang diberikan melalui konteks (Ji dkk. 2023). Sebuah jawaban disebut *faithful* apabila setiap klaim yang dihasilkan model didukung oleh informasi yang terdapat dalam konteks yang diberikan, bukan semata-mata berasal dari pengetahuan parametrik model yang tidak dapat diverifikasi. Huang dkk. (2025) membedakan dua jenis halusinasi, yaitu *faithfulness hallucination* dan *factual hallucination*. *Faithfulness hallucination* terjadi ketika jawaban tidak sesuai dengan konteks yang diberikan, sedangkan *factual hallucination* terjadi ketika model menghasilkan fakta yang salah meskipun dapat diverifikasi secara independen.

Grounded generation merupakan pengembangan lebih lanjut dari konsep *faithful generation*. Pada pendekatan ini, setiap klaim dalam jawaban tidak hanya harus konsisten dengan konteks, tetapi juga harus dapat ditelusuri kembali ke sumber pengetahuan tertentu (Khalifa, Messaoud, dan Mkaouer 2026). Dengan demikian, sistem tidak hanya menghasilkan jawaban yang relevan, tetapi juga menyediakan dasar yang jelas mengenai dari mana informasi tersebut diperoleh.

II.7.2 Provenance sebagai Mekanisme Keterlacakan

Dalam sistem berbasis *knowledge graph* (KG), konsep *grounded generation* diwujudkan melalui mekanisme *provenance*. *Provenance* merupakan kemampuan sistem untuk menyimpan dan menampilkan informasi mengenai asal-usul fakta yang digunakan dalam proses pembentukan jawaban (Ali dkk. 2026). Pada GraphRAG, mekanisme ini dapat diimplementasikan dengan menyertakan referensi ke simpul dan relasi pada KG yang menjadi sumber konteks bagi jawaban yang dihasilkan.

Mekanisme *provenance* memberikan beberapa manfaat penting dalam penelitian ini. Pertama, setiap klaim pada jawaban dapat diverifikasi kembali dengan menelusuri sumbernya pada KG. Kedua, pada KG berbobot berbasis Stack Overflow, *provenance* juga membawa informasi tingkat kepercayaan komunitas. Sebagai contoh, jawaban yang berasal dari posting dengan skor tinggi atau berstatus *accepted answer* dapat dianggap lebih terpercaya dibandingkan sumber yang tidak memiliki validasi komunitas. Ketiga, *provenance* meningkatkan *explainability* sistem karena pengguna dapat memahami alasan mengapa suatu jawaban dihasilkan berdasarkan sumber tertentu.

II.7.3 *Dual-Constraint* sebagai Mekanisme *Grounded Generation*

Untuk memastikan bahwa jawaban yang dihasilkan tetap *grounded*, penelitian ini mengadopsi mekanisme *dual-constraint* yang terdiri atas dua lapisan kendali yang bekerja secara bersamaan (Khalifa, Messaoud, dan Mkaouer 2026).

Lapisan pertama adalah kendala berbasis KG (*graph-based constraint*). Pada tahap ini, LLM diarahkan untuk menghasilkan jawaban sesuai fakta yang direpresentasikan oleh subgraf yang diberikan. Setiap tripel pada subgraf direpresentasikan dalam bentuk pasangan (Entitas--Relasi--Entitas) + segmen teks sumber untuk memperjelas batas faktual yang harus diikuti model.

Lapisan kedua adalah mekanisme *RAG grounding*, yaitu proses yang menghubungkan jawaban dengan segmen teks sumber yang dapat diverifikasi dari Stack Overflow. Melalui kombinasi kedua lapisan tersebut, sistem mendorong LLM agar tetap menghasilkan jawaban sesuai konteks pengetahuan yang tersedia, sekaligus menyediakan jalur *provenance* yang dapat ditelusuri kembali ke simpul KG berbasis Stack Overflow (Ali dkk. 2026).

II.8 Evaluasi Keandalan *Large Language Model*

II.8.1 Tantangan Evaluasi pada Sistem GraphRAG

Evaluasi keandalan *Large Language Model* (LLM), khususnya pada sistem berbasis GraphRAG, merupakan proses yang kompleks karena melibatkan dua komponen utama yang saling berkaitan, yaitu kualitas pengambilan informasi (*retrieval*) dan kualitas generasi jawaban. Evaluasi tidak hanya bertujuan menilai benar atau salahnya jawaban, tetapi juga mengidentifikasi sumber kesalahan dalam sistem, misalnya apakah kesalahan berasal dari proses *retrieval*, *entity linking*, atau proses generasi yang tidak *grounded* (Kamalipour, Asadi, dan Chimeh 2026).

Evaluasi berbasis anotasi manusia umumnya menghasilkan penilaian yang lebih akurat karena melibatkan pertimbangan pakar. Namun, pendekatan ini sulit diterapkan dalam skala besar karena membutuhkan waktu dan biaya yang tinggi. Sebaliknya, evaluasi berbasis *benchmark* lebih mudah diskalakan, tetapi memiliki risiko kontaminasi data apabila dataset evaluasi pernah digunakan pada proses pelatihan model (Gu, Jiang, Shi, dkk. 2026).

Pada domain pemrograman, proses evaluasi memiliki karakteristik yang berbeda dibandingkan domain umum. Jawaban teknis umumnya dapat diverifikasi secara objektif karena kode yang benar dapat dijalankan, sedangkan kode yang salah akan

menghasilkan kesalahan atau gagal dikompilasi. Selain itu, Stack Overflow menyediakan referensi jawaban yang telah divalidasi komunitas melalui mekanisme *upvote* dan *accepted answer*. Karakteristik tersebut menjadikan Stack Overflow sebagai sumber *ground truth* yang relatif reliabel untuk mengevaluasi kualitas jawaban sistem (Silva, Samhi, dan Khomh 2025).

II.8.2 Dimensi Evaluasi pada Sistem GraphRAG

Evaluasi sistem GraphRAG perlu dilakukan secara berlapis karena kualitas sistem tidak dapat diukur hanya dari satu aspek. Secara umum, terdapat empat dimensi utama yang digunakan untuk mengevaluasi sistem GraphRAG (Kamalipour, Asadi, dan Chimeh 2026).

1. Relevansi Retrieval (*Retrieval Relevance*)

Dimensi ini mengukur kemampuan sistem dalam mengambil informasi yang relevan terhadap pertanyaan pengguna. Pada GraphRAG berbasis Stack Overflow, evaluasi dilakukan dengan menilai sejauh mana subgraf yang diambil memuat entitas dan relasi yang sesuai dengan kueri pemrograman.

Metrik yang umum digunakan meliputi Precision@K dan Recall@K. Precision@K mengukur proporsi dokumen relevan dari sejumlah K hasil teratas yang diambil sistem, sedangkan Recall@K mengukur seberapa banyak informasi relevan yang berhasil ditemukan dari seluruh informasi yang tersedia. Selain itu, metrik seperti *Mean Reciprocal Rank* (MRR) dan *Normalized Discounted Cumulative Gain* (nDCG) digunakan untuk mempertimbangkan kualitas peringkat hasil retrieval (Kamalipour, Asadi, dan Chimeh 2026).

2. Keterlacakan Konteks (*Groundedness* dan *Faithfulness*)

Dimensi ini mengevaluasi apakah jawaban yang dihasilkan benar-benar didukung oleh konteks yang diberikan kepada model. Jawaban yang memiliki tingkat *groundedness* tinggi berarti seluruh klaim dalam jawaban dapat ditelusuri kembali ke informasi yang terdapat pada KG atau dokumen sumber (Kamalipour, Asadi, dan Chimeh 2026).

Dalam penelitian ini, *faithfulness* diukur dengan membandingkan isi jawaban LLM terhadap triple dan konteks yang terdapat pada subgraf hasil retrieval. Semakin tinggi nilai *faithfulness*, semakin kecil kemungkinan model menghasilkan informasi yang tidak didukung oleh sumber yang diberikan.

3. Relevansi Jawaban (*Answer Relevance*)

Dimensi ini mengukur kesesuaian jawaban terhadap kebutuhan informasi pengguna. Jawaban dapat saja konsisten dengan konteks yang diberikan, tetapi tetap tidak relevan terhadap pertanyaan pengguna apabila retrieval yang dilakukan tidak tepat.

Dalam domain pemrograman, relevansi jawaban dapat dilihat dari sejauh mana solusi yang diberikan mampu menyelesaikan permasalahan teknis yang ditanyakan, misalnya melalui ketepatan penggunaan API, kelengkapan solusi, atau kemampuan menangani kesalahan yang muncul.

4. Kebenaran Faktual (*Factual Correctness*)

Dimensi ini mengukur kesesuaian jawaban terhadap fakta atau referensi yang diketahui benar. Dalam penelitian ini, jawaban Stack Overflow dengan skor tinggi dan status *accepted answer* digunakan sebagai referensi *ground truth*. Pada level kode, evaluasi faktual juga dapat dilakukan melalui pengujian otomatis, misalnya dengan menjalankan unit test atau memeriksa keberhasilan kompilasi kode. Pendekatan ini menjadi keunggulan evaluasi pada domain pemrograman karena kebenaran jawaban dapat diuji secara deterministik (Alomari dkk. 2026).

II.8.3 Kerangka RAGAS sebagai Protokol Evaluasi

Untuk mengukur kualitas sistem secara konsisten, penelitian ini menggunakan kerangka RAGAS (*Retrieval-Augmented Generation Assessment*). RAGAS merupakan kerangka evaluasi yang dirancang khusus untuk sistem RAG dan memungkinkan evaluasi dilakukan tanpa memerlukan jawaban referensi untuk setiap pertanyaan (Kamalipour, Asadi, dan Chimeh 2026). RAGAS menyediakan beberapa metrik utama yang sesuai dengan kebutuhan evaluasi pada penelitian ini. Tabel II.3 menunjukkan metrik-metrik tersebut beserta interpretasinya dalam konteks GraphRAG berbasis Stack Overflow. Nilai RAGAS yang mendekati 1,0 menunjukkan bahwa

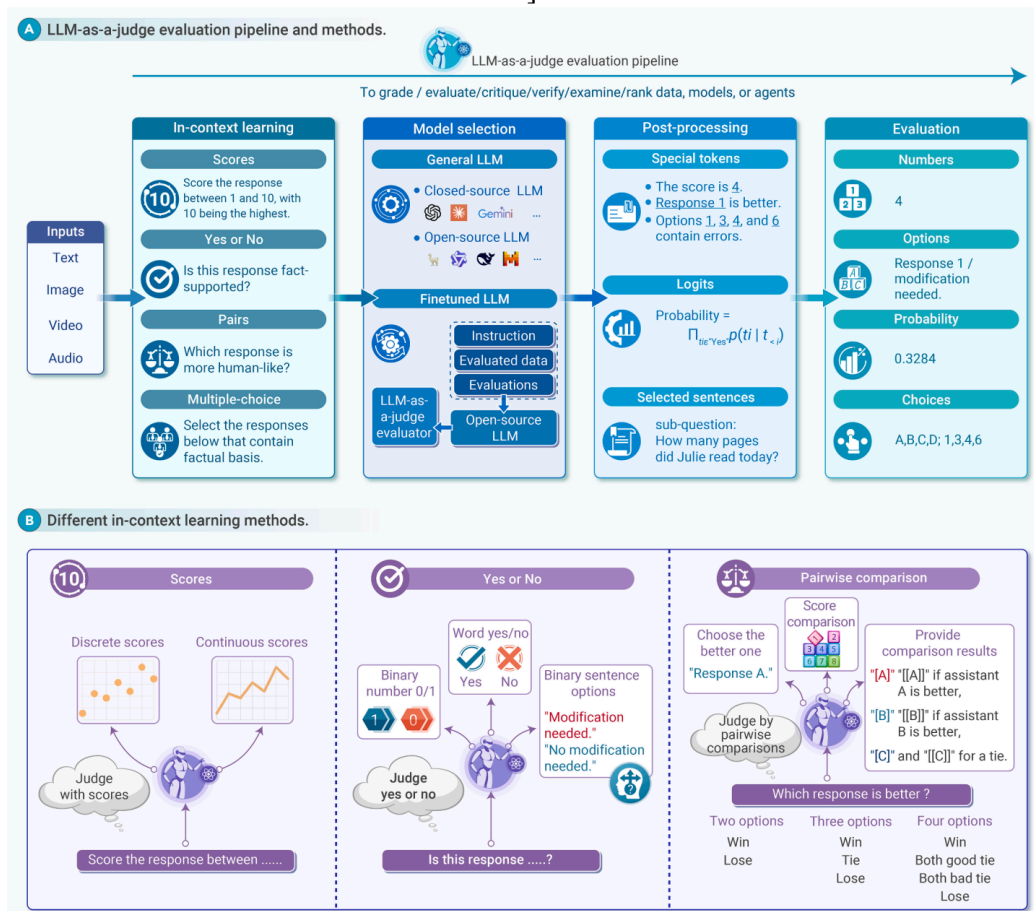
Tabel II.3 Metrik RAGAS dalam evaluasi GraphRAG berbasis Stack Overflow

Kategori	Metrik	Interpretasi
Retrieval	<i>context_relevancy</i>	Relevansi subgraf terhadap kueri pemrograman
Retrieval	<i>context_recall</i>	Kelengkapan informasi yang berhasil diambil
Generation	<i>faithfulness</i>	Tingkat keterlacakan jawaban terhadap KG
Generation	<i>answer_relevance</i>	Kesesuaian jawaban terhadap kebutuhan pengguna
Overall	<i>ragas_score</i>	Nilai keseluruhan kualitas sistem

sistem mampu mengambil konteks yang relevan sekaligus menghasilkan jawaban yang sesuai dan dapat diverifikasi. Dalam penelitian ini, evaluasi dilakukan terhadap tiga pendekatan, yaitu LLM murni, RAG konvensional, dan GraphRAG berbasis Stack Overflow, menggunakan protokol evaluasi yang sama agar hasil dapat dibandingkan secara adil.

II.8.4 Paradigma *LLM-as-a-Judge*

Evaluasi manual terhadap ratusan jawaban pemrograman memerlukan sumber daya yang besar. Oleh karena itu, penelitian ini memanfaatkan paradigma *LLM-as-a-Judge*, yaitu pendekatan yang menggunakan LLM sebagai penilai otomatis terhadap keluaran model lain (Gu, Jiang, Shi, dkk. 2026). Pendekatan ini memungkinkan evaluasi dilakukan secara lebih cepat dan skalabel, khususnya untuk menilai kualitas seperti *faithfulness*, relevansi jawaban, dan kegunaan solusi yang dihasilkan.



Gambar II.8 Tahapan evaluasi *LLM-as-a-Judge* yang meliputi pemberian konteks, inferensi model penilai, *post-processing*, dan agregasi skor (Gu, Jiang, Shi, dkk. 2026).

Secara umum, proses evaluasi *LLM-as-a-Judge* terdiri atas empat tahap, yaitu pemberian konteks dan instruksi evaluasi, inferensi model penilai, *post-processing*, dan agregasi skor hasil evaluasi. Untuk meningkatkan reliabilitas penilaian, beberapa strategi dapat diterapkan, seperti penggunaan rubrik evaluasi terstruktur, pendekatan *Chain-of-Thought*, dan kombinasi beberapa model penilai (*ensemble*) (Gu, Jiang, Shi, dkk. 2026).

Keandalan sistem penilai otomatis perlu dibandingkan dengan penilaian manusia menggunakan metrik seperti Cohen's κ dan korelasi Spearman. Nilai Cohen's $\kappa \geq 0,6$ umumnya dianggap menunjukkan tingkat kesepakatan yang baik antara penilai otomatis dan penilai manusia.

Dalam konteks penelitian ini, paradigma *LLM-as-a-Judge* menjadi relevan karena memungkinkan evaluasi *faithfulness* dilakukan pada skala besar. Selain itu, rubrik penilaian dapat disesuaikan dengan karakteristik domain pemrograman, misalnya dengan mengevaluasi ketepatan penggunaan API, kesesuaian versi pustaka, dan penanganan *exception*, yang merupakan sumber umum terjadinya halusinasi pada LLM (Alomari dkk. 2026).

II.9 Sintesis Penelitian Terkait dan *Research Gap*

II.9.1 Analisis Penelitian Terkait

Tinjauan terhadap penelitian terdahulu menunjukkan bahwa penelitian terkait dapat dikelompokkan ke dalam empat area utama, yaitu evaluasi kemampuan LLM pada domain pemrograman, pemanfaatan LLM untuk kualitas kode, penggunaan Stack Overflow sebagai sumber konteks, dan penerapan GraphRAG pada domain teknis. Tabel II.4 merangkum posisi masing-masing penelitian terhadap tiga dimensi utama dalam penelitian ini.

1. **Evaluasi Komparatif LLM dan Stack Overflow.** Silva, Samhi, dan Khomh (2025) dan Kabir dkk. (2024) menunjukkan bahwa LLM masih memiliki keterbatasan pada domain pemrograman jika dibandingkan dengan pengetahuan komunitas Stack Overflow. Hasil penelitian menunjukkan bahwa jawaban LLM masih sering mengandung ketidakakuratan teknis dan belum mampu melampaui kualitas solusi yang divalidasi komunitas. Meskipun demikian, kedua penelitian tersebut hanya berfokus pada evaluasi performa dan belum menawarkan mekanisme integrasi pengetahuan komunitas ke dalam sistem LLM.
2. **LLM untuk Kualitas Kode.** Penelitian oleh Alomari dkk. (2026) dan Jiang dkk. (2026) menunjukkan bahwa LLM rentan menghasilkan halusinasi dan jawaban yang tidak konsisten pada tugas rekayasa perangkat lunak, seperti refaktorisasi dan deteksi duplikasi kode. Strategi yang digunakan masih berfokus pada *prompting* dan *fine-tuning*. Hingga saat ini, belum banyak penelitian yang memanfaatkan GraphRAG maupun pengetahuan komunitas tervalidasi sebagai mekanisme *grounding* untuk mengurangi halusinasi pada domain pemrograman.
3. **RAG Berbasis Konten Stack Overflow.** Nashid, Sintaha, dan Mesbah (2023)

Tabel II.4 Posisi penelitian terkait terhadap tiga dimensi utama penelitian ini

Penelitian		Fokus	SO sebagai sumber	Sinyal kepercayaan	GraphRAG
Silva, dan (2025)	Samhi, Khomh	Evaluasi komparatif LLM vs Stack Overflow	✓ (benchmark)	×	×
Kabir (2024)	dkk.	Karakteristik jawaban ChatGPT	✓ (benchmark)	×	×
Alomari (2026)	dkk.	LLM untuk kualitas kode	×	×	×
Jiang (2026)	dkk.	Deteksi duplikasi kode	×	×	×
Nashid, dan (2023)	Sintaha, Mesbah	RAG berbasis konten Stack Overflow	✓ (konteks)	×	×
Wang (2026)	dkk.	GraphRAG (PRIME)	×	×	✓
Ali dkk. (2026)		GraphRAG multimodal	×	×	✓
Zhou (2026)	dkk.	KG domain-spesifik + RAG	×	×	✓
Penelitian ini		GraphRAG berbasis Stack Overflow	✓✓	✓✓	✓✓

menunjukkan bahwa penggunaan konteks dari Stack Overflow dapat meningkatkan kualitas jawaban LLM pada tugas pemrograman. Namun, pendekatan tersebut masih menggunakan *dense retrieval* berbasis kemiripan vektor tanpa memanfaatkan struktur relasional maupun sinyal kepercayaan komunitas seperti *upvote* dan *accepted answer*. Akibatnya, seluruh informasi diperlakukan dengan bobot yang sama meskipun tingkat validasinya berbeda.

4. **GraphRAG pada Domain Teknis.** Penelitian seperti PRIME (Wang dkk. 2026), PythiaRAG (Ali dkk. 2026), dan IC-HRAG (Zhou dkk. 2026) membuktikan bahwa GraphRAG mampu meningkatkan akurasi retrieval sekaligus menurunkan halusinasi pada berbagai domain teknis. Namun, *knowledge graph* yang digunakan umumnya dibangun dari dokumen teknis biasa dan belum memanfaatkan sumber pengetahuan komunitas dengan validasi sosial yang kuat seperti Stack Overflow.

II.9.2 Identifikasi Kesenjangan

Sintesis dari empat area literatur menunjukkan adanya tiga kesenjangan metodologis utama yang menjadi dasar pengembangan penelitian ini.

1. Tidak adanya mekanisme *grounding* arsitektural berbasis komunitas Stack Overflow. Studi evaluatif oleh Silva, Samhi, dan Khomh (2025) dan Kabir dkk. (2024) telah menunjukkan keterbatasan LLM dibandingkan komunitas Stack Overflow dalam menjawab pertanyaan pemrograman. Namun, penelitian tersebut hanya berfokus pada evaluasi performa dan belum mengusulkan solusi pada tingkat arsitektur sistem. Dengan demikian, pertanyaan mengenai bagaimana merancang sistem yang mampu mengintegrasikan pengetahuan komunitas Stack Overflow ke dalam mekanisme kerja LLM masih belum terjawab. Kondisi ini menunjukkan adanya kesenjangan antara hasil evaluasi empiris dan implementasi arsitektur solusi.
2. RAG berbasis Stack Overflow belum memanfaatkan struktur relasional dan sinyal kepercayaan komunitas. Nashid, Sintaha, dan Mesbah (2023) menunjukkan bahwa konten Stack Overflow dapat meningkatkan kualitas jawaban LLM ketika digunakan sebagai sumber konteks. Namun, pendekatan yang digunakan masih berupa RAG konvensional berbasis *dense retrieval*, sehingga hanya mempertimbangkan kemiripan vektor. Pendekatan tersebut belum memanfaatkan hubungan relasional antar entitas maupun sinyal kepercayaan komunitas seperti *upvote*, *accepted answer*, dan reputasi kontributor. Padahal, sinyal-sinyal tersebut merupakan karakteristik unik Stack Overflow yang tidak dimiliki oleh repositori teks biasa. Studi Zhou dkk. (2026) juga menunjukkan bahwa keterbatasan representasi relasional menyebabkan akurasi

RAG konvensional berhenti pada tingkat 86,95%.

3. GraphRAG belum diterapkan pada *knowledge graph* berbasis komunitas teknis tervalidasi sosial di domain pemrograman. Penelitian oleh Wang dkk. (2026), Ali dkk. (2026), dan Zhou dkk. (2026) membuktikan bahwa GraphRAG mampu meningkatkan kemampuan penalaran dan akurasi pada berbagai domain teknis. Namun, sumber pengetahuan yang digunakan masih berasal dari dokumen teknis atau korpus umum tanpa memasukkan sinyal kualitas sosial. Berdasarkan tinjauan literatur, belum ditemukan penelitian yang secara khusus:
 - (a) membangun *knowledge graph* dari konten Stack Overflow,
 - (b) merepresentasikan sinyal kepercayaan komunitas sebagai bobot eksplisit pada simpul dan relasi graf, dan
 - (c) menggunakan graf berbobot tersebut sebagai sumber pengetahuan utama dalam GraphRAG untuk domain pemrograman.

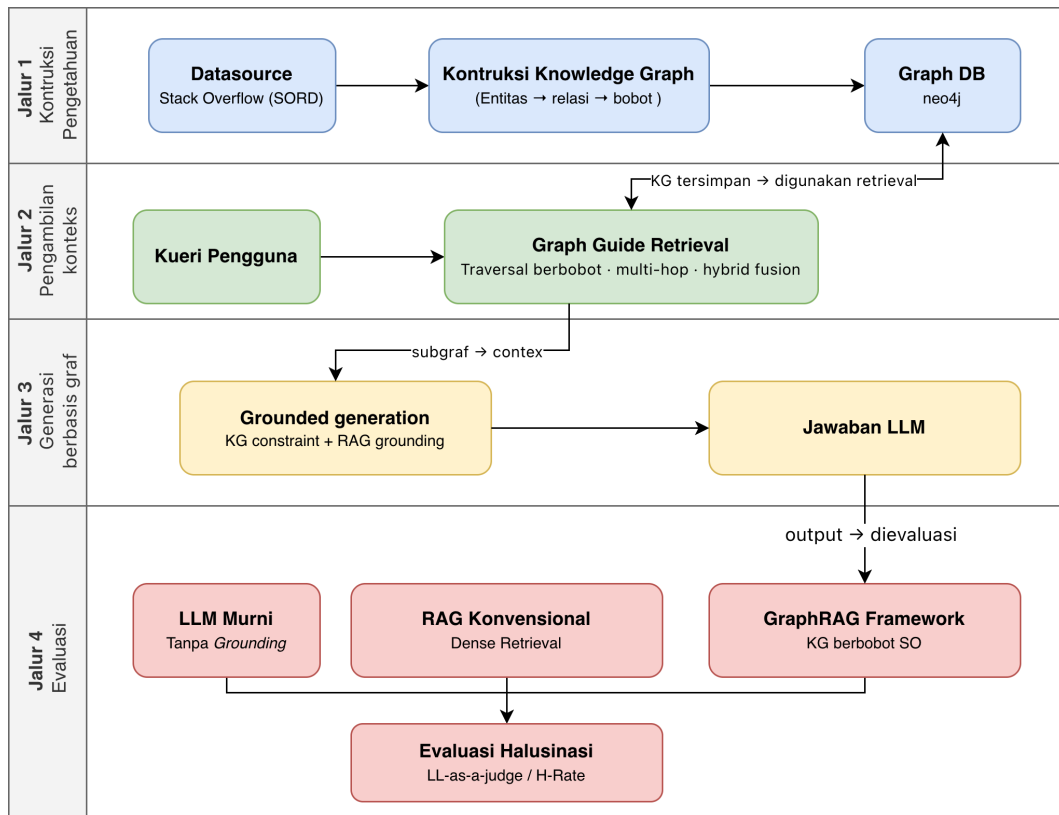
Ketiga kesenjangan tersebut menjadi dasar pengembangan penelitian ini, yaitu membangun *GraphRAG Framework* yang mengintegrasikan *knowledge graph* berbobot dari komunitas Stack Overflow ke dalam *pipeline* generasi jawaban LLM pada domain pemrograman. Pendekatan ini diharapkan dapat menggabungkan keunggulan struktur relasional GraphRAG dengan validasi sosial yang dimiliki oleh komunitas Stack Overflow.

Berdasarkan analisis kesenjangan di atas, kontribusi utama penelitian ini dapat dirumuskan sebagai berikut. Penelitian ini merancang dan mengimplementasikan sebuah GraphRAG Framework yang membangun *knowledge graph* berbobot dari Stack Overflow, di mana sinyal validasi komunitas (*upvote*, *accepted answer*, dan reputasi kontributor) diintegrasikan sebagai bobot eksplisit pada simpul dan relasi graf, sehingga mekanisme hybrid retrieval dan grounded generation secara langsung memprioritaskan pengetahuan yang telah tervalidasi secara sosial, dengan tujuan meningkatkan *faithfulness* dan menurunkan *hallucination rate* pada LLM untuk domain pemrograman. Pada sisi representasi pengetahuan, sinyal sosial komunitas digunakan sebagai bobot KG. Hal ini belum dilakukan pada GraphRAG sebelumnya. Pada sisi mekanisme retrieval, traversal berbobot komunitas diintegrasikan ke dalam *hybrid retrieval* sehingga jalur penelusuran yang dihasilkan tidak hanya relevan secara semantik tetapi juga terpercaya secara komunitas.

II.9.3 Kerangka Konseptual Penelitian

Berdasarkan studi literatur, kerangka konseptual penelitian ini (Gambar II.9) mengorganisasi komponen-komponen teoritis ke dalam empat jalur yang saling terhubung

secara sekuensial, yaitu (1) konstruksi pengetahuan, (2) pengambilan konteks, (3) generasi berbasis graf, dan (4) evaluasi.



Gambar II.9 Kerangka konseptual penelitian

Jalur 1 - Konstruksi pengetahuan. Jalur pertama membangun fondasi pengetahuan terstruktur melalui tiga aktivitas berurutan. Pertama, data diekstraksi dari SORD (Fatima dan Maqbool 2026) yang bersumber dari *Stack Exchange Data Dump* dan mencakup pertanyaan, jawaban, komentar, tag, serta metadata sinyal kepercayaan komunitas. Kedua, data tersebut diproses melalui *pipeline* konstruksi KG berbasis LLM menggunakan pendekatan *schema-based*. Proses ini meliputi NER, *entity linking*, dan ekstraksi tripel (subjek-predikat-objek). Pada tahap ini, bobot komunitas seperti *upvote*, *accepted answer*, dan reputasi kontributor diintegrasikan sebagai atribut eksplisit dalam graf. Ketiga, seluruh tripel yang dihasilkan disimpan ke dalam *graph database* Neo4j disertai *vector embedding* pada setiap simpul untuk mendukung mekanisme pencarian hibrid. KG berbobot yang tersimpan kemudian menjadi indeks pengetahuan tunggal bagi seluruh jalur berikutnya.

Jalur 2 - Pengambilan konteks. Jalur kedua merupakan inti teknis yang membedakan sistem ini dari GraphRAG konvensional, dan secara langsung mengoperasionalkan mekanisme *hybrid retrieval*. Proses dimulai dari kueri pengguna yang

diproses melalui NER dan *entity linking* untuk memetakan entitas teknis ke simpul yang relevan di dalam KG. Tahap ini merupakan tahap yang krusial karena sifat ambigu terminologi teknis pada Stack Overflow. Selanjutnya, dua mekanisme retrieval dijalankan secara terintegrasi:

1. *graph traversal* berbobot secara *multi-hop* yang mengikuti jalur relasi dari entitas awal sambil memprioritaskan simpul dengan bobot komunitas tinggi, sehingga rantai penalaran seperti `KeyError` $\xrightarrow{\text{raised_by}}$ `dict.__getitem__` $\xrightarrow{\text{similar_to}}$ `DataFrame.loc` $\xrightarrow{\text{solved_by}}$ `.get()` pattern dapat ditelusuri secara terstruktur (Wang dkk. 2026).
2. *semantic expansion* berbasis *embedding* vektor yang menemukan simpul relevan yang tidak terhubung secara eksplisit dalam graf.

Hasilnya adalah subgraf yang sekaligus kaya secara relasional, presisi secara semantik, dan terpercaya secara komunitas (Zhou dkk. 2026). Subgraf inilah yang menjadi satu-satunya sumber konteks bagi proses generasi di Jalur 3.

Jalur 3 - Generasi berbasis graf. Subgraf yang dihasilkan pada Jalur 2 diproses melalui mekanisme *grounded generation*. Setiap *triple* pada subgraf direpresentasikan dalam bentuk pasangan (Entitas-Relasi-Entitas) + segmen teks sumber dan disertakan sebagai *structured prompt* ke LLM. Mekanisme *dual-constraint* diterapkan melalui dua lapisan kendali yang bekerja bersamaan. Lapisan pertama adalah kendala berbasis KG yang mengharuskan LLM menghasilkan jawaban sesuai fakta yang direpresentasikan oleh subgraf. Lapisan kedua adalah mekanisme *RAG grounding* yang mengaitkan setiap klaim dengan sumber tekstual Stack Overflow yang dapat diverifikasi (Khalifa, Messaoud, dan Mkaouer 2026). Hasilnya adalah jawaban yang tidak hanya akurat secara faktual, tetapi juga menyediakan jalur *provenance* yang dapat ditelusuri kembali ke simpul KG beserta informasi bobot komunitas sumber. Pendekatan ini memungkinkan pengguna mengetahui bahwa jawaban berasal dari solusi yang telah divalidasi komunitas pengembang (Ali dkk. 2026).

Jalur 4 - Evaluasi dan validasi. Jawaban yang dihasilkan pada Jalur 3 dievaluasi melalui tiga skenario perbandingan bertingkat: (1) LLM murni tanpa mekanisme *grounding*, (2) RAG konvensional berbasis *dense retrieval*, dan (3) GraphRAG Framework yang diusulkan dengan KG berbobot dari Stack Overflow. Desain tiga skenario ini memungkinkan isolasi kontribusi masing-masing komponen yaitu mengukur kontribusi RAG secara umum dan mengukur kontribusi struktur relasional KG dan bobot komunitas yang merupakan novelty utama penelitian ini. Evaluasi dilakukan menggunakan protokol *LLM-as-a-Judge* yang reliabilitasnya divalidasi melalui Cohen's $\kappa \geq 0,6$ (Gu, Jiang, Shi, dkk. 2026), dengan metrik meliputi

hallucination rate (H-Rate), *faithfulness*, *context relevance*, dan *answer relevance* melalui kerangka RAGAS (Kamalipour, Asadi, dan Chimeh 2026).

Keempat jalur tersebut membentuk suatu sistem yang terintegrasi secara kohesif. KG yang dibangun pada Jalur 1 berfungsi sebagai indeks pengetahuan bagi proses retrieval pada Jalur 2, subgraf hasil retrieval menjadi konteks bagi proses generasi pada Jalur 3, sedangkan jawaban yang dihasilkan LLM menjadi objek evaluasi pada Jalur 4. Konsistensi antara permasalahan penelitian, solusi yang diusulkan, dan artefak yang dirancang merupakan prasyarat penting dalam menjaga kualitas metodologis penelitian berbasis desain (Pefferers dkk. 2007).

BAB III

Analisis dan Desain

Bab ini merupakan kelanjutan langsung dari analisis *research gap* yang diidentifikasi pada Bab II. Tiga kesenjangan yang ditemukan, yaitu G1 (belum adanya mekanisme *grounding* arsitektural berbasis komunitas Stack Overflow), G2 (RAG berbasis Stack Overflow yang belum memanfaatkan dimensi relasional dan sinyal kepercayaan komunitas), serta G3 (GraphRAG yang belum diaplikasikan pada *knowledge graph* dari komunitas teknis bervalidasi sosial), diterjemahkan ke dalam keputusan analisis dan desain yang konkret. Secara garis besar, bab ini membahas hal-hal berikut.

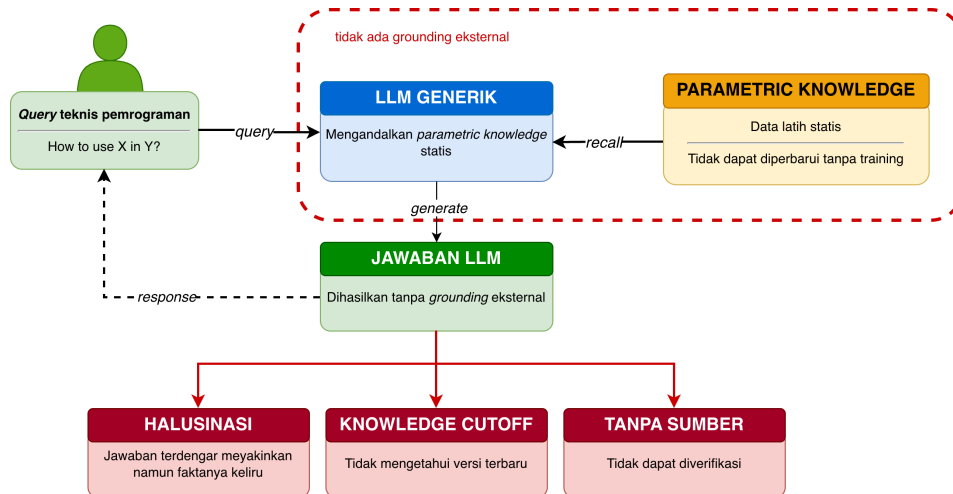
1. Deskripsi hasil analisis kondisi saat ini dan persoalan yang ditangani, merujuk pada analisis dari studi-studi yang diidentifikasi pada Bab II,
2. Analisis kebutuhan komponen *GraphRAG Framework*, mencakup kebutuhan fungsional dan nonfungsional yang diturunkan dari rumusan masalah dan hipotesis penelitian yang ditetapkan di Bab I, dan
3. Analisis pemilihan solusi beserta desain konseptual *GraphRAG Framework* yang merespons ketiga kesenjangan secara sistematis, termasuk rancangan *knowledge graph* berbobot, mekanisme *hybrid retrieval*, dan protokol evaluasi komparatif.

III.1 Analisis Kondisi Saat Ini

III.1.1 Model Konseptual Sistem yang Ada

Sistem tanya-jawab teknis berbasis *Large Language Model* (LLM) yang umum digunakan saat ini masih bekerja menggunakan mekanisme satu arah. Pengguna memberikan pertanyaan terkait pemrograman, kemudian model menghasilkan jawaban berdasarkan pengetahuan internal yang telah dipelajari selama proses pra-pelatihan (*pre-training*) (Huang dkk. 2025). Pengetahuan tersebut dikenal sebagai *parametric knowledge*, yaitu informasi yang tersimpan dalam parameter atau bobot model. Dengan mekanisme ini, LLM menghasilkan respons berdasarkan apa yang telah dipelajari sebelumnya tanpa mengambil informasi tambahan dari sumber eksternal saat proses inferensi berlangsung. Konsekuensinya, sistem tidak memiliki mekanisme untuk melakukan *grounding* terhadap sumber pengetahuan yang terverifikasi, seperti dokumentasi resmi, basis pengetahuan komunitas, maupun repositori teknis lainnya. Akibatnya, kualitas jawaban sangat bergantung pada cakupan dan repre-

sentativitas data latih yang digunakan selama pelatihan model (Huang dan Huang 2026). Kondisi tersebut dapat menimbulkan beberapa permasalahan, seperti ketidakakuratan informasi, hilangnya konteks teknis tertentu, serta munculnya halusinasi ketika model menghadapi pertanyaan yang tidak tercakup dengan baik pada data pelatihan. Gambar III.1 memperlihatkan model konseptual sistem yang digunakan saat ini beserta titik-titik potensi kegagalan yang menjadi fokus pada penelitian ini.



Gambar III.1 Model konseptual sistem tanya-jawab berbasis LLM generik tanpa mekanisme pengambilan konteks dari sumber terverifikasi.

III.1.2 Analisis Keterbatasan Sistem Saat Ini

Studi Silva, Samhi, dan Khomh (2025) menyediakan bukti kuantitatif paling komprehensif tentang keterbatasan LLM ketika menangani pertanyaan komunitas Stack Overflow. Studi tersebut mengevaluasi ChatGPT-3.5 dan LLaMA-2-7b menggunakan pertanyaan dari Stack Overflow yang mencakup berbagai domain *software engineering*. Temuan yang relevan dengan penelitian ini dirangkum pada Tabel III.1.

Studi Kabir dkk. (2024) memperkuat temuan ini dengan melaporkan bahwa 52% jawaban ChatGPT untuk pertanyaan SO bersifat tidak benar ketika dievaluasi secara manual, Angka yang jauh lebih tinggi dari yang diklaim dalam evaluasi berbasis metrik otomatis. Ketidakakuratan ini paling parah pada pertanyaan yang membutuhkan pengetahuan tentang versi *library* atau *framework* terkini, yang merupakan manifestasi konkret dari kesenjangan G2 dan G3.

Selain itu, Nashid, Sintaha, dan Mesbah (2023) mendemonstrasikan bahwa augmentasi konteks dari Stack Overflow meningkatkan relevansi jawaban LLM pada tugas

pemrograman, namun pendekatan berbasis *dense retrieval* yang digunakan tidak mengeksplorasi struktur relasional dan sinyal kepercayaan yang tersedia dalam konten Stackoverflow dan sinyal kepercayaan yang tersedia dalam konten Stack Overflow. Studi ini menunjukkan peningkatan yang menjanjikan tetapi terbatas, mengkonfirmasi bahwa mekanisme RAG konvensional belum sepenuhnya memanfaatkan potensi sumber daya komunitas teknis seperti Stack Overflow, yang menjadi inti dari kesenjangan G2 dan G3.

Tabel III.1 Temuan empiris Silva, Samhi, dan Khomh (2025) yang menjadi titik acuan pengukuran *improvement* penelitian ini.

Dimensi	Temuan	Implikasi untuk Penelitian Ini
Kemiripan tekstual keseluruhan	Sekitar 85% pertanyaan memiliki <i>cosine similarity</i> $> 0,5$ terhadap jawaban diterima SO	15% pertanyaan—yang terkait domain Framework & Library—adalah target utama <i>improvement</i>
Domain paling lemah	<i>Frameworks and Libraries</i> : LLM gagal karena <i>knowledge cutoff</i> dan absennya <i>community conventions</i>	GraphRAG Framework harus secara eksplisit menjawab kegagalan pada domain ini
Keterlacakan sumber	LLM menghasilkan 0% referensi ke sumber SO	Setiap jawaban GraphRAG harus menyertakan kutipan ke SO thread
Dampak pada komunitas	Penurunan signifikan aktivitas posting SO pasca-rilis ChatGPT, namun pengguna kembali ke SO ketika LLM gagal	Mengkonfirmasi bahwa pengetahuan komunitas SO belum dapat digantikan LLM

III.1.3 Identifikasi Masalah

Berdasarkan ketiga studi di atas, masalah utama yang ditangani dalam penelitian ini dijabarkan sebagai berikut:

1. Halusinasi Faktual pada Domain *Framework* dan *Library*. LLM menghasilkan jawaban yang terdengar meyakinkan namun faktanya keliru, khususnya ketika pertanyaan menyentuh *library* atau *framework* yang rilis atau diperbarui setelah batas pengetahuan (*knowledge cutoff*) model. Kondisi ini merupakan bentuk *factual hallucination* dan *extrinsic hallucination* sebagaimana diklasifikasikan oleh Ji dkk. (2023) dan Huang dkk. (2025) pada Bab II. Ini adalah manifestasi dari absennya mekanisme *grounding* eksternal (G1).

2. Kegagalan Memanfaatkan Sinyal Kepercayaan Komunitas Stackoverflow. Informasi signal kepercayaan seperti *upvote*, status *accepted answer*, reputasi kontributor, hubungan rekomendasi yang tersedia di StackOverflow namun tidak dieksploitasi oleh LLM maupun sistem RAG konvensional. Ini adalah inti kesenjangan G2.
3. Ketidakterlacakan Sumber Jawaban. Jawaban LLM tidak dapat dilacak ke diskusi komunitas yang menjadi dasarnya. Ketidakterlacakan ini menghambat verifikasi independen dan mengurangi kepercayaan pengguna, terutama pada konteks profesional.
4. Pengetahuan yang Tidak Dapat Diperbarui Tanpa *Re-training*. LLM tidak dapat mengadaptasi pengetahuannya secara dinamis ketika ekosistem *library* berkembang, tanpa proses *fine-tuning* ulang yang mahal secara komputasi. Keterbatasan statis pengetahuan parametrik ini merupakan salah satu motivasi utama pendekatan berbasis RAG dan KG (Huang dan Huang 2026; Lewis dkk. 2020).

III.2 Analisis Kebutuhan

III.2.1 Identifikasi Kebutuhan Pengguna

Pengguna utama *GraphRAG Framework* adalah *software developer* dan *software engineer* yang menggunakan LLM untuk mendapatkan jawaban atas pertanyaan teknis di domain pemrograman, pertanyaan yang biasanya diajukan ke Stack Overflow. Tabel III.2 memetakan masalah pengguna ke kebutuhan solusi yang harus dipenuhi artefak.

Tabel III.2 Pemetaan masalah pengguna ke kebutuhan solusi *GraphRAG Framework*.

No.	Masalah Pengguna	Akar Penyebab (Gap)	Kebutuhan Solusi
M1	LLM memberikan jawaban keliru atau halusinasi pada domain Framework & Library	G1: Absennya <i>grounding</i> eksternal	Jawaban di- <i>ground</i> pada konten SO terverifikasi
M2	Kualitas jawaban tidak mencerminkan konsensus komunitas SO	G2: Sinyal kepercayaan tidak dieksploitasi	Bobot kepercayaan (<i>upvote, accepted</i>) dikodekan dalam KG
M3	Tidak dapat menelusuri dari mana jawaban berasal	G1 + G2: Tidak ada mekanisme <i>citation</i>	Setiap jawaban menyertakan referensi ke SO thread
M4	Jawaban tidak dapat menangani pertanyaan relasional kompleks	G3: Tidak ada penalaran berbasis struktur graf	Traversal graf untuk menghubungkan entitas terkait
M5	Pengetahuan LLM tidak mencakup versi <i>library</i> terbaru	G1 + G2: <i>Knowledge cutoff</i> tanpa mekanisme pembaruan	Pembaruan KG tanpa <i>re-training</i> LLM

III.2.2 Kebutuhan Fungsional

Berdasarkan pemetaan masalah di atas, kebutuhan fungsional *GraphRAG Framework* dirumuskan sebagai berikut:

- F1.** Konstruksi *Knowledge Graph* Berbobot dari Stack Overflow. Sistem harus dapat memproses dataset Stack Overflow mencakup pertanyaan, jawaban diterima, komentar, tag, dan data rekomendasi untuk mengekstrak entitas dan relasi. Pipeline konstruksi yaitu penyusunan ontologi domain *schema-based*, praproses dokumen, NER dan ekstraksi relasi, *entity linking*, serta peningkatan kualitas graf. Sistem kemudian membangun KG berbobot di mana bobot pada simpul dan sisi merepresentasikan sinyal kepercayaan komunitas SO (skor *upvote*, status *accepted answer*, reputasi kontributor, dan kekuatan rekomendasi antar-*thread*). Kebutuhan ini secara langsung menjawab G2 dan G3.
- F2.** Pengindeksan Vektor Teks Konten Stack Overflow Sistem harus mengindeks teks pertanyaan dan jawaban dari Stack Overflow ke dalam *vector store* menggunakan model *embedding* untuk mendukung pencarian semantik berbasis kemiripan kosinus sebagai jalur komplementer terhadap traversal graf.
- F3.** Pemahaman dan Dekomposisi Kueri. Sistem harus mampu mengekstrak entitas kunci dari kueri pengguna menggunakan *named entity recognition* (NER) dan memetakan entitas tersebut ke simpul yang ada di KG Stack Overflow melalui *entity linking*.
- F4.** *Hybrid Retrieval*. Mekanisme pengambilan konteks menjalankan tiga tahap sekuensial, yaitu *Entity Anchoring*, *Graph Traversal* multi-hop mengikuti sisi berbobot tertinggi, dan *Semantic Expansion* yang menggunakan simpul hasil Graph Traversal sebagai anchor pencarian vektor di FAISS untuk menemukan konten relevan yang tidak terhubung eksplisit di KG. Hasil ketiga tahap difusikan melalui proses reranking, deduplikasi, dan packing menjadi *unified context window*. Kebutuhan ini menjawab G2 dan G3.
- F5.** *Grounded Generation* dengan *Dual-Constraint* dan *Citation*. Berdasarkan *unified context window* dari *hybrid retrieval*, sistem membangkitkan jawaban melalui mekanisme *dual-constraint*: (a)*RAG Grounding* memastikan setiap klaim bersumber dari konteks yang diberikan (*context-faithful generation*), dan (b)*KG-Constraint* memastikan jawaban konsisten dengan fakta yang terkodekan dalam KG berbobot. Output sistem terdiri atas tiga elemen: jawaban faktual, kutipan ke SO *thread* sumber, dan jalur *provenance* pada KG. Ini menjawab G1.
- F6.** Protokol Evaluasi Komparatif Tiga Kondisi. Sistem harus menyediakan *pipe-*

line evaluasi yang dapat membandingkan tiga kondisi secara *apple-to-apple*: (a) LLM murni sebagai *baseline* (Silva, Samhi, dan Khomh 2025), (b) RAG konvensional sebagai *ablation*, dan (c) *GraphRAG Framework* yang diusulkan menggunakan dataset evaluasi yang sama. Evaluasi menggunakan kerangka RAGAS dengan metrik *hallucination rate*, *faithfulness*, *context relevance*, dan *answer relevance* (Kamalipour, Asadi, dan Chimeh 2026), serta paradigma *LLM-as-a-Judge* yang reliabilitasnya divalidasi dengan Cohen's $\kappa \geq 0,6$ (Gu, Jiang, Shi, dkk. 2026).

- F7. Pembaruan KG Inkremental. Sistem harus mendukung penambahan konten SO baru ke dalam KG tanpa *re-training* model LLM, sehingga pengetahuan dapat diperbarui secara efisien seiring perkembangan ekosistem *library*. Ini merupakan keunggulan arsitektural RAG dan KG dibandingkan pendekatan parametrik murni.

III.2.3 Kebutuhan Nonfungsional

Tabel III.3 merinci keenam kebutuhan nonfungsional yang diturunkan dari empat pertimbangan yang saling berkaitan. Pertama, dimensi kinerja dan skalabilitas, sistem harus mampu melakukan traversal KG berbobot dan pencarian vektor dalam waktu yang dapat diterima untuk penggunaan interaktif, mengingat dataset Stack Overflow berskala besar. Kedua, dimensi keterlacakan (*traceability*), setiap jawaban harus menyertakan jalur *provenance* yang dapat ditelusuri ke simpul KG sumber. Ketiga, dimensi *reprodusibilitas* evaluasi, protokol tiga kondisi (A, B, C) harus dapat dijalankan ulang dengan hasil yang konsisten, sesuai persyaratan validitas evaluasi komparatif. Keempat, dimensi *modularitas*, komponen KG, *retriever*, dan *generator* harus dapat diganti atau dikonfigurasi secara independen agar eksperimen ablasi dapat dilakukan dengan mengubah satu variabel pada satu waktu.

Tabel III.3 Kebutuhan nonfungsional *GraphRAG Framework*.

No.	Kategori	Deskripsi	Kriteria Sukses
NF1	Akurasi	Jawaban lebih faktual dari kondisi <i>baseline</i>	<i>Hallucination rate</i> lebih rendah dari Silva, Samhi, dan Khomh (2025) secara statistik signifikan ($\alpha = 0,05$)
NF2	Keterlacakan	Setiap jawaban memiliki referensi ke SO thread	100% jawaban dengan ≥ 1 kutipan sumber
NF3	Latensi	Waktu respons yang wajar untuk pengguna	≤ 15 detik per kueri pada konfigurasi pengujian standar
NF4	Reprodusibilitas	Hasil evaluasi dapat direproduksi	Dataset evaluasi dan kode dipublikasikan secara terbuka
NF5	Modularitas	Komponen KG, <i>retrieval</i> , dan LLM dapat dikonfigurasi	Setiap komponen dapat diganti melalui file konfigurasi
NF6	Skalabilitas	Mendukung subset Stack Overflow skala menengah	KG dengan > 10.000 simpul tanpa degradasi latensi yang signifikan

III.3 Analisis Pemilihan Solusi

III.3.1 Alternatif Solusi

Berdasarkan identifikasi masalah dan kebutuhan di atas, tiga kondisi dipertimbangkan sekaligus membentuk *framework* evaluasi komparatif penelitian ini.

Kondisi A. LLM Murni (*Baseline* Silva, Samhi, dan Khomh (2025)).

LLM menjawab kueri pengguna langsung dari *parametric knowledge* tanpa augmentasi konteks eksternal. Ini adalah kondisi yang dievaluasi oleh Silva, Samhi, dan Khomh (2025) dan Kabir dkk. (2024), dan menjadi titik acuan pengukuran *improvement*. Keterbatasan yang sudah terdokumentasi adalah halusinasi pada domain Framework & Library, absennya *citation*, kegagalan pada pertanyaan relasional kompleks, ketidakmampuan mengadaptasi pengetahuan tanpa *re-training*.

Kondisi B. RAG Konvensional (*Ablation Study*).

Konten SO dari SORD (Fatima dan Maqbool 2026) diindeks sebagai *flat chunks* ke dalam *vector store*. Pada waktu inferensi, potongan teks paling mirip secara semantik diambil dan dimasukkan ke dalam *prompt* LLM (*dense retrieval*). Keterbatasan arsitektural adalah tidak mengeksplorasi struktur relasional KG, sinyal kepercayaan komunitas SO (G2) tidak dimanfaatkan, ti-

dak mampu menjawab kueri multi-hop, rentan terhadap *knowledge noise* dari *chunk* yang kurang relevan (Zhou dkk. 2026). Studi Nashid, Sintaha, dan Mesbah (2023) menunjukkan peningkatan yang menjanjikan tetapi terbatas karena ketiadaan struktur relasional.

Kondisi C. *GraphRAG Framework* (Solusi yang Diusulkan).

Konten SO dari SORD (Fatima dan Maqbool 2026) direpresentasikan sebagai KG berbobot di mana sinyal kepercayaan dikodekan secara eksplisit. Mekanisme *hybrid retrieval* menjalankan tiga tahap sekuensial, yaitu *Entity Anchoring*, *Graph Traversal* multi-hop berbobot, dan *Semantic Expansion* berbasis simpul subgraf. Hasil ketiga tahap difusikan menjadi *unified context window* berformat E–R–E. Pembangkitan jawaban menggunakan mekanisme *dual-constraint: RAG Grounding* (jawaban harus bersumber dari konteks) dilanjutkan *KG-Constraint* (jawaban harus konsisten dengan fakta KG). Output akhir berupa jawaban faktual, kutipan SO *thread* dan jalur *provenance* pada KG. Keunggulan arsitektural ini menjawab G1, G2, dan G3.

III.3.2 Analisis Penentuan Solusi

Tabel III.4 menyajikan matriks komparasi ketiga kondisi terhadap kebutuhan yang telah diidentifikasi. Matriks ini sekaligus mendefinisikan posisi *improvement* yang diklaim penelitian ini. Berdasarkan matriks ini, Kondisi C (*GraphRAG Framework*) dipilih sebagai solusi yang dikembangkan. Table III.5 menyajikan ringkasan dari perbandingan ketiga kondisi tersebut, sekaligus menunjukkan posisi penelitian ini terhadap baseline Silva, Samhi, dan Khomh (2025) dan kondisi pembanding RAG konvensional.

Sebagai konsekuensinya, Kondisi C dipilih sebagai solusi yang dikembangkan pada penelitian ini. Sementara itu, Kondisi A dan Kondisi B tetap diimplementasikan sebagai pembanding dalam protokol evaluasi komparatif (F6), sehingga peningkatan kinerja yang diperoleh dapat diukur secara kuantitatif dan direproduksi terhadap *baseline* Silva, Samhi, dan Khomh (2025).

III.4 Desain Konseptual *GraphRAG Framework*

III.4.1 Gambaran Umum Arsitektur

Arsitektur *GraphRAG Framework* yang diusulkan terdiri atas tiga blok utama yang saling terhubung, serta satu blok evaluasi untuk membandingkan kinerja sistem. Gambaran keseluruhan arsitektur dapat dilihat pada Gambar III.2. Ketiga blok utama tersebut dikembangkan berdasarkan kerangka konseptual yang telah dijelaskan pada Bab II (Gambar II.9) dan dirancang untuk mengatasi kesenjangan penelitian

Tabel III.4 Perbandingan ketiga kondisi evaluasi terhadap kebutuhan sistem dan tujuan penelitian.

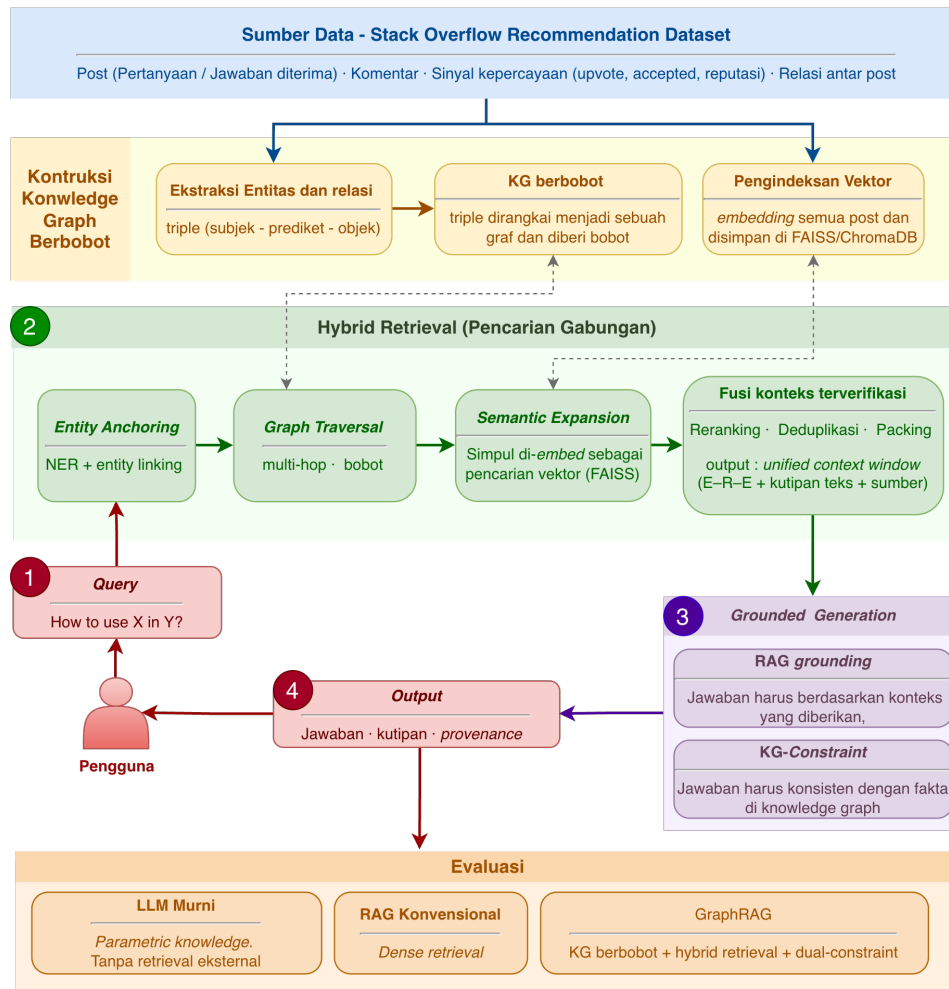
Kebutuhan / Kriteria	Kondisi A LLM Murni	Kondisi B RAG Konvensional	Kondisi C GraphRAG Framework
Reduksi halusinasi (M1)	Tidak	Sebagian	Ya
Pemanfaatan sinyal kepercayaan Stack Overflow (M2, F1)	Tidak	Tidak	Ya
Keterlacakan sumber (<i>citation</i>) (M3, F5)	Tidak	Sebagian	Ya
Penalaran relasional dan <i>multi-hop</i> (M4, F4)	Tidak	Tidak	Ya
Pembaruan pengetahuan tanpa <i>re-training</i> (M5, F7)	Tidak	Ya	Ya
Kinerja pada domain <i>framework</i> dan <i>library</i>	Rendah	Sedang	Tinggi
Kesesuaian dengan temuan G1	Tidak	Tidak	Ya
Kesesuaian dengan temuan G2	Tidak	Tidak	Ya
Kesesuaian dengan temuan G3	Tidak	Tidak	Ya
Kesesuaian keseluruhan	Rendah	Sedang	Tinggi

Tabel III.5 Perbandingan pemenuhan kebutuhan sistem pada tiga kondisi penelitian.

Kriteria Kebutuhan	Kondisi A LLM Murni	Kondisi B RAG Konvensional	Kondisi C GraphRAG
Grounding arsitektural berbasis Stack Overflow (G1)	×	~	✓
Pengetahuan library/API terkini (G1)	×	✓	✓
Sinyal kepercayaan komunitas Stack Overflow (G2)	×	×	✓
Retrieval relasional dan multi-hop (G2, G3)	×	×	✓
<i>Semantic retrieval</i> berbasis vektor	×	✓	✓
<i>Grounded generation</i> dan sitasi (G1)	×	~	✓
<i>Provenance</i> dan keterlacakan sumber	×	~	✓
Protokol evaluasi komparatif	✓	✓	✓
Pembaruan pengetahuan tanpa <i>re-training</i>	×	~	✓

✓ = Terpenuhi ~ = Sebagian terpenuhi × = Tidak terpenuhi

yang telah diidentifikasi sebelumnya.



Gambar III.2 Arsitektur *GraphRAG Framework* yang terdiri atas tiga blok utama (Konstruksi KG Berbobot, *Hybrid Retrieval*, dan *Grounded Generation*) beserta blok evaluasi komparatif.

1. Konstruksi *Knowledge Graph* Berbobot.

Dataset SORD (Fatima dan Maqbool 2026) diproses melalui tiga tahapan utama. Pertama, dilakukan *ekstraksi entitas dan relasi* untuk menghasilkan triple (subjek-predikat-objek) dari data Stack Overflow. Kedua, triple yang dihasilkan digunakan untuk membangun *knowledge graph* berbobot di Neo4j. Bobot pada setiap relasi ditentukan berdasarkan sinyal kepercayaan komunitas Stack Overflow, seperti status *accepted answer*, jumlah *upvote*, dan reputasi kontributor. Ketiga, seluruh teks *post* diindeks ke dalam basis data vektor menggunakan FAISS atau ChromaDB dengan bantuan model *embedding*. Hasil dari tahap ini berupa *knowledge graph* berbobot dan *vector store* yang akan digunakan pada proses pencarian informasi.

2. *Hybrid Retrieval* (Pencarian Gabungan).

Ketika pengguna mengajukan pertanyaan, sistem melakukan proses pencarian informasi melalui beberapa tahap. Tahap pertama adalah *entity anchoring*, yaitu mengidentifikasi entitas teknis dalam pertanyaan dan menghubungkannya dengan simpul yang sesuai pada *knowledge graph*. Tahap kedua adalah *graph traversal*, yaitu menelusuri hubungan antar simpul pada graph untuk menemukan informasi yang paling relevan. Penelusuran dilakukan dengan mempertimbangkan bobot relasi yang berasal dari sinyal kepercayaan komunitas. Tahap ketiga adalah *semantic expansion*, yaitu menggunakan hasil penelusuran graph sebagai dasar pencarian pada basis data vektor sehingga sistem dapat menemukan informasi relevan yang tidak terhubung secara langsung dalam graph. Informasi yang diperoleh dari ketiga tahap tersebut kemudian digabungkan melalui proses *reranking*, deduplikasi, dan penyusunan konteks. Hasil akhirnya adalah satu kumpulan konteks terintegrasi (*unified context window*) yang berisi fakta dari *knowledge graph*, kutipan teks, serta tautan ke sumber asalnya.

3. *Grounded Generation*.

Konteks yang telah diperoleh dari tahap *retrieval* digunakan sebagai dasar bagi LLM untuk menghasilkan jawaban. Pada tahap ini diterapkan dua mekanisme pengendalian. Pertama, *RAG Grounding* memastikan bahwa jawaban yang dihasilkan didasarkan pada informasi yang tersedia dalam konteks sehingga mengurangi kemungkinan munculnya informasi yang tidak didukung sumber. Kedua, *KG-Constraint* memastikan bahwa jawaban tetap konsisten dengan fakta yang tersimpan dalam *knowledge graph* berbobot. Output akhir sistem terdiri atas tiga komponen, yaitu: (a) jawaban yang dihasilkan oleh model, (b) kutipan menuju *thread* Stack Overflow yang menjadi sumber informasi, dan (c) jalur *provenance* pada *knowledge graph* yang menunjukkan asal-usul informasi beserta tingkat kepercayaannya.

Selain ketiga blok utama tersebut, terdapat Blok Evaluasi yang digunakan untuk membandingkan tiga pendekatan berbeda, yaitu Kondisi A (LLM murni tanpa *retrieval*), Kondisi B (RAG konvensional dengan *dense retrieval*), dan Kondisi C (*GraphRAG Framework* yang menggabungkan *knowledge graph* berbobot, *hybrid retrieval*, dan mekanisme *dual-constraint*). Gambar III.2 menunjukkan hubungan antar komponen dalam arsitektur *GraphRAG Framework* secara keseluruhan.

III.4.2 Desain *Knowledge Graph* Berbobot dari SORD

Knowledge graph (KG) berbobot merupakan komponen utama dalam framework yang diusulkan. Komponen ini dirancang untuk menjawab G2, yaitu pemanfaatan sinyal kepercayaan komunitas, serta G3, yaitu penerapan pendekatan *GraphRAG*

pada data komunitas teknis yang telah melalui validasi sosial. Sumber data yang digunakan adalah SORD (Fatima dan Maqbool 2026), yaitu dataset yang dikurasi dari Stack Overflow dan mencakup pertanyaan, jawaban, komentar, serta tag beserta seluruh metadata sinyal kepercayaan komunitas. Desain KG mencakup jenis simpul (*node*), jenis relasi (*edge*), dan mekanisme pembobotan yang digunakan untuk merepresentasikan tingkat kepercayaan informasi.

III.4.2.1 Tipe Simpul

Setiap simpul dalam KG merepresentasikan objek atau konsep yang terdapat pada data Stack Overflow. Berikut empat tipe simpul yang digunakan.

- (a) *Question*, yaitu simpul yang merepresentasikan pertanyaan pada Stack Overflow. Simpul ini menyimpan informasi seperti *id*, *title*, *body*, *score*, *viewCount*, *creationDate*, dan *domainTag*.
- (b) *Answer*, yaitu simpul yang merepresentasikan jawaban terhadap suatu pertanyaan. Atribut yang disimpan meliputi *id*, *body*, *score*, *isAccepted*, dan *authorReputation*.
- (c) *Tag*, yaitu simpul yang merepresentasikan teknologi, bahasa pemrograman, atau domain tertentu yang digunakan pada Stack Overflow. Simpul ini memiliki atribut *name* dan *questionCount*.
- (d) *Concept*, yaitu simpul yang merepresentasikan konsep atau entitas teknis yang diekstraksi dari teks pertanyaan maupun jawaban, seperti nama *library*, jenis *error*, atau metode API. Simpul ini memiliki atribut *name*, *type*, dan *frequency*.

Keempat tipe simpul di atas memungkinkan pembentukan tripel domain pemrograman, misalnya (Q#1234, *hasAcceptedAnswer*, A#9823) atau (A#9823, *mentions*, OAuth2PasswordBearer).

III.4.2.2 Tipe Sisi dan Skema Bobot Kepercayaan

Hubungan antar simpul direpresentasikan dalam bentuk sisi (*edge*). Setiap sisi diberikan bobot yang mencerminkan tingkat kepercayaan berdasarkan sinyal yang tersedia pada Stack Overflow. Dengan demikian, kualitas informasi yang telah divalidasi oleh komunitas dapat direpresentasikan secara eksplisit di dalam KG. Sinyal kepercayaan yang digunakan meliputi status *accepted answer*, jumlah *upvote*, serta reputasi pengguna yang memberikan kontribusi. Operasionalisasi dari setiap sinyal tersebut menjadi bobot relasi dijelaskan pada Tabel III.6.

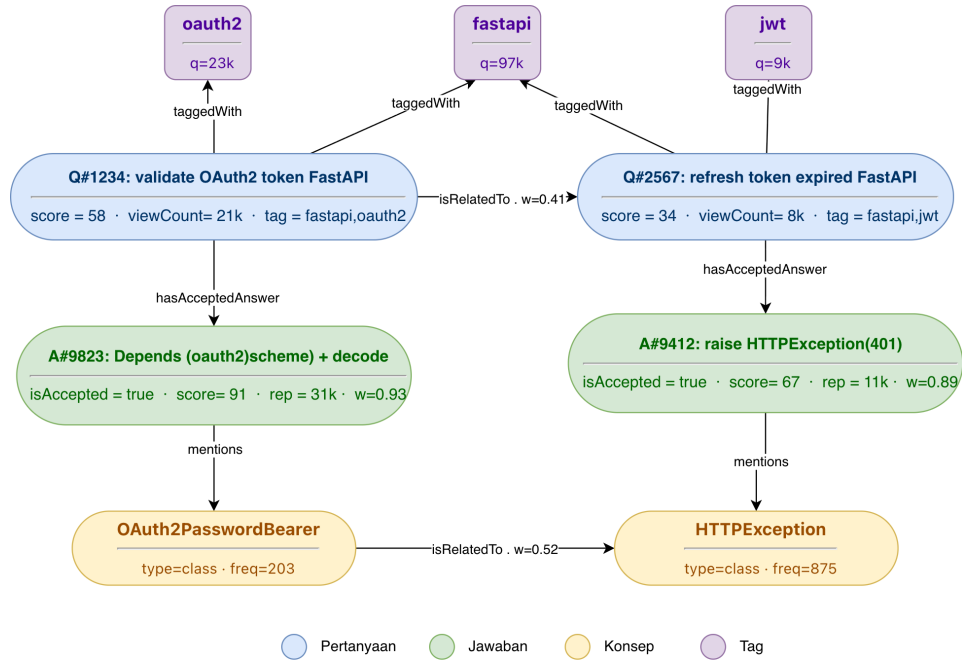
Untuk memberikan gambaran mengenai struktur KG yang dibangun, Gambar III.3 menampilkan contoh subgraf dari domain autentikasi pada *framework* FastAPI. Su-

Tabel III.6 Operasionalisasi sinyal kepercayaan Stack Overflow sebagai bobot sisi pada *knowledge graph*.

Typ e Sisi	Arah	Makna Semantik	Bobot w
hasAcceptedAnswer	$Q \rightarrow A$	Jawaban yang diterima sebagai solusi terbaik oleh penanya	$w = 1.0$
hasAnswer	$Q \rightarrow A$	Jawaban lain dengan skor tinggi	$w = \text{normalize}(\text{score})$
taggedWith	$Q \rightarrow \text{Tag}$	Pertanyaan dikategorikan dalam domain teknologi tertentu	$w = 1.0$
isRelatedTo	$Q \leftrightarrow Q$	Pasangan rekomendasi dalam dataset SORD	$w = \text{similarity}(q_i, q_j)$
mentions	$Q/A \rightarrow \text{Concept}$	Pertanyaan atau jawaban menyebut entitas teknis tertentu	$w = \text{TF-IDF}(\text{concept}, \text{doc})$
authorTrust	$A \rightarrow \text{Author}$	Kepercayaan terhadap kontributor jawaban	$w = \text{normalize}(\text{reputation})$

bgraf tersebut memuat dua simpul *Question* yang saling terhubung: Q#1234 mengenai validasi token OAuth2 pada FastAPI (score=58, viewCount=21k), dan Q#2567 mengenai penanganan *refresh token expired* (score=34, viewCount=8k). Keduanya dihubungkan melalui relasi *isRelatedTo* dengan bobot $w=0,41$, yang mencerminkan keterkaitan topik antar-*thread* tanpa hubungan langsung berupa *accepted answer*.

Masing-masing pertanyaan memiliki simpul *Answer* yang diterima (*accepted*): A#9823 menjawab Q#1234 dengan solusi Depends(oauth2_scheme) + decode ($w=0,93$), sedangkan A#9412 menjawab Q#2567 dengan raise HTTPException(401) ($w=0,89$). Perbedaan nilai w di antara keduanya mencerminkan perbedaan sinyal kepercayaan komunitas, yaitu skor *upvote* dan reputasi kontributor. Kedua pertanyaan juga terhubung ke tiga simpul *Tag*, yaitu oauth2, fastapi, dan jwt, melalui relasi *taggedWith*. Simpul fastapi berperan sebagai *bridge node* yang menghubungkan kedua *thread* secara tidak langsung, sehingga traversal dari salah satu pertanyaan dapat menjangkau pertanyaan lainnya melalui tag yang sama. Pada lapisan *Concept*, A#9823 menyebutkan kelas OAuth2PasswordBearer (freq=203) dan A#9412 menyebutkan HTTPException (freq=875), keduanya diekstraksi melalui NER berbasis LLM dari teks *body* jawaban. Kedua konsep ini dihubungkan dengan relasi *isRelatedTo* ($w=0,52$) karena sering muncul bersama dalam implementasi autentikasi FastAPI. Bobot kepercayaan pada setiap sisi merepresentasikan hasil validasi komunitas Stack Overflow dan menjadi penentu prioritas jalur traversal pada tahap *hybrid retrieval*.



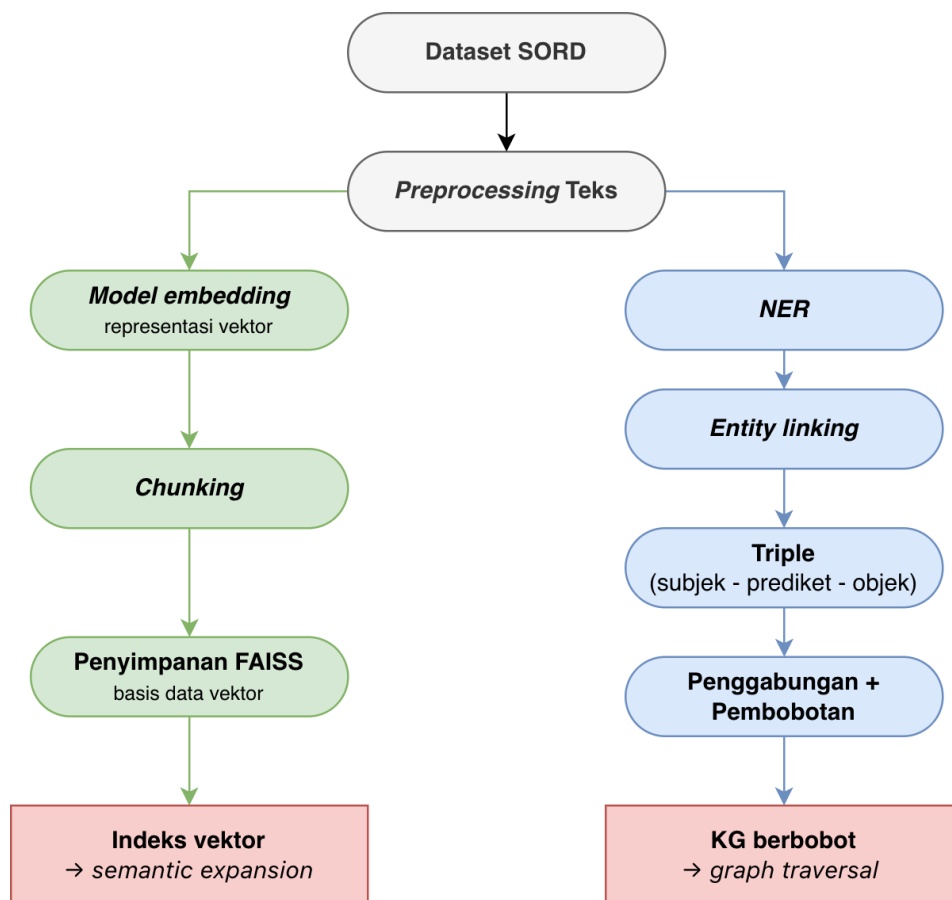
Gambar III.3 Contoh subgraf *knowledge graph* berbobot dari SORD pada domain autentikasi FastAPI

III.4.2.3 Proses Konstruksi KG

Gambar III.4 menunjukkan alur pembangunan *knowledge graph* (KG) berbobot dari dataset SORD (Fatima dan Maqbool 2026). Proses dimulai dari satu titik masukan bersama, yaitu tahap *preprocessing* teks, yang kemudian bercabang menjadi dua jalur paralel yaitu untuk membangun KG berbobot dan untuk membangun indeks vektor. Kedua jalur dijalankan secara bersamaan dan menghasilkan dua komponen output yang berbeda namun saling melengkapi.

Teks yang telah diproses pada tahap *preprocessing* diteruskan ke proses *Named Entity Recognition* (NER) untuk mengidentifikasi entitas teknis seperti nama *library*, jenis *error*, dan metode API. Hasil NER kemudian diproses melalui *entity linking* yang memetakan setiap entitas ke simpul yang tepat dalam skema KG, sekaligus menyelesaikan ambiguitas terminologi teknis yang umum terjadi pada konten Stack Overflow. Keluaran dari *entity linking* berupa himpunan triple dalam format (subjek - predikat - objek), misalnya (A#9823, mentions, OAuth2PasswordBearer). Triple-triple tersebut kemudian melewati dua langkah lanjutan secara berurutan. Pertama, penggabungan triple, yaitu proses konsolidasi seluruh triple dari berbagai *post* menjadi satu struktur graf tunggal disertai resolusi duplikasi. Kedua, pembobotan relasi, yaitu penetapan nilai w pada setiap sisi berdasarkan sinyal kepercayaan komunitas Stack Overflow, yang meliputi status *accepted answer*, jumlah *upvote*, dan reputasi kontributor. Ha-

sil akhir jalur ini adalah KG berbobot yang disimpan di Neo4j dan digunakan pada tahap *graph traversal* saat inferensi.



Gambar III.4 Flowchart konstruksi *knowledge graph* berbobot dari dataset SORD.

Secara paralel, teks hasil *preprocessing* juga diteruskan ke model *embedding* yang mengubah setiap *post* menjadi representasi vektor padat (*dense vector*). Vektor yang dihasilkan kemudian melewati proses *chunking*, yaitu pemecahan teks menjadi potongan-potongan berukuran lebih kecil disertai metadata yang relevan seperti *postId*, *score*, *isAccepted*, dan *tag*. Penyertaan metadata ini memungkinkan sistem menyaring dan mengurutkan hasil pencarian berdasarkan sinyal kepercayaan komunitas, bahkan pada tahap pencarian berbasis vektor. Seluruh *chunk* beserta metadata-nya kemudian disimpan dalam basis data vektor FAISS. Hasil akhir jalur ini adalah indeks vektor yang digunakan pada tahap *semantic expansion* saat inferensi.

Dengan demikian, proses konstruksi KG menghasilkan dua komponen yang bekerja secara komplementer yaitu KG berbobot di Neo4j yang menyediakan pengetahuan relasional terstruktur, dan indeks vektor di FAISS yang menyediakan pencarian se-

mantik berbasis kemiripan. Keduanya digunakan bersama-sama pada mekanisme *hybrid retrieval* yang dijelaskan pada subbagian berikutnya.

III.4.3 Desain Mekanisme *Hybrid Retrieval*

Mekanisme *hybrid retrieval* merupakan implementasi Blok 2 dalam *GraphRAG Framework*. Komponen ini dirancang untuk menggabungkan keunggulan pencarian berbasis *knowledge graph* dan pencarian berbasis representasi vektor. Pencarian berbasis graf memiliki kemampuan yang baik dalam menemukan hubungan antarentitas secara eksplisit, namun cakupannya terbatas pada informasi yang telah terhubung dalam graf. Sebaliknya, pencarian berbasis vektor mampu menemukan informasi yang relevan secara semantik, tetapi kurang efektif dalam menangani pertanyaan yang memerlukan hubungan relasional yang kompleks. Oleh karena itu, penelitian ini menggabungkan kedua pendekatan tersebut melalui tiga tahap yang dilakukan secara berurutan.

III.4.3.1 *Entity Anchoring*

Tahap pertama bertujuan untuk mengidentifikasi entitas teknis yang terdapat pada pertanyaan pengguna. Proses ini dilakukan menggunakan metode *Named Entity Recognition* (NER) yang dilanjutkan dengan *entity linking* untuk memetakan setiap entitas ke simpul yang sesuai pada *knowledge graph* SORD (Fatima dan Maqbool 2026). Sebagai contoh, pada kueri "*How to implement OAuth2 login with FastAPI?*", NER mengekstraksi entitas teknis FastAPI, OAuth2, dan login, kemudian *entity linking* memetakan entitas-entitas tersebut ke simpul Q#1234, Q#2567, dan OAuth2PasswordBearer di dalam KG SORD. Tahap ini penting karena istilah teknis pada Stack Overflow sering kali memiliki makna yang bergantung pada konteks (nama fungsi, metode, atau paket tertentu) dapat digunakan pada berbagai teknologi yang berbeda. Hasil dari tahap ini berupa sekumpulan simpul awal (*seed nodes*) yang akan digunakan sebagai titik awal pencarian pada tahap berikutnya.

III.4.3.2 *Graph Traversal*

Setelah memperoleh *seed nodes*, sistem melakukan penelusuran graf (*graph traversal*) untuk menemukan informasi yang berkaitan dengan entitas tersebut. Penelusuran dilakukan secara *multi-hop* dengan mengikuti relasi yang memiliki bobot kepercayaan paling tinggi. Bobot relasi ditentukan berdasarkan sinyal kepercayaan komunitas Stack Overflow, seperti status *accepted answer*, jumlah *upvote*, dan reputasi kontributor. Sebagai ilustrasi, dari simpul Q#1234 sistem mengikuti relasi *hasAcceptedAnswer* dengan bobot $w=1,0$ menuju A#9823, kemudian dari A#9823 mengikuti relasi *mentions* menuju OAuth2PasswordBearer. Secara paralel, sis-

tem juga menelusuri relasi *isRelatedTo* dari Q#1234 menuju Q#2567. Melalui proses ini, sistem dapat menemukan rantai hubungan antar konsep yang relevan serta memperoleh subgraf yang berisi informasi yang telah divalidasi oleh komunitas.

III.4.3.3 *Semantic Expansion*

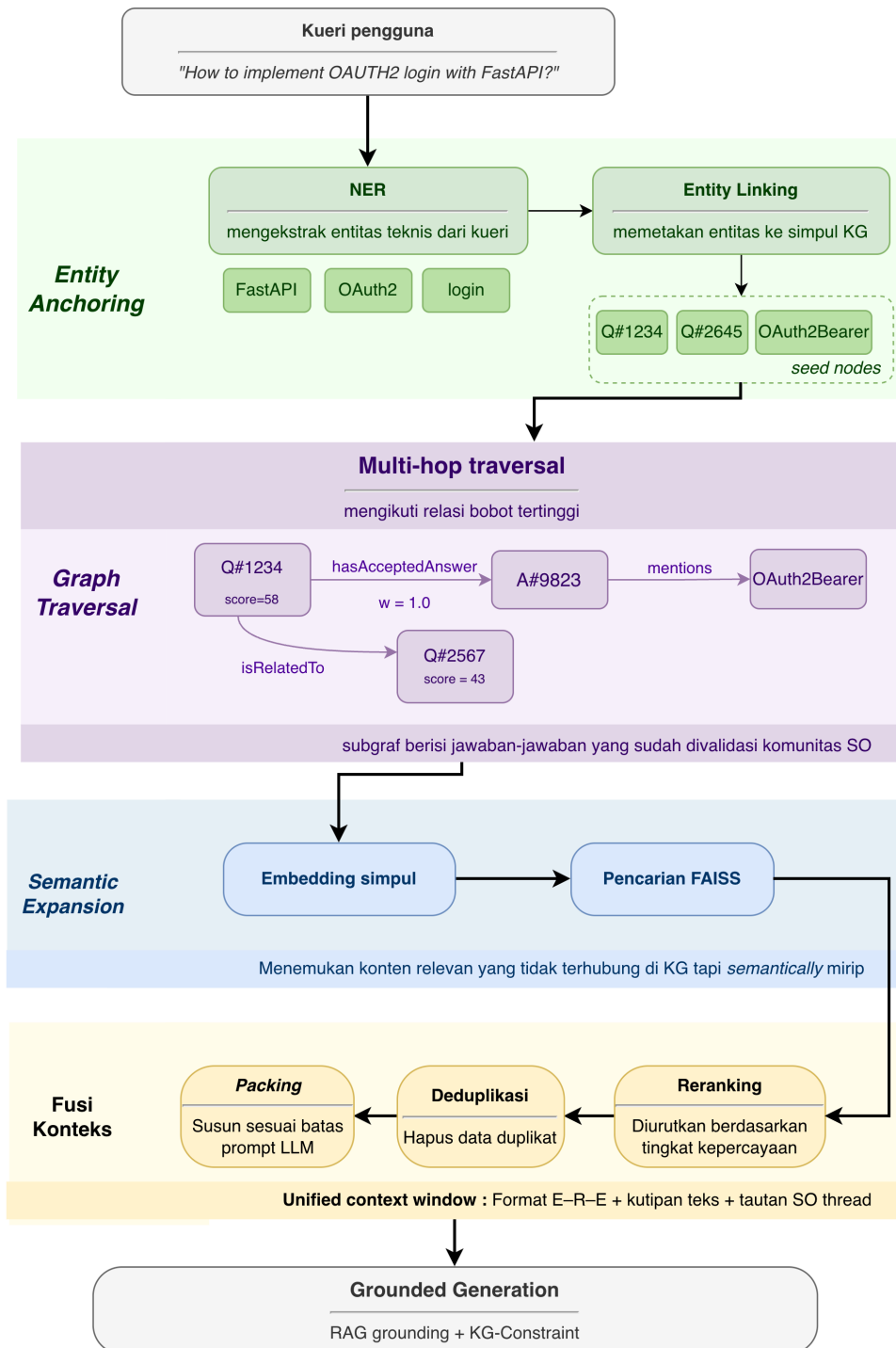
Subgraf yang diperoleh dari tahap sebelumnya kemudian digunakan untuk memperluas pencarian secara semantik. Simpul-simpul pada subgraf diubah menjadi representasi vektor menggunakan model *embedding* dan digunakan sebagai acuan pencarian pada *vector store* yang disimpan di FAISS. Pendekatan ini memungkinkan sistem menemukan informasi yang relevan meskipun tidak memiliki hubungan langsung dalam *knowledge graph*. Dengan demikian, cakupan informasi dapat diperluas tanpa menghilangkan konteks relasional yang telah diperoleh dari proses *graph traversal*. Berbeda dengan pendekatan RAG konvensional yang melakukan pencarian langsung dari kueri pengguna, tahap ini memanfaatkan simpul-simpul hasil penelusuran graf sebagai dasar pencarian semantik. Hasil dari tahap ini adalah kumpulan informasi yang relevan baik secara relasional maupun semantik, yang kemudian akan digabungkan pada tahap berikutnya untuk membentuk konteks yang akan digunakan dalam pembangkitan jawaban.

III.4.3.4 *Fusi Konteks Terverifikasi*

Sebelum digunakan oleh LLM, seluruh informasi hasil pencarian digabungkan menjadi satu konteks terintegrasi. Proses ini terdiri atas tiga langkah berurutan, yaitu *re-ranking*, deduplikasi, dan *packing*. Tahap *reranking* mengurutkan kembali seluruh hasil pencarian berdasarkan tingkat kepercayaan yang diperoleh dari sinyal komunitas Stack Overflow. Selanjutnya, deduplikasi dilakukan untuk menghilangkan informasi yang berulang dari kedua jalur pencarian. Terakhir, proses *packing* menyusun seluruh informasi ke dalam *unified context window* sesuai dengan batas panjang *prompt* yang dapat diproses oleh model.

Konteks yang dihasilkan menggunakan format Entitas-Relasi-Entitas (E-R-E) dan dilengkapi dengan kutipan teks serta tautan ke *thread* Stack Overflow asal. Informasi tersebut berfungsi sebagai jalur *provenance* yang memungkinkan sumber informasi ditelusuri kembali pada tahap pembangkitan jawaban.

Gambar III.5 mengilustrasikan alur pemrosesan kueri secara lengkap, mencakup tiga tahap *hybrid retrieval* dan *fusi konteks terverifikasi* yang menghasilkan *unified context window* untuk tahap *Grounded Generation*.



Gambar III.5 Alur mekanisme *hybrid retrieval* pada *GraphRAG Framework*.

III.4.4 Desain *Grounded Generation*

Grounded generation merupakan implementasi Blok 3 dalam *GraphRAG Framework*. Tahap ini bertujuan menghasilkan jawaban yang berlandaskan pada pengetahuan yang telah diperoleh melalui proses *hybrid retrieval*. Oleh karena itu, LLM tidak hanya mengandalkan *parametric knowledge* yang tersimpan dalam model, tetapi juga menggunakan konteks yang berasal dari komunitas Stack Overflow dan telah direpresentasikan dalam KG SORD. Masukan utama pada tahap ini adalah *unified context window*, yaitu kumpulan konteks yang menggabungkan hasil *text retrieval* dan *graph retrieval*. Seluruh informasi tersebut diberikan kepada LLM sebagai dasar untuk menyusun jawaban.

III.4.4.1 Mekanisme *Dual-Constraint*

Untuk memastikan jawaban yang dihasilkan tetap *grounded*, penelitian ini menerapkan mekanisme *dual-constraint* (Khalifa, Messaoud, dan Mkaouer 2026). Mekanisme ini terdiri atas dua lapisan kendali yang saling melengkapi. Lapisan pertama membatasi jawaban agar tetap sesuai dengan konteks yang diberikan, sedangkan lapisan kedua membatasi jawaban agar tetap konsisten dengan fakta yang tersimpan dalam *knowledge graph*.

Lapisan pertama adalah *RAG Grounding*. Pada lapisan ini, LLM diarahkan untuk melakukan *context-faithful generation*, yaitu menghasilkan jawaban yang berlandaskan pada informasi yang tersedia di dalam *unified context window*. Dengan pendekatan ini, setiap klaim dalam jawaban diharapkan memiliki dukungan dari konteks yang diberikan. Lapisan ini membantu mengurangi *faithfulness hallucination*, yaitu kondisi ketika model menghasilkan informasi yang tidak sesuai dengan konteks yang digunakan (Huang dkk. 2025).

Lapisan kedua adalah *KG-Constraint*. Pada lapisan ini, LLM diarahkan untuk mematuhi fakta-fakta yang direpresentasikan dalam *knowledge graph*. Informasi hasil *graph retrieval* direpresentasikan dalam bentuk tripel (Entitas-Relasi-Entitas) beserta segmen teks pendukungnya, kemudian dimasukkan ke dalam *prompt*. Kehadiran fakta terstruktur tersebut membantu mengurangi *factual hallucination*, yaitu munculnya fakta yang tidak didukung oleh sumber pengetahuan yang digunakan (Huang dkk. 2025).

Sebagai contoh, pada domain autentikasi FastAPI, relasi (A#9823, mentions, OAuth2PasswordBearer) yang diperoleh dari KG digunakan sebagai batasan faktual dalam proses generasi jawaban. Dengan demikian, model tetap mengacu pada solusi yang telah divalidasi

oleh komunitas Stack Overflow.

Kombinasi *RAG Grounding* dan *KG-Constraint* memungkinkan sistem menghasilkan jawaban yang relevan terhadap pertanyaan, didukung oleh konteks yang tersedia, serta konsisten dengan pengetahuan terstruktur yang terdapat dalam KG tanpa memerlukan pelatihan ulang model (Khalifa, Messaoud, dan Mkaouer 2026).

III.4.4.2 *Provenance* sebagai Mekanisme Keterlacakan

Selain menghasilkan jawaban, sistem juga menyediakan informasi *provenance* untuk menunjukkan asal-usul pengetahuan yang digunakan dalam proses generasi. Melalui mekanisme ini, setiap jawaban dapat ditelusuri kembali ke simpul dan relasi pada *knowledge graph* yang menjadi sumber konteksnya (Ali dkk. 2026).

Pada penelitian ini, *provenance* mencakup dua jenis informasi. Pertama, asal informasi berupa *thread* Stack Overflow yang digunakan sebagai referensi. Kedua, tingkat kepercayaan sumber yang diperoleh dari sinyal komunitas, seperti skor *upvote*, status *accepted answer*, dan reputasi kontributor. Dengan adanya *provenance*, pengguna tidak hanya mengetahui dari mana jawaban berasal, tetapi juga dapat menilai tingkat kepercayaan sumber yang mendasarinya.

III.4.4.3 Output Sistem

Output akhir *GraphRAG Framework* terdiri atas tiga komponen utama.

1. Jawaban yang dihasilkan LLM berdasarkan konteks hasil *hybrid retrieval*.
2. Referensi ke *thread* Stack Overflow yang menjadi sumber informasi sehingga pengguna dapat melakukan verifikasi secara langsung.
3. Informasi *provenance* yang menunjukkan simpul dan relasi pada *knowledge graph* yang berkontribusi terhadap pembentukan jawaban beserta bobot kepercayaannya.

Sebagai ilustrasi, untuk kueri “*How to implement OAuth2 login with FastAPI?*”, sistem menghasilkan jawaban mengenai penggunaan `OAuth2PasswordBearer` dan `Depends()`. Jawaban tersebut disertai referensi ke A#9823 serta jalur *provenance* Q#1234 → A#9823 → `OAuth2PasswordBearer`. Melalui informasi tersebut, pengguna dapat menelusuri sumber pengetahuan yang digunakan sekaligus memahami tingkat kepercayaan yang dimiliki sumber tersebut berdasarkan validasi komunitas.

III.4.5 Desain Evaluasi

Evaluasi dilakukan untuk mengukur kinerja *GraphRAG Framework* yang diusulkan serta menguji hipotesis penelitian yang telah dirumuskan pada Bab I. Untuk memastikan perbandingan yang adil, seluruh eksperimen menggunakan protokol yang

sama dan hanya berbeda pada mekanisme pencarian informasi (*retrieval*) dan *grounding* yang digunakan. Dengan demikian, perbedaan hasil yang diperoleh dapat dikaitkan secara langsung dengan kontribusi metode yang diusulkan.

Tiga kondisi pengujian digunakan dalam penelitian ini, yaitu:

- Kondisi A – LLM Murni (*Baseline*) Pada kondisi ini, model LLM menjawab pertanyaan pengguna secara langsung berdasarkan pengetahuan yang telah dipelajari selama proses pelatihan (*parametric knowledge*). Tidak digunakan sumber informasi eksternal maupun mekanisme *retrieval*.
- Kondisi B – RAG Konvensional Pada kondisi ini, sistem menggunakan pendekatan *Retrieval-Augmented Generation* (RAG) berbasis *dense retrieval*. Pencarian konteks dilakukan melalui basis data vektor tanpa memanfaatkan *knowledge graph*, sinyal kepercayaan komunitas, maupun mekanisme *dual-constraint*.
- Kondisi C – *GraphRAG Framework* Kondisi ini merupakan pendekatan yang diusulkan dalam penelitian. Sistem memanfaatkan *knowledge graph* berbobot, mekanisme *hybrid retrieval* yang menggabungkan penelusuran graf dan pencarian vektor, serta mekanisme *dual-constraint* yang terdiri atas *RAG Grounding* dan *KG-Constraint*.

Tabel III.7 Metrik evaluasi komparatif *GraphRAG Framework*.

Metrik	Alasan Pemilihan	Kondisi A/B/C
<i>Cosine Similarity</i>	Metrik utama Silva, Samhi, dan Khomh (2025); memastikan komparabilitas <i>apple-to-apple</i>	Semua
ROUGE-1/2/L	Kemiripan berbasis <i>n-gram</i> dengan jawaban <i>ground truth</i>	Semua
BERTScore	Kemiripan semantik berbasis <i>embedding</i> kontekstual	Semua
<i>Hallucination Rate</i>	Persentase klaim yang tidak terdukung dalam jawaban (dinilai LLM-as-Judge (Gu, Jiang, Shi, dkk. 2026))	Semua
<i>Attribution Coverage</i>	Persentase jawaban yang mencantumkan ≥ 1 kutipan SO thread	Semua (C sebagai target 100%)
<i>LLM-as-Judge Score</i>	Penilaian kualitas holistik: <i>faithfulness, comprehensiveness, relevance</i>	Semua
Uji Statistik	<i>Paired t-test</i> ($\alpha = 0,05$) untuk signifikansi perbedaan antar kondisi	A vs C, B vs C

Untuk membandingkan ketiga kondisi tersebut, digunakan prosedur evaluasi seba-

gai berikut.

1. Dataset Evaluasi.

Evaluasi menggunakan subset pertanyaan yang bersumber dari SORD (Fatima dan Maqbool 2026). Sebagai *ground truth*, digunakan jawaban yang berstatus *accepted answer* pada Stack Overflow, yang konsisten dengan protokol evaluasi Silva, Samhi, dan Khomh (2025) melalui repositori chat-stack (zenodo.15086541). Pemilihan sumber data ini memungkinkan hasil penelitian dibandingkan secara langsung dengan penelitian sebelumnya yang menggunakan konten Stack Overflow dari periode yang sama.

2. Kontrol Variabel LLM.

Ketiga kondisi menggunakan model LLM yang sama. Dengan demikian, pengaruh perbedaan kemampuan model dapat dieliminasi sehingga evaluasi berfokus pada efektivitas mekanisme *retrieval*, *grounding*, dan pemanfaatan *knowledge graph*.

3. Metrik Evaluasi.

Kinerja sistem dievaluasi menggunakan sejumlah metrik yang mengukur kualitas jawaban, kesesuaian dengan sumber referensi, serta tingkat kepercayaan hasil yang diberikan. Rincian metrik yang digunakan beserta alasan pemilihannya disajikan pada Tabel III.7.

III.5 Ringkasan Desain

Tabel III.8 merangkum komponen teknis *GraphRAG Framework* beserta kaitannya dengan kesenjangan yang dijawab dan kebutuhan yang dipenuhi.

Secara keseluruhan, desain yang diusulkan telah dirancang untuk memenuhi seluruh kebutuhan fungsional (F1–F7) dan kebutuhan nonfungsional (NF1–NF6) yang telah diidentifikasi pada tahap analisis. Selain itu, rancangan ini juga menjaga keterkaitan antara kesenjangan penelitian yang dibahas pada Bab II, hipotesis penelitian yang dirumuskan pada Bab I, dan solusi teknis yang diusulkan dalam penelitian ini.

Bab ini berfokus pada perancangan arsitektur dan komponen utama sistem. Implementasi dari setiap komponen, proses pengembangan sistem, serta hasil evaluasi terhadap pendekatan yang diusulkan akan dijelaskan pada bab-bab berikutnya.

Tabel III.8 Ringkasan komponen *GraphRAG Framework* dan kaitannya dengan kesenjangan penelitian

No.	Komponen	Fungsi	Gap	Teknologi Kandidat
1	Data Loader	Memuat dan memarsing dataset SORD	G2, G3	Python pandas / JSON
2	Preprocessor	Pembersihan, <i>deduplication</i> , dan tokenisasi	G2	spaCy / NLTK
3	KG Builder (NER)	Ekstraksi triple (<i>s, p, o</i>) dari teks	G3	GPT-4 API + LangChain
4	Weight Calculator	Komputasi bobot kepercayaan SO pada setiap sisi graf	G2	Python (normalisasi numerik)
5	Graph Database	Penyimpanan dan proses traversal <i>knowledge graph</i> berbobot	G2, G3	Neo4j / NetworkX
6	Embedding Model	Pengkodean teks untuk pencarian berbasis vektor	—	text-embedding-3-small
7	Vector Store	Indeks dan pencarian top- <i>k</i>	—	FAISS / ChromaDB
8	Retrieval Engine	<i>Graph-guided retrieval</i> dan pencarian berbasis vektor	G2, G3	Custom (Python)
9	LLM Generator	<i>Grounded generation</i> dan mekanisme <i>citation</i>	G1	GPT-4o (API OpenAI)
10	Evaluator	Komputasi metrik evaluasi komparatif	G1–G3	ROUGE, BERTScore, DeepEval

DAFTAR PUSTAKA

- Ali, Zafar, Yi Huang, Asad Khan, Guilin Qi, Yuxin Zhang, Junlan Feng, Chao Deng, dan Pavlos Kefalas. 2026. "Pythia-RAG: Retrieval-Augmented Generation over a Unified Multimodal Knowledge Graph for Enhanced Question Answering". *Knowledge-Based Systems* 335:115200. <https://doi.org/10.1016/j.knosys.2025.115200>.
- Alomari, Nawaf, Moussa Redah, Ahmad Ashraf, dan Mohammad Alshayeb. 2026. "Using LLMs to Enhance Code Quality: A Systematic Literature Review". *Information and Software Technology* 190:107960. <https://doi.org/10.1016/j.infsof.2025.107960>.
- Awotunde, Ifeoluwa, Dharmendra Reddy Chitte, Yongwei Shan, Weihua Sheng, dan Hossein Khaleghian. 2026. "Domain-Specific SQL Generation with LLMs: A Hybrid Framework Combining Knowledge Graphs and Retrieval-Augmentation". *Advanced Engineering Informatics* 73:104560. <https://doi.org/10.1016/j.aei.2026.104560>.
- Chang, Zongyu, Feihong Lu, Ziqin Zhu, Qian Li, Cheng Ji, dkk. 2026. "A Survey of LLM Alignment: Instruction Understanding, Intention Reasoning, and Reliable Dialogue Generation". *Neurocomputing* 687:133629. <https://doi.org/10.1016/j.neucom.2025.133629>.
- Cong, Yangfan, Suihuai Yu, Jianjie Chu, Pavan Tejaswi Velivela, Pengchao Wang, Yaoyao Fiona Zhao, dan Stephen Jia Wang. 2026. "A Retrieval-Augmented Method for Explainable Product Ideation: Unifying Conceptual Design Knowledge Graph and Large Language Models". *Advanced Engineering Informatics* 74:104770. <https://doi.org/10.1016/j.aei.2026.104770>.
- Cruz Cabello, Miguel De la, Tiago Prince Sales, dan Marcos R. Machado. 2025. "AIOps for Log Anomaly Detection in the Era of LLMs: A Systematic Literature Review". *Intelligent Systems with Applications* 28:200608. <https://doi.org/10.1016/j.iswa.2025.200608>.
- Cui, Jinman, dan Xiaolong Qu. 2026. "Modality-aware instruction tuning of large language models for multimodal knowledge graph completion". *Information Fusion* 134:104387. <https://doi.org/10.1016/j.inffus.2026.104387>.

- Fatima, Arjumand, dan Onaiza Maqbool. 2026. “The Stack Overflow Recommendations Dataset (SORD) – A Large-Scale Curated Dataset of Recommendations Related Stack Overflow Questions, Answers and Comments”. In Press, Journal Pre-proof, *Data in Brief*, 112818. <https://doi.org/10.1016/j.dib.2026.112818>. <https://doi.org/10.1016/j.dib.2026.112818>.
- Gu, Jiawei, Xuhui Jiang, Zhichao Shi, dkk. 2026. “A Survey on LLM-as-a-Judge”. *The Innovation* 7:101253. <https://doi.org/10.1016/j.xinn.2025.101253>.
- Haque, Md Asraful. 2025. “LLMs: A Game-Changer for Software Engineers?” *Benchmark Council Transactions on Benchmarks, Standards and Evaluations* 5 (1): 100204. <https://doi.org/10.1016/j.tbench.2025.100204>.
- Hendawi, Samar, Tarek Kanan, Mohammed Elbes, Ala Mughaid, dan Shadi Alzu’bi. 2026. “Automated Prompt Engineering Pipelines: Fine-Tuning LLMs for Enhanced Response Accuracy”. *Expert Systems with Applications* 298:129689. <https://doi.org/10.1016/j.eswa.2026.129689>.
- Hogan, Aidan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, dkk. 2022. “Knowledge Graphs”. *ACM Computing Surveys* 54 (4): 1–37. <https://doi.org/10.1145/3447772>.
- Huang, Lei, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, dkk. 2025. “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”. *ACM Transactions on Information Systems* 43 (2): 42:1–42:55. <https://doi.org/10.1145/3703155>.
- Huang, Yizheng, dan Jimmy Xiangji Huang. 2026. “A Survey on Retrieval-Augmented Text Generation for Large Language Models”. *ACM Computing Surveys* 58 (12): 300:1–300:38. <https://doi.org/10.1145/3805774>.
- Ji, Ziwei, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, dkk. 2023. “Survey of Hallucination in Natural Language Generation”. *ACM Computing Surveys* 55 (12): 1–38. <https://doi.org/10.1145/3571730>.
- Jiang, Bowen, Mitchell Ruffolo, Aaditya Kulkarni, Teia Canini, dan Eman Abdullah AlOmar. 2026. *A systematic mapping study on the research landscape of LLM-based code clone detection*. <https://doi.org/10.1016/j.infsof.2026.108096>.

- Kabir, Samia, David N. Udo-Imeh, Bonan Kou, dan Tianyi Zhang. 2024. "Is Stack Overflow Obsolete? An Empirical Study of ChatGPT Answers to Stack Overflow Questions". Dalam *CHI Conference on Human Factors in Computing Systems*, 1–17. <https://doi.org/10.1145/3613904.3642596>.
- Kamalipour, Amir Abbas, Shahrokh Asadi, dan Mohammad Mahyar Amiri Chimeh. 2026. "From Vectors to Knowledge Graphs: A Comprehensive Analysis of Modern Retrieval-Augmented Generation Architectures". *Computer Science Review* 61:100925. <https://doi.org/10.1016/j.cosrev.2026.100925>.
- Khalifa, Ilyes Ben, Montassar Ben Messaoud, dan Mohamed Wiem Mkaouer. 2026. "LARK: License Analysis with Retrieval-Augmented Generation and Knowledge Graphs". *Information and Software Technology* 197:108164. <https://doi.org/10.1016/j.infsof.2026.108164>.
- Kumar, Pranjal. 2024. "Large Language Models (LLMs): Survey, Technical Frameworks, and Future Challenges". *Artificial Intelligence Review* 57. <https://doi.org/10.1007/s10462-024-10888-y>.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, dkk. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". Dalam *Advances in Neural Information Processing Systems*, 33:9459–9474. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- Li, Ruihao, Chong Chen, Ying Liu, Tao Wang, Haidong Shao, dan Lianglun Cheng. 2026. "A Collaborative Approach Based on Large Language Model and Knowledge Graphs for Information Integration Towards Smart Manufacturing". *Engineering Applications of Artificial Intelligence* 176:114785. <https://doi.org/10.1016/j.engappai.2026.114785>.
- Malamas, Nikolaos, Emmanouil Tsardoulis, Konstantinos Panayiotou, dan Andreas L. Symeonidis. 2025. "Toward Efficient Vibe Coding: An LLM-Based Agent for Low-Code Software Development". *Journal of Computer Languages* 85:101367. <https://doi.org/10.1016/j.cola.2025.101367>.
- Moia, Vitor Hugo Galhardo, Igor Jochem Sanz, Gabriel Antonio Fontes Rebello, Rodrigo Duarte de Meneses, Briland Hitaj, dan Ulf Lindqvist. 2026. "LLM in the Middle: A Systematic Review of Threats and Mitigations to Real-World LLM-Based Systems". *Computer Science Review* 61:100916. <https://doi.org/10.1016/j.cosrev.2026.100916>.

- Nashid, Noor, Mifta Sintaha, dan Ali Mesbah. 2023. “Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning”. Dalam *International Conference on Software Engineering*, 2450–2462. <https://doi.org/10.1109/ICSE48619.2023.00205>.
- Oarga, Alexandru, Matthew Hart, Andres M. Bran, Magdalena Lederbauer, dan Philippe Schwaller. 2026. “Scientific knowledge graph and ontology generation using open large language models”. *Digital Discovery* 5:1269–1279. <https://doi.org/10.1039/d5dd00275c>.
- Pan, Shirui, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, dan Xindong Wu. 2024. “Unifying Large Language Models and Knowledge Graphs: A Roadmap”. *IEEE Transactions on Knowledge and Data Engineering* 36 (7): 3580–3599. <https://doi.org/10.1109/TKDE.2024.3352100>.
- Peffer, Ken, Tuure Tuunanen, Marcus A. Rothenberger, dan Samir Chatterjee. 2007. “A Design Science Research Methodology for Information Systems Research”. *Journal of Management Information Systems* 24 (3): 45–77. <https://doi.org/10.2753/MIS0742-1222240302>.
- Ronanki, Krishna, Beatriz Cabrero-Daniel, Tomas Herda, Stefan Sitkovich, Jennifer Horkoff, dan Christian Berger. 2026. “Recommendations for Efficient and Responsible LLM Adoption within Industrial Software Development”. *Information and Software Technology* 196:108171. <https://doi.org/10.1016/j.infsof.2026.108171>.
- Silva, Leuson Da, Jordan Samhi, dan Foutse Khomh. 2025. “LLMs and Stack Overflow Discussions: Reliability, Impact, and Challenges”. *Journal of Systems and Software* 230:112541. <https://doi.org/10.1016/j.jss.2025.112541>.
- Tao, Ye, Yonghao Liu, Xiugong Qin, Hui Li, dan Xiaoni Wang. 2026. “Semantic Perception and Response Generation for Home Scene Based on Knowledge Graphs and Large Language Models”. *Neurocomputing* 695:134003. <https://doi.org/10.1016/j.neucom.2026.134003>.
- Vizniuk, Artem, Grygorii Diachenko, Ivan Laktionov, Agnieszka Siwocha, Min Xiao, dan Jacek Smoląg. 2024. “A Comprehensive Survey of Retrieval-Augmented Large Language Models for Decision Making in Agriculture: Unsolved Problems and Research Opportunities”. *Journal of Artificial Intelligence and Soft Computing Research* 15 (2): 115–146. ISSN: 24496499. <https://doi.org/10.2478/jaiscr-2025-0007>.

- Wang, Jing, Zhiwei Xu, Siyuan Liu, Ye Lu, dan Tao Li. 2026. “Towards Efficient Graph-RAG via Structure-Aware Intermediate Representation: Incremental Collaborative Exploration on Knowledge Graph”. *Knowledge-Based Systems* 342:115884. <https://doi.org/10.1016/j.knosys.2026.115884>.
- Zhou, Xingwei, Shijiang Li, Ying Liu, Wanran Yang, Chengyue Xiong, dan Liang Hou. 2026. “LLM-Based Domain-Specific Knowledge Graph Construction and Hybrid Retrieval-Augmented Generation”. *Journal of Manufacturing Systems* 86:1022–1038. <https://doi.org/10.1016/j.jmansys.2026.01.002>.
- Zhu, Zulun, Tiancheng Huang, Kai Wang, Junda Ye, Xinghe Chen, dan Siqiang Luo. 2026. “Graph-Based Approaches and Functionalities in Retrieval-Augmented Generation: A Comprehensive Survey”. *ACM Computing Surveys*, <https://doi.org/10.1145/3795880>.

